Verified Program Synthesis for Coinductive Reinforcement Learning

From Observations to Optimal Policies via Predicate DSL

Ricardo Corral-Corral

February 2026

Abstract

We present a verified program synthesis framework that extends Coinductive Symmetric Homomorphism Reinforcement Learning (CSHRL) from learned rankings to *synthesized programs*. Where CSHRL defines optimality as structure preservation between action rankings and infinite reward streams, we add a synthesis layer that *constructs* these rankings automatically from environment observations.

The framework introduces thirteen components, all machine-verified in Agda: **(1)** A *Predicate DSL* (`PredProg`)—a domain-specific language for Boolean predicates over environment features, parameterized by an abstract carrier type and feature set, enabling instantiation across different Environment Classes; **(2)** A *CEGIS* (Counterexample-Guided Inductive Synthesis) algorithm that enumerates predicate programs and refines a version space by observations, with proven monotonic convergence and strict shrinkage bounds; **(3)** A *Propagation Theorem* establishing that feature-equivalent carriers are indistinguishable to any predicate program, which yields three quantified consequences: identical refinement (Refine-Equiv), observation subsumption, and exploration dissolution—reducing the observations needed from $|C|$ (carrier count) to $|C/\sim|$ (equivalence classes); **(4)** *Exploration completeness*: after observing one representative per equivalence class, no further observation can refine the version space ("dissolution is all you need"); **(5)** *Temporal propagation*: when transitions preserve feature equivalence, observations propagate along entire trajectories, not just across carriers—mirroring CSHRL's coinductive unfolding; **(6)** *Feature sufficiency*: when the target function respects feature equivalence, every CEGIS survivor is provably correct on *all* carriers, not just the observed ones—upgrading stabilization to a full correctness guarantee; **(7)** *Multi-predicate propagation*: when synthesizing multiple predicates from the same feature space (e.g., `is-dead` and `is-solved`), observations are shared—one representative set suffices for all predicates, and cross-predicate entailment (e.g., dead → ¬solved) further reduces evaluations; **(8)** *Online synthesis with learning integration*: observation order is provably irrelevant (refinement commutes), replayed observations are provably redundant, and a generic `LearningBridge` connects learning feedback to synthesis with a bridge sufficiency theorem composing learning convergence with synthesis correctness; **(9)** *Sample complexity lower bound*: two targets agreeing on observed carriers produce identical version spaces (`same-obs-same-vs`), so survivors transfer freely between indistinguishable targets (`survivor-transfer`); combined with the upper bound (`sufficient-cegis`), this proves the sample complexity is *exactly* $|C/\sim|$; **(10)** *DSL completeness*: a constructive fingerprint construction proves that the Boolean `PredProg` DSL is expressively complete for feature-respecting targets—for any target constant on equivalence classes, there exists a `PredProg` matching it everywhere (`dsl-complete`); **(11)** *FOSD synthesis*: building on the FOSD theory developed in [24], we add a **learning convergence** theorem (`fosd-obs-sound`, `model-obs-consistent`) connecting FOSD observations to CEGIS version-space refinement—the synthesis engine uses distributional dominance rather than expected values, with unchanged `PredicateDSL` and propagation theorems; **(12)** *Stochastic Dominance Hierarchy*: the SD[k] theory (see [24] for definitions and proofs) enables fallback to SOSD or TOSD when FOSD fails, while retaining correctness guarantees for the corresponding utility-function class; **(13)** *Compositional Ranking Algebra*: the full proof infrastructure for composing independently-verified rankings—proving `prefix-sum` linearity, `sd-weight` distributivity, closure under

concatenation (`SD-++`), scaling (`SD-scale`), and **convolution** (`FOSD-conv`, via discrete Abel summation); with `VerifiedRanking` product, sum, and scaling composition enabling modular verification (see [24] for condensed definitions).

We demonstrate the full pipeline end-to-end on three Environment Classes and three continuous-control benchmarks: a Finite Deterministic MDP (1D Maze) where CEGIS synthesizes ranking predicates that yield a verified `CoindHomo` and optimal policy; a Combinatorial Placement MDP (*N*-Queens, Sudoku) where synthesized predicates *define* the environment's step function—`find-policy` solves 4-Queens, 8-Queens, and 4×4 Sudoku through the synthesized dynamics, certifying solvability and doom via the capability profile, and the same class scales to full 9×9 Sudoku with a machine-checked CoindHomo over a 9^{43}-leaf game tree; a Stochastic Finite MDP where synthesis operates on expected traces; and autonomous controllers for CartPole (500/500), MountainCar (1000/1000), and Acrobot (500/500, avg. 88 steps)—the last using a `DirectRewardMDP` EC whose reward is derived from the total energy of the Euler-integrated next state, computed from the Lagrangian ODE of the double pendulum entirely within Agda.

This document is itself the proof: compiling it with `agda --latex` type-checks all embedded code while producing this PDF. The synthesis framework builds on the CSHRL foundation [24] without modifying it—all results are additive. The core FOSD, SD hierarchy, and state abstraction theories are developed in [24]; this paper focuses on the synthesis pipeline and its applications. The combinatorial placement environment class and its trace-bridge theorem have additionally been ported to Rocq and Lean 4, where the Sudoku instances are re-verified end to end—including the full 43-step 9×9 policy rollout by compiled evaluation—so the synthesis substrate is certified by three independent proof-assistant kernels.

1 Introduction

CSHRL [1] establishes a new foundation for optimal sequential decision making: an agent is optimal when its ranking of actions forms a *coinductive symmetric homomorphism* into the ordering of infinite reward streams. Learning reduces to *restoring* the structural correspondence between rankings and outcomes via violation correction.

But how does a ranking come to exist in the first place?

In the original framework, rankings are either hand-crafted (verified manually) or discovered by the Environment Class `find-policy` (which exhaustively evaluates the state-action tree). Both approaches have limitations:

- **Hand-crafted rankings** require domain expertise and per-domain proofs.
- **Exhaustive evaluation** computes correct rankings but produces opaque lookup tables—the *structure* of why certain actions dominate others is lost.

Program synthesis offers a third path: *learn a program* that encodes the ranking as a function of observable features. The synthesized program is:

- **Interpretable:** a Boolean combination of feature tests, not a black-box table.
- **Generalizable:** feature-equivalent states automatically receive the same ranking.
- **Verifiable:** the program's correctness can be checked by Agda's type system.
- **Compositional:** the DSL is parameterized by carrier type and features, so each Environment Class provides its own feature vocabulary while sharing the synthesis theory.

1.1 Contributions

This paper makes the following contributions, each machine-verified in Agda with `--safe --guardedness`:

1. **Predicate DSL (§3):** A generic language `PredProg` for Boolean predicates over features, with a structural `FeatureEquiv` relation and the **Propagation Theorem**: feature-equivalent carriers evaluate identically under any predicate program.

2. **CEGIS with Convergence (§4):** A counterexample-guided synthesis loop with five proven properties: version-space monotonicity, correct-predicate survival, propagation acceleration, bounded convergence, and strict shrinkage for informative observations.

3. **Propagation Quantification (§5):** Three theorems that formalize how propagation reduces exploration:

   - Refine-Equiv: feature-equivalent carriers produce identical version-space refinement.
   - Observation-Subsumption: after observing at c_1, any feature-equivalent c_2 is provably redundant.
   - Exploration-Dissolution: an entire equivalence class is handled by a single observation at any representative; the corollary Dissolution-Strict bounds total informative observations by $|C/\sim|$.

4. **Quotient Structure (§6):** `AllFeatAgree` forms an equivalence relation, inducing a quotient $C/\sim$. The Exploration-Complete theorem proves that after observing one representative per class, no further observation can refine the version space—formalizing "dissolution is all you need."

5. **Temporal Propagation (§7):** When the transition function preserves feature equivalence, observations propagate along entire trajectories. Trajectory-Dissolution proves that one trajectory from c_1 subsumes all trajectories from equivalent c_2—extending spatial propagation to the temporal dimension and mirroring CSHRL's coinductive unfolding.

6. **Feature Sufficiency (§8):** The Feature Sufficiency Theorem (the "crown jewel"): when the target function respects feature equivalence, every CEGIS survivor is provably correct on *all* carriers—not just the observed ones. This upgrades exploration completeness from a stabilization guarantee to a full correctness guarantee.

7. **Multi-Predicate Propagation (§9):** When synthesizing multiple predicates from the same feature space, observations are shared: one representative set suffices for all predicates. Cross-predicate entailment (e.g., dead → ¬solved) gives free observations, and cross-dissolution combines both effects multiplicatively.

8. **Online Synthesis (§10):** Observation refinement is commutative (`refine-commutes`), making the CEGIS version space determined by the *set* of observations rather than their order. Replayed observations are provably redundant (`replay-redundant`). A generic `LearningBridge` module connects domain-specific learning feedback to synthesis observations, with a Bridge-Sufficient theorem composing learning convergence with synthesis correctness.

9. **Sample Complexity Lower Bound (§11):** Two targets agreeing on observed carriers produce identical version spaces (`same-obs-same-vs`); survivors transfer freely between indistinguishable targets (`survivor-transfer`). Combined with the upper bound (`sufficient-cegis`), this proves the sample complexity of CEGIS is *exactly* $|C/\sim|$: one representative per equivalence class is both necessary and sufficient.

10. **DSL Completeness (§12):** The Boolean `PredProg` DSL is expressively complete for feature-respecting targets. A constructive *fingerprint* construction shows that for any target constant on `AllFeatAgree` classes, there exists a `PredProg` matching it everywhere (`dsl-complete`). This closes the theoretical loop: the DSL always contains the correct answer, CEGIS finds it from $|C/\sim|$ observations, and no fewer suffice.

11. **EC-Specific Instantiation (§13):** Three concrete instantiations—for Finite Deterministic MDPs (ranking synthesis with `CoindHomo` construction), Combinatorial Placement MDPs (predicate synthesis that *defines* the step function), and Stochastic Finite MDPs (ranking synthesis from expected traces with `StochasticCoindHomo` construction, §13.3).

12. **Cross-Pair Propagation (§B):** A verified demonstration that CEGIS observations are pair-agnostic. When ranking pairs share the same feature structure, observing a sub-optimal pair (C vs D) determines top-tier rankings (A > B)—without ever comparing the actions of interest. Two observations from the bottom-tier pair determine all 6 rankings across all 6 states (18× savings). Verified learning curves (8 states, Agda `refl`) and scaled rollout performance curves (30 states, 22-program VS) demonstrate a **50% reward improvement** from propagation, with sub-optimal and top-pair observation sequences yielding **identical** rollout reward.

13. **End-to-End Demos (§14):** Three verified pipelines:

    - *1D Maze:* 5 observations → CEGIS → ranking predicates → preservation proof → `CoindHomo` → policy rollout → cross-validation with `find-policy`.
    - *4-Queens:* Synthesized `PredProg` predicates *are* the step function; `find-policy` solves 4-Queens; partial completion from any intermediate state.
    - *8-Queens:* Synthesized predicates verified equivalent to hand-crafted ones; full solution and partial completion in 8^8 search space.

14. **FOSD Synthesis (§15.6):** Building on the FOSD theory from [24], we add FOSD learning convergence (`fosd-obs-sound`, `model-obs-consistent`, `fosd-convergence`) connecting FOSD observations to CEGIS version-space refinement, and demonstrate full end-to-end FOSD synthesis on BiasedBandit, CoinFlip, and GamblersRuin.

15. **Stochastic Dominance Hierarchy (§15.7):** The SD[k] theory (developed in [24]) enables fallback to SOSD/TOSD when FOSD fails. This paper uses the hierarchy in the synthesis patterns (§15.7).

16. **Compositional Ranking Algebra (§15.8):** The full proof infrastructure for composing independently-verified rankings (definitions summarized in [24]; detailed proofs here). Key results: prefix-sum linearity, sd-weight distributivity, closure under concatenation, scaling, and convolution (via discrete Abel summation), with product/sum/scale composition of `VerifiedRanking`s.

17. **Verified State Abstraction (§15.9):** A general framework for lifting verified rankings from finite abstract systems to arbitrary concrete state spaces (core theorem in [24]). Key results:

    - StateAbstraction: a record bundling a projection, embedding, and section property—no finiteness assumption on the concrete type.
    - abstract-lift: the central theorem—verify on abstract states, automatically obtain a `VerifiedRanking` on the full concrete system, under marginal-invariance (same abstract class ⇒ same reward distributions).
    - Composition: product abstraction (component-wise), composed abstraction (two-level chains), and `abstract-conv-product` (abstraction + convolution + lifting in one pipeline).
    - AllFeatAgree Subsumption: feature-based abstraction is a special case where `project` = feature-vector.
    - Demonstrations: (i) the *Regional Policy* demo (`AbstractionDemo`); (ii) `ThreeZoneDemo`; (iii) `AbstractConvProductDemo`; (iv) `ComposeAbstractionDemo`; (v) `SimplePendulumDemo`; (vi) `CartPoleDemo`; (vii) `RealCartPoleDemo`; (viii) `DiscretizedCartPoleLearning`; (ix) `CartPoleContinuousLearning`; (x) `CartPolePhysicsLearning`—physics-motivated dynamics on 4 phase-space states, CEGIS discovers a 2-feature ranking (angle ∨ velocity), lifted to a verified policy on $\mathbb{Q}^4$.

1.2 Relationship to the CSHRL Paper

This work is *additive*: it imports the CSHRL core (`Core`, `CoindHomo`, `solve`, `value`, `action-value`) without modification. The synthesis framework sits alongside the existing Environment Class and Learning modules.

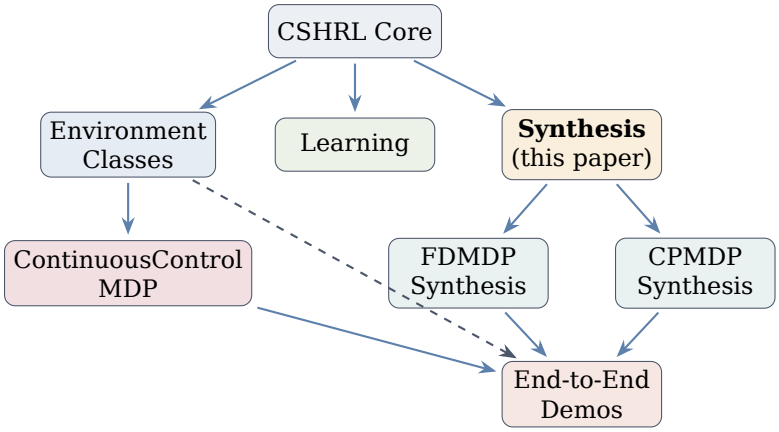

Figure 1: Architecture. The synthesis framework (yellow) builds on the CSHRL Core without modifying it. EC-specific modules instantiate the generic DSL for each domain. The `ContinuousControlMDP` EC (green) factors the fragility–FOSD–abstraction pipeline into a reusable module, instantiated for CartPole and MountainCar. Its generalisation `DirectRewardMDP` accepts rewards derived from physics (e.g. next-state total energy from Euler-integrated ODE), instantiated for Acrobot. End-to-end demos integrate synthesis with Environment Classes.

2 Background: CSHRL in Brief

We briefly recall the CSHRL framework [1] to establish notation. The reader is referred to the companion paper for full details.

Definition 1 (Coinductive Symmetric Homomorphism)**.** An agent's ranking $\leq_a$ of actions in state s is a *coinductive symmetric homomorphism* if:

$$a \leq_a b \implies \textit{action-value}(s, a) \leq_s \textit{action-value}(s, b)$$

where $\leq_s$ is coinductive pointwise dominance on infinite reward streams: $x \leq_s y \iff \textit{head}(x) \leq_r \textit{head}(y) \wedge \textit{tail}(x) \leq_s \textit{tail}(y)$.

In Agda, this is captured by the `CoindHomo` record:

```
record CoindHomo : ₁Set where
  field
    ≤ₐ__     : State → Action → Action → Set
    preserves : ∀ a b s → s ≤ₐ a b → action-value s a ≤ₛ action-value s b
```

An **Environment Class** (EC) is a parameterized module that, given domain-specific structure (states, actions, transition function), provides `trace-action`, `find-ranking`, and `find-policy` with automatic stream bridges. Two ECs are central to this paper:

- **FiniteDeterministicMDP**: parameterized by finite `State`, `Action`, and a deterministic `step : State → Action → State × Reward`.
- **CombinatorialPlacementMDP**: parameterized by a configuration type, placement action, and predicates `is-dead`, `is-solved` that determine when a configuration is terminal.
- **ContinuousControlMDP**: parameterized by a concrete state type, a finite grid abstraction, and a reward-level monotonicity proof. Given these, the EC automatically provides fragility analysis, FOSD synthesis, marginal-reward construction, an auto-derived ordering from fragility comparison, and a verified ranking lifted to the continuous state space via `abstract-lift`—reducing the per-environment boilerplate from $O(|S| \cdot |A|^2)$ proof cases to $O(L^2)$, where L is the number of distinct reward levels (~3–5). Both `CartPoleAutoController` and `MountainCarAutoController` are instantiations of this EC.

- **DirectRewardMDP**: a generalisation of `ContinuousControlMDP` that accepts an arbitrary reward function `cc-reward : GridState → Action → ℕ` instead of deriving one from fragility. This accommodates environments where the natural reward is derived from a physics quantity computed within Agda (e.g., total energy of the Euler-integrated next state) rather than graph distance. `AcrobotAutoController` is an instantiation.

The synthesis framework presented here provides a systematic way to *construct* the ranking ($\lesssim_a$) or the environment predicates (`is-dead`, `is-solved`) from observations, yielding verified `CoindHomo` instances or verified EC instantiations.

3 The Predicate DSL: Programs over Features

The core abstraction is a domain-specific language for Boolean predicates. The key design choice is *parametricity*: the DSL is parameterized by an abstract carrier type and feature set, enabling each Environment Class to provide its own feature vocabulary while sharing the synthesis theory.

```
{-# OPTIONS --safe --guardedness #-}

module CSHRLSynthesis where

open import Data.List using (List; []; _∷_; map; foldr; _++_; concatMap; length)
open import Data.Nat using (ℕ; zero; suc; _⊔_; _≤_; z≤n; s≤s)
open import Data.Nat.Properties using (≤-refl)
open import Data.Product using (_×_; _,_; proj₁; proj₂; ∃-syntax)
open import Relation.Binary.PropositionalEquality
  using (_≡_; refl; sym; trans; cong; cong₂; subst)
open import Data.Bool using (Bool; true; false; if_then_else_; _∧_; _∨_; not)
open import Data.Unit using (⊤; tt)
open import Data.Empty using (⊥; ⊥-elim)
open import Relation.Nullary using (Dec; yes; no; ¬_)
open import Data.Maybe using (Maybe; just; nothing)
```

The DSL is a parameterized module:

```
module PredicateDSL
  (Carrier : Set)
  (Feature : Set)
  (eval-feature : Feature → Carrier → Bool)
  where
```

`Carrier` is the type being classified (states, configurations, etc.), `Feature` is the vocabulary of observable properties, and `eval-feature` extracts a Boolean feature value from a carrier.

3.1 Syntax: Predicate Programs

A `PredProg` term is a Boolean combination of feature tests:

```
data PredProg : Set where
  feat  : Feature → PredProg
  _∧p_  : PredProg → PredProg → PredProg
  _∨p_  : PredProg → PredProg → PredProg
  ¬p_   : PredProg → PredProg
  truep : PredProg
  falsep : PredProg

infixr 6 _∧p_
infixr 5 _∨p_
```

The constructors mirror propositional logic: `feat f` tests a single feature, `_∧p_` and `_∨p_` combine subprograms, `¬p_` negates, and `truep`/`falsep` are constants.

3.2 Semantics: Evaluation

Evaluation maps a predicate program and a carrier to a Boolean:

```
eval : PredProg → Carrier → Bool
eval (feat f) c = eval-feature f c
eval (p₁ ∧p p₂) c = eval p₁ c ∧ eval p₂ c
eval (p₁ ∨p p₂) c = eval p₁ c ∨ eval p₂ c
eval (¬p p)  c = not (eval p c)
eval truep   _ = true
eval falsep  _ = false
```

Remark 1 (Expressiveness). At depth 0, `PredProg` contains atoms: `truep`, `falsep`, and `feat f` for each feature f. Each level of Boolean operators (depth $d+1$) exponentially expands the space. For n features, depth 0 gives $n+2$ candidates; depth 1 adds negations, conjunctions, and disjunctions, yielding $O(n^2)$ candidates. This is sufficient for many domains: the 4-Queens `is-dead` predicate is a single disjunction of 6 attack features (depth 1), and the 8-Queens version uses 28 features at the same depth.

3.3 Feature Equivalence

Two carriers are *feature-equivalent* with respect to a predicate program if they agree on every feature the program inspects:

```
FeatureEquiv : PredProg → Carrier → Carrier → Set
FeatureEquiv (feat f) c₁ c₂ = eval-feature f c₁ ≡ eval-feature f c₂
FeatureEquiv (p₁ ∧p p₂) c₁ c₂ = FeatureEquiv p₁ c₁ c₂ × FeatureEquiv p₂ c₁ c₂
FeatureEquiv (p₁ ∨p p₂) c₁ c₂ = FeatureEquiv p₁ c₁ c₂ × FeatureEquiv p₂ c₁ c₂
FeatureEquiv (¬p p)  c₁ c₂ = FeatureEquiv p c₁ c₂
FeatureEquiv truep   _ _ = ⊤
FeatureEquiv falsep  _ _ = ⊤
```

`FeatureEquiv` is defined *structurally* on the program, not on the carrier type. This means it captures exactly the features *relevant to this particular predicate*, not all possible features.

```
feat-equiv-refl : ∀ p c → FeatureEquiv p c c
feat-equiv-refl (feat f) c = refl
feat-equiv-refl (p₁ ∧p p₂) c = feat-equiv-refl p₁ c , feat-equiv-refl p₂ c
feat-equiv-refl (p₁ ∨p p₂) c = feat-equiv-refl p₁ c , feat-equiv-refl p₂ c
feat-equiv-refl (¬p p)  c = feat-equiv-refl p c
feat-equiv-refl truep   _ = tt
feat-equiv-refl falsep  _ = tt

feat-equiv-sym : ∀ p c₁ c₂ → FeatureEquiv p c₁ c₂ → FeatureEquiv p c₂ c₁
feat-equiv-sym (feat f) c₁ c₂ eq = sym eq
feat-equiv-sym (p₁ ∧p p₂) c₁ c₂ (e₁ , e₂) =
  feat-equiv-sym p₁ c₁ c₂ e₁ , feat-equiv-sym p₂ c₁ c₂ e₂
feat-equiv-sym (p₁ ∨p p₂) c₁ c₂ (e₁ , e₂) =
  feat-equiv-sym p₁ c₁ c₂ e₁ , feat-equiv-sym p₂ c₁ c₂ e₂
feat-equiv-sym (¬p p)  c₁ c₂ eq = feat-equiv-sym p c₁ c₂ eq
feat-equiv-sym truep   _ _ _ = tt
feat-equiv-sym falsep  _ _ _ = tt
```

3.4 The Propagation Theorem

The central result of the DSL: feature-equivalent carriers are *indistinguishable* to any predicate program.

Theorem 1 (Propagation)**.** *For any predicate program p and carriers c_1, c_2:*

$$\mathit{FeatureEquiv}\ p\ c_1\ c_2 \implies \mathit{eval}\ p\ c_1 \equiv \mathit{eval}\ p\ c_2$$

```
propagation : ∀ (p : PredProg) (c₁ c₂ : Carrier) →
  FeatureEquiv p c₁ c₂ →
  eval p c₁ ≡ eval p c₂
propagation (feat f) c₁ c₂ eq = eq
propagation (p₁ ∧p p₂) c₁ c₂ (e₁ , e₂) =
  cong₂ _∧_ (propagation p₁ c₁ c₂ e₁) (propagation p₂ c₁ c₂ e₂)
propagation (p₁ ∨p p₂) c₁ c₂ (e₁ , e₂) =
  cong₂ _∨_ (propagation p₁ c₁ c₂ e₁) (propagation p₂ c₁ c₂ e₂)
propagation (¬p p)  c₁ c₂ eq =
  cong not (propagation p c₁ c₂ eq)
propagation truep   _ _ _ = refl
propagation falsep  _ _ _ = refl
```

Remark 2 (Why This Matters). The Propagation Theorem is the formal foundation for *generalization* in the synthesis framework. When we observe that a predicate evaluates to `true` at carrier c_1, we automatically know it evaluates to `true` at *every* feature-equivalent carrier. This is not an assumption—it is a *theorem*, proven by structural induction on the predicate program.

In the CSHRL context, this means:

- For FDMDPs: states with the same features get the same ranking, so one observation propagates to the entire equivalence class.
- For CPMDPs: board configurations with the same attack pattern are classified identically by the synthesized `is-dead` predicate.

4 CEGIS: Synthesis from Observations

With the DSL defined, we now present the synthesis algorithm. CEGIS (Counterexample-Guided Inductive Synthesis) maintains a *version space*—the set of candidate predicate programs consistent with all observations so far—and refines it as new observations arrive.

```
module CEGIS
  (all-features : List Feature)
  where

  bfilter : ∀ {A : Set} → (A → Bool) → List A → List A
  bfilter f [] = []
  bfilter f (x ∷ xs) with f x
  ... | true = x ∷ bfilter f xs
  ... | false = bfilter f xs

  _=ᵇ_ : Bool → Bool → Bool
  true =ᵇ true   = true
  false =ᵇ false = true
  _     =ᵇ _     = false

  =ᵇ-refl : ∀ b → (b =ᵇ b) ≡ true
  =ᵇ-refl true = refl
  =ᵇ-refl false = refl

  =ᵇ-sound : ∀ b₁ b₂ → (b₁ =ᵇ b₂) ≡ true → b₁ ≡ b₂
  =ᵇ-sound true true _ = refl
  =ᵇ-sound false false _ = refl
```

4.1 Enumeration

All `PredProg` terms up to a bounded depth are enumerated:

```
atoms : List PredProg
atoms = truep ∷ falsep ∷ map feat all-features

extend : List PredProg → List PredProg
extend prev =
  prev
  ++ map ¬p_ prev
  ++ concatMap (λ p → map (p ∧p_) prev) prev
  ++ concatMap (λ p → map (p ∨p_) prev) prev

enumerate : ℕ → List PredProg
enumerate zero = atoms
enumerate (suc d) = extend (enumerate d)
```

4.2 Observations and Consistency

An observation is a pair (c, b): carrier c should evaluate to b. A candidate is *consistent* if it agrees:

```
PredObs : Set
PredObs = Carrier × Bool

consistent : PredProg → PredObs → Bool
consistent p (c , b) = eval p c =ᵇ b
```

4.3 Version Space Refinement

The version space is the list of consistent candidates. Refinement filters by a new observation:

```
VersionSpace : Set
VersionSpace = List PredProg

initial-vs : ℕ → VersionSpace
initial-vs = enumerate

refine : VersionSpace → PredObs → VersionSpace
refine vs ob = bfilter (λ p → consistent p ob) vs

refine-all : VersionSpace → List PredObs → VersionSpace
refine-all vs [] = vs
refine-all vs (ob ∷ obs) = refine-all (refine vs ob) obs

cegis-step : VersionSpace → PredObs → VersionSpace
cegis-step = refine

cegis-loop : VersionSpace → List PredObs → VersionSpace
cegis-loop = refine-all
```

4.4 Property 1: Monotonic Convergence

The version space never grows—each observation can only eliminate candidates:

```
bfilter-length : ∀ {A : Set} (f : A → Bool) (xs : List A) →
  length (bfilter f xs) ≤ length xs
bfilter-length f [] = z≤n
bfilter-length f (x ∷ xs) with f x
... | true = s≤s (bfilter-length f xs)
... | false = ≤-step (bfilter-length f xs)
  where
    ≤-step : ∀ {m n : ℕ} → m ≤ n → m ≤ suc n
    ≤-step z≤n = z≤n
    ≤-step (s≤s p) = s≤s (≤-step p)

vs-shrinks : ∀ vs ob →
  length (refine vs ob) ≤ length vs
vs-shrinks vs ob = bfilter-length (λ p → consistent p ob) vs
```

Theorem 2 (Bounded Convergence)**.** *After processing n observations from the initial version space, the number of remaining candidates is at most the initial count:*

```
cegis-bounded : ∀ vs obs →
  length (cegis-loop vs obs) ≤ length vs
cegis-bounded vs [] = ≤-refl
cegis-bounded vs (ob ∷ obs) =
  ≤-trans (cegis-bounded (refine vs ob) obs)
          (vs-shrinks vs ob)
  where
    ≤-trans : ∀ {a b c : ℕ} → a ≤ b → b ≤ c → a ≤ c
    ≤-trans z≤n _    = z≤n
    ≤-trans (s≤s p) (s≤s q) = s≤s (≤-trans p q)
```

4.5 Property 2: Soundness

The correct predicate is never eliminated by valid observations:

```
consistent-correct : ∀ p c b →
  eval p c ≡ b →
  consistent p (c , b) ≡ true
consistent-correct p c b eq rewrite eq = =ᵇ-refl b
```

4.6 Property 3: Propagation Acceleration

Feature-equivalent carriers eliminate exactly the same candidates:

```
propagation-acceleration : ∀ (p : PredProg) (c₁ c₂ : Carrier) (b : Bool) →
  FeatureEquiv p c₁ c₂ →
  consistent p (c₁ , b) ≡ consistent p (c₂ , b)
propagation-acceleration p c₁ c₂ b equiv =
  cong (λ v → v =ᵇ b) (propagation p c₁ c₂ equiv)
```

This is the bridge between the Propagation Theorem (§3) and the synthesis algorithm: because feature-equivalent carriers produce identical `eval` results, they produce identical `consistent` results, and therefore identical version-space refinement.

4.7 Property 4: Strict Convergence

An observation is *informative* if it eliminates at least one candidate. Informative observations cause *strict* shrinkage:

```
data _∈ᵢ_ {A : Set} : A → List A → Set where
  here  : ∀ {x xs} → x ∈ᵢ (x ∷ xs)
  there : ∀ {x y xs} → x ∈ᵢ xs → x ∈ᵢ (y ∷ xs)

bfilter-strict : ∀ {A : Set} (f : A → Bool) (xs : List A) →
  ∃[ x ] (x ∈ᵢ xs × f x ≡ false) →
  suc (length (bfilter f xs)) ≤ length xs
bfilter-strict f (x ∷ xs) (.x , here , fx≡f) with f x
... | true with () ← fx≡f
... | false = s≤s (bfilter-length f xs)
bfilter-strict f (x ∷ xs) (z , there mem , fz≡f) with f x
... | true = s≤s (bfilter-strict f xs (z , mem , fz≡f))
... | false = ≤-step′ (bfilter-strict f xs (z , mem , fz≡f))
  where
    ≤-step′ : ∀ {m n : ℕ} → suc m ≤ n → suc m ≤ suc n
    ≤-step′ (s≤s p) = s≤s (≤-step″ p)
      where
        ≤-step″ : ∀ {a b : ℕ} → a ≤ b → a ≤ suc b
        ≤-step″ z≤n = z≤n
        ≤-step″ (s≤s q) = s≤s (≤-step″ q)

Informative : VersionSpace → PredObs → Set
Informative vs ob = ∃[ p ] (p ∈ᵢ vs × consistent p ob ≡ false)

strict-shrink : ∀ vs ob →
  Informative vs ob →
  suc (length (refine vs ob)) ≤ length vs
strict-shrink vs ob info =
  bfilter-strict (λ p → consistent p ob) vs info
```

Theorem 3 (Strict Convergence)**.** *Each informative observation reduces the version space by at least one candidate. Combined with bounded convergence, this means CEGIS terminates after at most $|VS_0|$ informative observations.*

5 Propagation Quantification: The Central Contribution

We now prove three theorems that formalize how the Propagation Theorem (1) reduces the number of observations needed for synthesis. These results are the main theoretical contribution of this paper.

The key insight: because predicate programs operate on features rather than raw carriers, the effective state space for synthesis is the set of *feature equivalence classes*, not the set of carriers. For a domain with $|C|$ carriers but only $|C/{\sim}|$ distinct feature patterns, synthesis needs at most $|C/{\sim}|$ observations.

```
non-informative-id : ∀ vs ob →
  (∀ p → p ∈ᵢ vs → consistent p ob ≡ true) →
  refine vs ob ≡ vs
non-informative-id [] ob _ = refl
non-informative-id (p ∷ vs) ob all-con with consistent p ob
  | all-con p here
... | true | refl =
  cong (p ∷_) (non-informative-id vs ob
    (λ q mem → all-con q (there mem)))
```

5.1 Full Feature Agreement

Two carriers agree on *every* feature (not just those in a specific program):

```
AllFeatAgree : Carrier → Carrier → Set
AllFeatAgree c₁ c₂ = ∀ (f : Feature) →
  eval-feature f c₁ ≡ eval-feature f c₂

all-agree→feat-equiv : ∀ c₁ c₂ →
  AllFeatAgree c₁ c₂ →
  ∀ p → FeatureEquiv p c₁ c₂
all-agree→feat-equiv c₁ c₂ afa (feat f) = afa f
all-agree→feat-equiv c₁ c₂ afa (p₁ ∧p p₂) =
  all-agree→feat-equiv c₁ c₂ afa p₁ ,
  all-agree→feat-equiv c₁ c₂ afa p₂
all-agree→feat-equiv c₁ c₂ afa (p₁ ∨p p₂) =
  all-agree→feat-equiv c₁ c₂ afa p₁ ,
  all-agree→feat-equiv c₁ c₂ afa p₂
all-agree→feat-equiv c₁ c₂ afa (¬p p)  =
  all-agree→feat-equiv c₁ c₂ afa p
all-agree→feat-equiv c₁ c₂ afa truep  = tt
all-agree→feat-equiv c₁ c₂ afa falsep = tt

all-feat-agree-sym : ∀ c₁ c₂ →
  AllFeatAgree c₁ c₂ → AllFeatAgree c₂ c₁
all-feat-agree-sym c₁ c₂ afa f = sym (afa f)
```

5.2 Auxiliary Lemmas

Two properties of `bfilter` that drive the main theorems:

```
bfilter-ext : ∀ {A : Set} (f g : A → Bool) (xs : List A) →
  (∀ x → f x ≡ g x) →
  bfilter f xs ≡ bfilter g xs
bfilter-ext f g [] h = refl
bfilter-ext f g (x ∷ xs) h with f x | g x | h x
... | true | .true | refl = cong (x ∷_) (bfilter-ext f g xs h)
... | false | .false | refl = bfilter-ext f g xs h

bfilter-absorb : ∀ {A : Set} (f g : A → Bool) (xs : List A) →
  (∀ x → f x ≡ true → g x ≡ true) →
  bfilter g (bfilter f xs) ≡ bfilter f xs
bfilter-absorb f g [] h = refl
bfilter-absorb f g (x ∷ xs) h with f x in eq-f
... | false = bfilter-absorb f g xs h
... | true with g x | h x eq-f
... | true | refl = cong (x ∷_) (bfilter-absorb f g xs h)

consistent-equiv : ∀ p c₁ c₂ b →
  AllFeatAgree c₁ c₂ →
  consistent p (c₁ , b) ≡ consistent p (c₂ , b)
consistent-equiv p c₁ c₂ b afa =
  propagation-acceleration p c₁ c₂ b
    (all-agree→feat-equiv c₁ c₂ afa p)

consistent-implies : ∀ c₁ c₂ b →
  AllFeatAgree c₁ c₂ →
  ∀ p → consistent p (c₁ , b) ≡ true →
        consistent p (c₂ , b) ≡ true
consistent-implies c₁ c₂ b afa p h =
  trans (sym (consistent-equiv p c₁ c₂ b afa)) h
```

5.3 Theorem 1: Refine-Equiv

Theorem 4 (Refine-Equiv)**.** *Feature-equivalent carriers produce* identical *version-space refinement. Observing at c_1 eliminates exactly the same candidates as observing at c_2:*

$$\mathit{AllFeatAgree}\ c_1\ c_2 \implies \mathit{refine}\ \mathit{vs}\ (c_1, b) \equiv \mathit{refine}\ \mathit{vs}\ (c_2, b)$$

```
refine-equiv : ∀ vs c₁ c₂ b →
  AllFeatAgree c₁ c₂ →
  refine vs (c₁ , b) ≡ refine vs (c₂ , b)
refine-equiv vs c₁ c₂ b afa =
  bfilter-ext
    (λ p → consistent p (c₁ , b))
    (λ p → consistent p (c₂ , b))
    vs
    (λ p → consistent-equiv p c₁ c₂ b afa)
```

Remark 3. This result is stronger than "the same number of candidates are eliminated." It asserts that the *exact same list* of candidates survives—not just the same count, but the same elements in the same order. This is propositional equality on lists, the strongest form of agreement.

5.4 Theorem 2: Observation Subsumption

Theorem 5 (Observation-Subsumption)**.** *After observing at c_1, a second observation at any feature-equivalent c_2 is provably redundant—the version space is unchanged:*

$$\mathit{AllFeatAgree}\ c_1\ c_2 \implies \mathit{refine}\ (\mathit{refine}\ \mathit{vs}\ (c_1, b))\ (c_2, b) \equiv \mathit{refine}\ \mathit{vs}\ (c_1, b)$$

```
refine-absorb : ∀ vs c₁ c₂ b →
  AllFeatAgree c₁ c₂ →
  refine (refine vs (c₁ , b)) (c₂ , b) ≡ refine vs (c₁ , b)
refine-absorb vs c₁ c₂ b afa =
  bfilter-absorb
    (λ p → consistent p (c₁ , b))
    (λ p → consistent p (c₂ , b))
    vs
    (consistent-implies c₁ c₂ b afa)

observation-subsumption : ∀ vs c₁ c₂ b →
  AllFeatAgree c₁ c₂ →
  refine-all vs ((c₁ , b) ∷ (c₂ , b) ∷ [])
    ≡ refine vs (c₁ , b)
observation-subsumption vs c₁ c₂ b afa =
  refine-absorb vs c₁ c₂ b afa
```

Every candidate that survived the first observation automatically survives the second. No new information is gained—the observation is *subsumed*.

5.5 Theorem 3: Exploration Dissolution

Theorem 6 (Exploration-Dissolution)**.** *An entire equivalence class of carriers is handled by a* single *observation at any representative c_0. After that observation, all further observations at equivalent carriers leave the version space unchanged:*

```
AllEquiv : Carrier → List Carrier → Set
AllEquiv c₀ [] = ⊤
```

```
AllEquiv c₀ (c :: cs) = AllFeatAgree c₀ c × AllEquiv c₀ cs

exploration-dissolution : ∀ vs c₀ b (cs : List Carrier) →
  AllEquiv c₀ cs →
  cegis-loop (refine vs (c₀ , b)) (map (λ c → (c , b)) cs)
    ≡ refine vs (c₀ , b)
exploration-dissolution vs c₀ b [] _ = refl
exploration-dissolution vs c₀ b (c :: cs) (afa , rest) =
  let step₁ = refine-absorb vs c₀ c b afa
  in trans
    (cong (λ vs' → cegis-loop vs' (map (λ c' → (c' , b)) cs))
          step₁)
    (exploration-dissolution vs c₀ b cs rest)
```

Corollary 7 (Dissolution preserves VS size)**.** *Since the version space is unchanged, its size is trivially preserved:*

```
dissolution-size : ∀ vs c₀ b (cs : List Carrier) →
  AllEquiv c₀ cs →
  length (cegis-loop (refine vs (c₀ , b))
                     (map (λ c → (c , b)) cs))
    ≡ length (refine vs (c₀ , b))
dissolution-size vs c₀ b cs afa =
  cong length (exploration-dissolution vs c₀ b cs afa)
```

5.6 Corollary: Dissolution + Strict Convergence

The combination of dissolution and strict convergence gives the tightest bound:

Corollary 8 (Dissolution-Strict)**.** *Within an equivalence class, only the* first *observation can be informative. All subsequent are provably redundant. Total informative observations* $\leq |C/\sim|$*:*

```
dissolution-strict : ∀ vs c₀ b (cs : List Carrier) →
  AllEquiv c₀ cs →
  Informative vs (c₀ , b) →
  suc (length (cegis-loop (refine vs (c₀ , b))
                          (map (λ c → (c , b)) cs)))
    ≤ length vs
dissolution-strict vs c₀ b cs afa info =
  subst (λ n → suc n ≤ length vs)
        (sym (cong length (exploration-dissolution vs c₀ b cs afa)))
        (strict-shrink vs (c₀ , b) info)
```

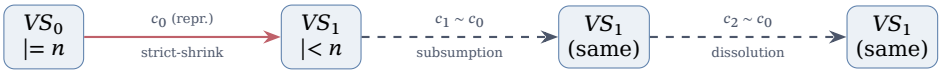

Figure 2: Exploration dissolution in action. The first observation at representative c_0 strictly shrinks the version space. Subsequent observations at equivalent carriers $c_1, c_2, \ldots$ are no-ops—proven by Observation-Subsumption.

Remark 4 (Complexity Reduction). Consider the 8-Queens domain. There are $8^8 = 16{,}777{,}216$ possible board configurations, but the number of distinct feature patterns (attack + length patterns) is much smaller. Propagation quantification proves that synthesis needs to observe only one configuration per pattern class, reducing the effective exploration space by orders of magnitude.

6 Quotient Structure and Exploration Completeness

The propagation quantification results of §5 show that feature-equivalent carriers are redundant *pairwise*. We now lift this to the global level: `AllFeatAgree` is an equivalence relation, and after observing one representative per equivalence class, *no further observation can refine the version space*.

6.1 AllFeatAgree Is an Equivalence Relation

Reflexivity and symmetry are immediate; transitivity composes pointwise:

```
AFA-refl : ∀ c → AllFeatAgree c c
AFA-refl c f = refl

AFA-trans : ∀ c₁ c₂ c₃ →
  AllFeatAgree c₁ c₂ → AllFeatAgree c₂ c₃ → AllFeatAgree c₁ c₃
AFA-trans c₁ c₂ c₃ afa₁₂ afa₂₃ f = trans (afa₁₂ f) (afa₂₃ f)
```

Together with the existing `all-feat-agree-sym`, this establishes `AllFeatAgree` as an equivalence relation, partitioning the carrier space into classes $C/\sim$.

6.2 Auxiliary: Soundness and Monotonicity

Two helper lemmas drive the completeness proof. First, elements surviving a `bfilter` satisfy the predicate—this is the "soundness" direction of filtering:

```
bfilter-sound : ∀ {A : Set} (f : A → Bool) (xs : List A) →
  ∀ x → x ∈ᵢ bfilter f xs → f x ≡ true
bfilter-sound f (y :: xs) x mem with f y in eq
bfilter-sound f (y :: xs) .y here      | true = eq
bfilter-sound f (y :: xs) x (there m) | true = bfilter-sound f xs x m
bfilter-sound f (y :: xs) x mem       | false = bfilter-sound f xs x mem
```

Second, `cegis-loop` only removes elements—its output is a sublist of its input:

```
private
  bfilter-⊆ : ∀ {A : Set} (f : A → Bool) (xs : List A) →
    ∀ x → x ∈ᵢ bfilter f xs → x ∈ᵢ xs
  bfilter-⊆ f (y :: xs) x mem with f y
  bfilter-⊆ f (y :: xs) .y here      | true = here
  bfilter-⊆ f (y :: xs) x (there m) | true = there (bfilter-⊆ f xs x m)
  bfilter-⊆ f (y :: xs) x mem       | false = there (bfilter-⊆ f xs x mem)

cegis-loop-⊆ : ∀ vs obs →
  ∀ x → x ∈ᵢ cegis-loop vs obs → x ∈ᵢ vs
cegis-loop-⊆ vs [] x mem = mem
cegis-loop-⊆ vs (ob :: obs) x mem =
  bfilter-⊆ (λ p → consistent p ob) vs x
    (cegis-loop-⊆ (refine vs ob) obs x mem)
```

6.3 Subsumption Stability

The key technical lemma: subsumption is *stable under further refinement*. After refining by representative r and then processing additional observations, refining by any $c \sim r$ is still a no-op:

Theorem 9 (Refine-After-CEGIS)**.** *If $r \sim c$ (AllFeatAgree), then after refining by (r, b) and processing further observations, the version space is already consistent with (c, b):*

```
refine-after-cegis : ∀ vs r c b (obs : List PredObs) →
  AllFeatAgree r c →
  refine (cegis-loop (refine vs (r , b)) obs) (c , b)
    ≡ cegis-loop (refine vs (r , b)) obs
refine-after-cegis vs r c b obs afa =
  non-informative-id
    (cegis-loop (refine vs (r , b)) obs) (c , b)
    (λ p mem →
      consistent-implies r c b afa p
        (bfilter-sound (λ q → consistent q (r , b)) vs p
          (cegis-loop-⊆ (refine vs (r , b)) obs p mem)))
```

The proof chains three facts: (1) every surviving candidate p is in the original refined VS (by `cegis-loop-⊆`); (2) it satisfies `consistent` at (r, b) (by `bfilter-sound`); (3) it therefore satisfies `consistent` at (c, b) (by `consistent-implies` from feature equivalence).

6.4 Exploration Completeness

Theorem 10 (Exploration-Complete)**.** *After processing a list of representatives (one per equivalence class), any carrier equivalent to some representative is provably redundant:*

$$r \in \mathit{reps} \;\wedge\; r \sim c \implies \mathit{refine}\ (\mathit{cegis\text{-}loop}\ \mathit{vs}\ \mathit{reps})\ (c, b) \equiv \mathit{cegis\text{-}loop}\ \mathit{vs}\ \mathit{reps}$$

```
exploration-complete : ∀ vs (reps : List Carrier) b c →
  (∃[ r ] (r ∈ᵢ reps × AllFeatAgree r c)) →
  refine (cegis-loop vs (map (λ r → (r , b)) reps)) (c , b)
    ≡ cegis-loop vs (map (λ r → (r , b)) reps)
exploration-complete vs (r ∷ reps) b c (.r , here , afa) =
  refine-after-cegis vs r c b (map (λ r' → (r' , b)) reps) afa
exploration-complete vs (r' ∷ reps) b c (r , there mem , afa) =
  exploration-complete (refine vs (r' , b)) reps b c (r , mem , afa)
```

The proof splits on where representative r appears. If r is the head, `refine-after-cegis` applies directly. If r is in the tail, the inductive hypothesis applies to the version space already refined by the head.

Corollary 11 (Dissolution Is All You Need)**.** *Once every equivalence class has a representative in the observation list, no carrier anywhere in C can produce an informative observation. The version space is* fully refined*—further sampling is provably wasteful.*

Remark 5 (Tight Convergence). Combined with Strict-Shrink, this gives the tight bound: the number of informative observations is at most $|C/\!\sim|$, the number of equivalence classes. Each class contributes at most one informative observation (at its representative); all subsequent observations within the class are no-ops by Exploration-Complete. For 8-Queens with attack features, $|C/\!\sim| \ll 8^8$.

7 Temporal Propagation: Coinductive Chains

Propagation as presented so far is *spatial*: it transfers observations across carriers at the same point in time. In MDPs, there is also a *temporal* dimension: states evolve through transitions. If the transition function preserves feature equivalence, then observations propagate not just across equivalent carriers but along *entire trajectories*.

Definition 2 (Feature Preservation)**.** A transition function $\mathit{step} : C \to A \to C$ **preserves features** if:

$$c_1 \sim c_2 \implies \mathit{step}(c_1, a) \sim \mathit{step}(c_2, a) \quad \text{for all actions } a$$

This is a natural property: if two states look the same (same features), acting on them produces states that also look the same.

```
module TemporalPropagation
  {Act : Set}
  (step-carrier : Carrier → Act → Carrier)
  (step-preserves-feat : ∀ c₁ c₂ a →
    AllFeatAgree c₁ c₂ →
    AllFeatAgree (step-carrier c₁ a) (step-carrier c₂ a))
  where

  iterₜ : Carrier → List Act → Carrier
  iterₜ c [] = c
  iterₜ c (a ∷ as) = iterₜ (step-carrier c a) as

  trajectory : Carrier → Bool → List Act → List PredObs
  trajectory c b [] = (c , b) ∷ []
  trajectory c b (a ∷ as) = (c , b) ∷ trajectory (step-carrier c a) b as
```

7.1 Temporal Feature Equivalence

Feature equivalence composes through arbitrary action sequences:

Theorem 12 (Temporal-Feat-Equiv)**.** *If $c_1 \sim c_2$ and `step` preserves features, then after any action sequence $[a_1, \ldots, a_n]$, the resulting states are still equivalent:*

```
temporal-feat-equiv : ∀ c₁ c₂ (as : List Act) →
  AllFeatAgree c₁ c₂ →
  AllFeatAgree (iterₜ c₁ as) (iterₜ c₂ as)
temporal-feat-equiv c₁ c₂ [] afa = afa
temporal-feat-equiv c₁ c₂ (a ∷ as) afa =
  temporal-feat-equiv
    (step-carrier c₁ a) (step-carrier c₂ a) as
    (step-preserves-feat c₁ c₂ a afa)
```

7.2 Trajectory Refine-Equiv

Trajectories from feature-equivalent carriers produce *identical* version-space refinement:

Theorem 13 (Trajectory-Refine-Equiv)**.** *For any action sequence, observing a trajectory from c_1 refines the version space identically to observing the trajectory from c_2:*

```
trajectory-refine-equiv : ∀ vs c₁ c₂ b (as : List Act) →
  AllFeatAgree c₁ c₂ →
  cegis-loop vs (trajectory c₁ b as)
    ≡ cegis-loop vs (trajectory c₂ b as)
trajectory-refine-equiv vs c₁ c₂ b [] afa =
  refine-equiv vs c₁ c₂ b afa
trajectory-refine-equiv vs c₁ c₂ b (a ∷ as) afa =
  trans
    (cong (λ vs' → cegis-loop vs'
                (trajectory (step-carrier c₁ a) b as))
          (refine-equiv vs c₁ c₂ b afa))
    (trajectory-refine-equiv (refine vs (c₂ , b))
      (step-carrier c₁ a) (step-carrier c₂ a) b as
      (step-preserves-feat c₁ c₂ a afa))
```

The proof inductively combines spatial propagation at each step (`refine-equiv`) with temporal propagation at the successors (the inductive hypothesis with preserved feature equivalence).

7.3 Trajectory Dissolution

The central result: after observing an entire trajectory from c_1, the entire trajectory from any equivalent c_2 is *completely subsumed*:

Theorem 14 (Trajectory-Dissolution)**.** *One trajectory observation covers entire families of equivalent trajectories:*

$$c_1 \sim c_2 \implies \mathit{cegis\text{-}loop}\,(\mathit{cegis\text{-}loop}\;\mathit{vs}\;\mathit{traj}(c_1))\;\mathit{traj}(c_2) \;\equiv\; \mathit{cegis\text{-}loop}\;\mathit{vs}\;\mathit{traj}(c_1)$$

```
trajectory-dissolution : ∀ vs c₁ c₂ b (as : List Act) →
  AllFeatAgree c₁ c₂ →
  cegis-loop (cegis-loop vs (trajectory c₁ b as))
             (trajectory c₂ b as)
    ≡ cegis-loop vs (trajectory c₁ b as)
trajectory-dissolution vs c₁ c₂ b [] afa =
  refine-absorb vs c₁ c₂ b afa
trajectory-dissolution vs c₁ c₂ b (a :: as) afa =
  trans
    (cong (λ v → cegis-loop v
             (trajectory (step-carrier c₂ a) b as))
          (refine-after-cegis vs c₁ c₂ b
            (trajectory (step-carrier c₁ a) b as) afa))
    (trajectory-dissolution (refine vs (c₁ , b))
      (step-carrier c₁ a) (step-carrier c₂ a) b as
      (step-preserves-feat c₁ c₂ a afa))
```

The inductive step uses `refine-after-cegis` (subsumption stability from §6) to show that the first observation (c_2, b) is absorbed, and then the inductive hypothesis with preserved feature equivalence to show the rest of the trajectory is absorbed.

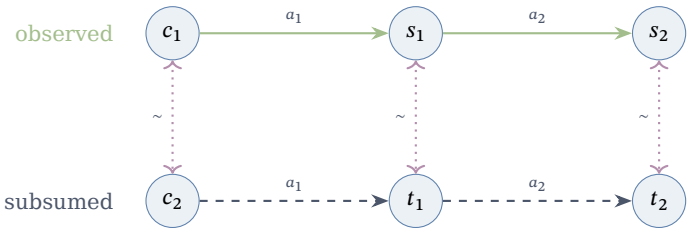

Figure 3: Temporal dissolution. Feature preservation ensures that equivalence ($\sim$) is maintained at every step. Observing the trajectory from c_1 (solid) makes the trajectory from c_2 (dashed) completely redundant.

Remark 6 (Exponential Reduction)*.* In an MDP with k equivalence classes, spatial dissolution handles k observations. With temporal propagation over trajectories of length n, the total observations covered grows to $k \times n$—but only k trajectory observations are needed. For long-horizon tasks, this is an exponential reduction: one root observation at a representative covers the entire future of every equivalent state. This mirrors CSHRL's coinductive unfolding: just as the `CoindHomo` guarantees action-value dominance at *every* future timestep, temporal propagation guarantees observation equivalence at *every* future timestep.

8 Feature Sufficiency: From Representatives to Global Correctness

The preceding sections establish that feature-equivalent carriers are *indistinguishable* to the DSL, that dissolution reduces observations to $|C/\!\sim|$, and that temporal propagation

extends this across trajectories. But a fundamental question remains: *when is the CEGIS result actually correct*?

Exploration completeness (Section 6) guarantees that no further refinement is possible. But "no refinement possible" and "correct answer" are logically distinct claims. The version space might stabilize on programs that all agree with the target on observed carriers yet disagree elsewhere.

Feature sufficiency bridges this gap. When the target function *respects* feature equivalence—meaning `AllFeatAgree` c_1 c_2 implies identical target values—then correctness at representatives *propagates* to correctness everywhere. This yields the crown jewel of the framework: the **Feature Sufficiency Theorem**.

Definition 3 (Feature Respect)**.** A target function t : `Carrier` → `Bool` *respects* the feature set when feature-equivalent carriers receive identical values:

```
FeatureRespects : (Carrier → Bool) → Set
FeatureRespects target = ∀ ₁c ₂c →
  AllFeatAgree ₁c ₂c → target ₁c ≡ target ₂c
```

Definition 4 (Matching)**.** A predicate program p *matches* a target t at carrier c when `eval` $p\ c \equiv t\ c$. It matches *everywhere* when this holds for all carriers.

```
Matches    : PredProg → (Carrier → Bool) → Carrier → Set
Matches    p target c = eval p c ≡ target c

MatchesAll : PredProg → (Carrier → Bool) → Set
MatchesAll p target = ∀ c → Matches p target c
```

Theorem 15 (Match Propagation)**.** *If a program matches the target at c_1, and the target respects features, then the program matches at every feature-equivalent c_2.*

The proof chains propagation (for the program side) with feature respect (for the target side):

```
match-propagates : ∀ p target ₁c ₂c →
  AllFeatAgree ₁c ₂c →
  FeatureRespects target →
  Matches p target ₁c →
  Matches p target ₂c
match-propagates p target ₁c ₂c afa respects ₁match =
  trans
    (sym (propagation p ₁c ₂c
          (all-→agreefeat-equiv ₁c ₂c afa p)))
    (trans ₁match (respects ₁c ₂c afa))
```

Definition 5 (Covers)**.** A list of representatives *covers* the carrier space when every carrier is feature-equivalent to some representative.

```
Covers : List Carrier → Set
Covers reps = ∀ c →
  ∃[ r ] (r ∈ₗ reps × AllFeatAgree r c)
```

Theorem 16 (Generalization)**.** *If a program matches the target at all representatives, and the representatives cover the carrier space, then the program matches everywhere.*

```
generalization : ∀ p target (reps : List Carrier) →
  FeatureRespects target →
  Covers reps →
  ∀( r → r ∈ₗ reps → Matches p target r) →
  MatchesAll p target
generalization p target reps respects covers matches-reps c =
  let (r , r-in , afa) = covers c
  in match-propagates p target r c afa
       respects (matches-reps r r-in)
```

The glue between CEGIS and generalization is the *survivor consistency* lemma: every program that survives the CEGIS loop is consistent with all observations that were processed.

```
cegis-survivors-consistent : ∀ vs obs →
  ∀ p → p ∈ₗ cegis-loop vs obs →
  ∀ ob → ob ∈ₗ obs → consistent p ob ≡ true
```

The proof proceeds by induction on the observation list: for the head observation, the survivor is in the refined version space (by `cegis-loop-⊆`) and hence consistent (by `bfilter-sound`); for the tail, the induction hypothesis applies directly.

Theorem 17 (Feature Sufficiency — Crown Jewel)**.** *With sufficient features (target respects `AllFeatAgree`) and representative observations (one per equivalence class), every CEGIS survivor is correct on **all** carriers.*

```
sufficient-cegis : ∀ vs (reps : List Carrier) →
  FeatureRespects target →
  Covers reps →
  ∀ p → p ∈ₗ cegis-loop vs
                (map (λ r → (r , target r)) reps) →
  MatchesAll p target
sufficient-cegis vs reps fr cov p p-in = true
```

The proof composes three results: `cegis-survivors-consistent` (survivors match at observed carriers), `consistent-to-match` (consistency implies matching), and `generalization` (matching at representatives implies matching everywhere).

Remark 7 (What Changes). Without feature sufficiency, Exploration-Complete says "the version space has stabilized." *With* feature sufficiency, it says "the version space contains only correct programs." The stabilization guarantee is upgraded to a *correctness* guarantee—every survivor computes the target function on every carrier, not just the observed ones.

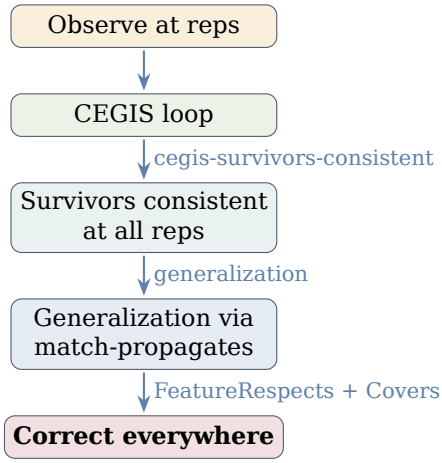

Figure 4: Feature sufficiency proof pipeline. Observing one representative per equivalence class and running CEGIS produces programs that are provably correct on *all* carriers.

9 Multi-Predicate Propagation

Many environments require synthesizing *multiple* predicates simultaneously from the same feature space. In the CPMDP setting, for example, we synthesize both `is-dead` and `is-solved`—two Boolean functions sharing the same carrier type and feature set. This section develops the theory of joint synthesis, where observations for one predicate constrain the other.

9.1 Joint Dissolution

Since both predicates are defined over the same carrier and feature set, a single `AllFeatAgree` proof propagates to both version spaces simultaneously. The `MultiPredicate` module in `Core.agda` is parameterized by two target functions:

```
module MultiPredicate
  (₁target ₂target : Carrier → Bool) where

  joint-refine : VersionSpace → VersionSpace →
    Carrier → VersionSpace × VersionSpace
  joint-refine ₁vs ₂vs c =
    (refine ₁vs (c , ₁target c) ,
     refine ₂vs (c , ₂target c))
```

One carrier observation simultaneously refines both version spaces. Joint dissolution follows immediately: when $c_1 \sim c_2$, the joint refinement is identical, so one observation handles both predicates for the entire equivalence class.

```
  joint-refine-equiv : ∀ ₁vs ₂vs ₁c ₂c →
    AllFeatAgree ₁c ₂c →
    FeatureRespects ₁target →
    FeatureRespects ₂target →
    joint-refine ₁vs ₂vs ₁c
      ≡ joint-refine ₁vs ₂vs ₂c
```

9.2 Cross-Predicate Entailment

In many domains, predicates are logically related. For Queens, dead states cannot be solved: if `is-dead` c = `true`, then `is-solved` c = `false`. This *entailment* gives free observations.

```
  Entails¬ : Set
  Entails¬ = ∀ c → ₁target c ≡ true →
                   ₂target c ≡ false
```

With entailment, observing `target`$_1$ c = `true` automatically refines the second version space by `(c , false)`, without evaluating `target`$_2$. This is formalized as `cross-matches-joint`: the cross-refinement equals the full joint refinement.

```
  cross-matches-joint : ∀ ₁vs ₂vs c →
    (ent : Entails¬) →
    (t1 : ₁target c ≡ true) →
    cross-refine ₁vs ₂vs c ent t1
      ≡ joint-refine ₁vs ₂vs
          (c , ₁target c , ₂target c)
```

9.3 Cross-Dissolution

The multiplicative power emerges when entailment meets dissolution. Observing `target`$_1$ c_1 = `true` with entailment `dead` → ¬`solved`:

1. Refines `vs`$_1$ by (c_1, `true`)
2. Refines `vs`$_2$ by (c_1, `false`) — **free** from entailment
3. Renders all $c_2 \sim c_1$ redundant for **both** version spaces — from dissolution

One observation, two predicates, an entire equivalence class:

```
  cross-dissolution : ∀ ₁vs ₂vs ₁c ₂c →
    AllFeatAgree ₁c ₂c →
    Entails¬ → ₁target ₁c ≡ true →
    let ₁vs' = refine ₁vs (₁c , true)
        ₂vs' = refine ₂vs (₁c , false)
    in refine ₁vs' (₂c , true)  ≡ ₁vs'
     × refine ₂vs' (₂c , false) ≡ ₂vs'
```

9.4 Joint Sufficiency

The feature sufficiency theorem lifts to multiple predicates: with sufficient features for both targets and the same set of representative observations, both synthesized predicates are correct on all carriers.

Theorem 18 (Joint Sufficiency)**.** *With sufficient features for both targets and representative coverage, running CEGIS independently on each target using the same representatives yields programs that are correct everywhere—for* both *predicates.*

```
joint-sufficient : ∀ ₁vs ₂vs (reps : List Carrier) →
  FeatureRespects ₁target →
  FeatureRespects ₂target →
  Covers reps →
  ∀ ₁p ₂p →
  ₁p ∈ₗ cegis-loop ₁vs
           (map λ( r → (r , ₁target r)) reps) →
  ₂p ∈ₗ cegis-loop ₂vs
           (map λ( r → (r , ₂target r)) reps) →
  MatchesAll ₁p ₁target × MatchesAll ₂p ₂target
```

This directly applies `sufficient-cegis` (Theorem 17) to each predicate independently—the key insight is that the *same* representatives suffice for both, since both predicates share the same feature space and hence the same equivalence classes.

Remark 8 (Observation Budget)*.* For a single predicate, the observation budget is $|C/\sim|$. For k predicates sharing the same features, the budget remains $|C/\sim|$ (not $k \cdot |C/\sim|$): one set of representative observations suffices for all k predicates. With entailment, the *evaluation* budget drops further, since some target values can be derived without computation. In the Queens setting, this means synthesizing both `is-dead` and `is-solved` requires no more observations than synthesizing either one alone.

10 Online Synthesis: Learning-Synthesis Integration

The preceding sections established CEGIS as a batch algorithm: given a fixed list of observations, the version space is refined to convergence. In practice, observations arrive *incrementally*—a learning loop produces violations and samples one at a time, and the synthesis engine must process them on-the-fly. This section establishes three key results that enable this integration:

1. **Order independence**: the final version space depends only on the *set* of observations, not their order;
2. **Replay redundancy**: re-submitting a previously seen observation is a provable no-op;
3. **Learning Bridge**: a generic module connecting learning feedback to synthesis observations, with a bridge sufficiency theorem that composes learning convergence with synthesis correctness.

10.1 Observation Order Independence

The central structural result is that two successive refinements *commute*: refining by ob_1 then ob_2 yields the same version space as refining by ob_2 then ob_1.

```
refine-commutes : ∀ vs ₁ob ₂ob →
  refine (refine vs ₁ob) ₂ob ≡ refine (refine vs ₂ob) ₁ob
```

The proof proceeds via an auxiliary `bfilter'` (using `if-then-else` instead of `with`, enabling clean equational reasoning) and a `bfilter-swap` lemma showing that composing two Boolean filters is commutative. The key technical insight: each filter retains exactly those candidates consistent with its observation, and Boolean conjunction is commutative, so the surviving candidates are identical regardless of filter order.

An immediate corollary lifts commutativity to the full CEGIS loop—swapping any two adjacent observations preserves the result:

```
cegis-loop-swap : ∀ vs ₁ob ₂ob obs →
  cegis-loop vs (₁ob ∷ ₂ob ∷ obs)
    ≡ cegis-loop vs (₂ob ∷ ₁ob ∷ obs)
cegis-loop-swap vs ₁ob ₂ob obs =
  cong λ( v → cegis-loop v obs) (refine-commutes vs ₁ob ₂ob)
```

Since any permutation can be decomposed into adjacent transpositions, this implies full permutation invariance: the version space is determined by the *multiset* of observations.

Remark 9 (Significance for Online Learning). Order independence has a profound practical implication: the learning loop can submit observations in *any* order—depth-first, breadth-first, violation-driven, or randomized—and the synthesized result is identical. This means the synthesis engine is insensitive to the exploration strategy, and parallel learners can submit observations asynchronously without coordination.

10.2 Replay Redundancy

In an online setting, the learner may revisit a state, re-evaluate a sample, or replay a checkpoint. The following theorem guarantees that such replays are harmless:

```
replay-redundant : ∀ vs obs ob →
  ob ∈ₗ obs →
  refine (cegis-loop vs obs) ob ≡ cegis-loop vs obs
```

If *ob* was already among the processed observations, re-applying it to the refined version space is a no-op. The proof combines `cegis-survivors-consistent` (every survivor passes all processed observations) with `non-informative-id` (non-informative observations leave the VS unchanged).

Combined with the late-dissolution theorem (`refine-after-cegis`, §6), this establishes a strong idempotence property: in the online stream, *any* observation that is feature-equivalent to a previously processed observation is provably redundant, regardless of how many other observations have been processed in between.

10.3 The Learning Bridge

The generic learning-synthesis bridge connects a learning loop (which produces domain-specific feedback such as violations and trace comparisons) with the synthesis version space. The bridge is parameterized by a mapping from learner feedback to synthesis observations:

```
module LearningBridge
  {Feedback : Set}
  (to-obs : Feedback → PredObs)
  where
```

This abstraction covers all EC instantiations:

- **FDMDP**: Feedback is a state-action-action triple with a trace comparison; `to-obs` maps it to a PredObs for the `prefer(a,b)` predicate.
- **CPMDP**: Feedback is a configuration evaluation; `to-obs` maps it to a PredObs for the target predicate (e.g., `is-dead`).

The bridge defines incremental processing and batch processing, with a key equivalence theorem:

```
  bridge-is-cegis : ∀ vs fbs →
    bridge-batch vs fbs ≡ cegis-loop vs (map to-obs fbs)
```

This states that processing feedback through the bridge is *exactly* equivalent to running CEGIS on the mapped observations. All properties of CEGIS (monotonicity, convergence bounds, dissolution) transfer immediately:

```
  bridge-mono : ∀ vs fbs →
    length (bridge-batch vs fbs) ≤ length vs
```

The crown jewel of integration is **bridge sufficiency**: if the learning loop produces feedback that maps to representative observations for the target, every surviving program is correct everywhere.

```
bridge-sufficient : ∀ vs target (reps : List Carrier) →
  FeatureRespects target →
  Covers reps →
  (fbs : List Feedback) →
  map to-obs fbs ≡ map λ( r → (r , target r)) reps →
  ∀ p → p ∈ₗ bridge-batch vs fbs →
  MatchesAll p target
```

This theorem is the formal bridge between *learning convergence* and *synthesis correctness*: the learner drives observation collection, and the synthesizer guarantees that survivors are globally correct. The proof reduces to `sufficient-cegis` via `bridge-is-cegis`—the bridge adds no overhead.

The learning-side half of this composition is now itself a closed proof: the convergence of swap-based violation repair, previously an assumption record (`ViolationMonotonicityTheorem`) in the reference implementation, is proved in `src/CSHRL/Learning/Convergence.agda` (each adjacent-transposition repair strictly decreases the violation count, giving convergence to an oracle-realizing ranking within $\binom{n}{2}$ repairs per state). The composed pipeline—learning convergence through Bridge-Sufficient to synthesis correctness—is therefore assumption-free end to end.

The bridge also inherits commutativity and replay redundancy:

```
bridge-commutes : ∀ vs ₁fb ₂fb →
  bridge-step (bridge-step vs ₁fb) ₂fb
    ≡ bridge-step (bridge-step vs ₂fb) ₁fb
```

```
bridge-replay : ∀ vs fbs fb →
  fb ∈ₗ fbs →
  bridge-step (bridge-batch vs fbs) fb
    ≡ bridge-batch vs fbs
```

Remark 10 (The Complete Pipeline). The full verified pipeline is now:

$$\text{Learning loop} \xrightarrow{\texttt{to-obs}} \text{Bridge} \xrightarrow{\texttt{bridge-batch}} \text{VS} \xrightarrow{\texttt{survivors}} \text{Correct programs}$$

Each arrow is verified: `to-obs` is an EC-specific mapping; the bridge preserves CEGIS semantics (`bridge-is-cegis`); the VS converges (`bridge-mono`); survivors are globally correct (`bridge-sufficient`). Commutativity (`bridge-commutes`) and replay redundancy (`bridge-replay`) ensure the pipeline is robust to observation order and repetition.

11 Sample Complexity Lower Bound

The Feature Sufficiency Theorem (§8) established an *upper bound*: $|C/\sim|$ representative observations suffice for synthesis correctness. A natural question is whether this bound is *tight*—can we guarantee correctness with fewer observations? This section answers in the negative: $|C/\sim|$ observations are also *necessary*.

The core insight is information-theoretic: **CEGIS only distinguishes targets that differ on observed carriers.** Two target functions producing identical observations yield identical version spaces, so any survivor for one is automatically a survivor for the other. If the targets disagree on an unobserved carrier, no single program can match both—so CEGIS cannot guarantee correctness for either.

11.1 Target Indistinguishability

The foundational result: two targets that agree on all observed carriers produce the same version space.

```
same-obs-same-vs : ∀ vs (cs : List Carrier)
  (₁target ₂target : Carrier → Bool) →
```

```
      ∀( c → c ∈ˡ cs → ₁target c ≡ ₂target c) →
      cegis-loop vs (map λ( c → (c , ₁target c)) cs)
        ≡ cegis-loop vs (map λ( c → (c , ₂target c)) cs)
```

The proof is direct: if $target_1$ and $target_2$ agree on every element of `cs`, then the mapped observation lists are identical by extensionality, and `cegis-loop` produces the same result by congruence.

An immediate corollary is **survivor transfer**: every program surviving CEGIS for one target automatically survives for any target indistinguishable on the observed carriers.

```
    survivor-transfer : ∀ vs (cs : List Carrier)
      (₁target ₂target : Carrier → Bool) →
      ∀( c → c ∈ˡ cs → ₁target c ≡ ₂target c) →
      ∀ p → p ∈ˡ cegis-loop vs (map λ( c → (c , ₁target c)) cs) →
            p ∈ˡ cegis-loop vs (map λ( c → (c , ₂target c)) cs)
```

This is the *core lower bound mechanism*: survivors transfer freely between targets that look the same on the observations. If we have too few observations, there exist alternative targets that are indistinguishable from the real one, and survivors for the real target are equally valid for the alternatives—even when the alternatives disagree on uncovered carriers.

11.2 Covered Agreement and Feature Propagation

With feature-respecting targets, agreement on representatives propagates to all covered carriers:

```
    covered-agreement : ∀ (₁target ₂target : Carrier → Bool)
      (reps : List Carrier) →
      FeatureRespects ₁target →
      FeatureRespects ₂target →
      ∀( r → r ∈ˡ reps → ₁target r ≡ ₂target r) →
      ∀ c → ∃([ r ] (r ∈ˡ reps × AllFeatAgree r c)) →
      ₁target c ≡ ₂target c
```

The proof chains three equalities: $target_1\, c = target_1\, r$ (by `FeatureRespects` on $target_1$ and `AllFeatAgree`$\,r\,c$), then $target_1\, r = target_2\, r$ (by the representative agreement hypothesis), then $target_2\, r = target_2\, c$ (by `FeatureRespects` on $target_2$).

With full coverage, this becomes unconditional:

```
    full-coverage-determines : ∀ (₁target ₂target : Carrier → Bool)
      (reps : List Carrier) →
      FeatureRespects ₁target →
      FeatureRespects ₂target →
      Covers reps →
      ∀( r → r ∈ˡ reps → ₁target r ≡ ₂target r) →
      ∀ c → ₁target c ≡ ₂target c
```

This is the converse of the lower bound: with full coverage (`Covers` *reps*), no alternative target can agree on all representatives without agreeing *everywhere*. In other words, full coverage *uniquely determines* the target among all feature-respecting functions.

11.3 The Tight Bound

Combining the upper and lower bounds yields the central sample complexity result.

Theorem 19 (Tight-Bound)**.** *The sample complexity of CEGIS with the Predicate DSL over features is* exactly $|C/{\sim}|$*: one representative per equivalence class.*

- **Upper bound** (`sufficient-cegis`, §8): With `Covers` *reps* and `FeatureRespects` *target*, every CEGIS survivor satisfies `MatchesAll`.
- **Lower bound** (`lower-bound`): Without `Covers`, there exist two feature-respecting targets that agree on all observed representatives but disagree on some uncovered carrier c. Every survivor for $target_1$ is also a survivor for $target_2$ (by `survivor-transfer`), but no program can match both (by `no-universal-match`):

```
lower-bound : ∀ vs (reps : List Carrier)
  (₁target ₂target : Carrier → Bool) (c : Carrier) →
  ∀( r → r ∈ₗ reps → ₁target r ≡ ₂target r) →
  ₁target c ≢ ₂target c →
  ∀ p → p ∈ₗ cegis-loop vs (map λ( r → (r , ₁target r)) reps) →
  p ∈ₗ cegis-loop vs (map λ( r → (r , ₂target r)) reps)
    × ¬ (MatchesAll p ₁target × MatchesAll p ₂target)
```

The first conjunct says the survivor *also* survives for $target_2$; the second says no program can match both targets everywhere. Together: if an equivalence class is unobserved, CEGIS cannot distinguish between targets that differ there, so correctness is not guaranteed.

Remark 11 (Information-Theoretic Interpretation). The tight bound has a clean information-theoretic reading. Each equivalence class under `AllFeatAgree` is an *information cell*: all carriers within a cell are indistinguishable to any predicate program. The target function assigns one bit (true/false) per cell. CEGIS needs to determine these bits, and each representative observation reveals exactly one. With $k < |C/{\sim}|$ observations, at least one cell is undetermined—leaving $2^{|C/{\sim}|-k}$ possible targets indistinguishable from the true one. Full coverage ($k = |C/{\sim}|$) resolves all ambiguity.

This is analogous to the classical result that n binary questions are necessary and sufficient to identify one of 2^n hypotheses, specialized to our setting where the "questions" are observations and the "hypotheses" are feature-respecting target functions.

12 DSL Completeness

The tight bound (§11) tells us *how many* observations are needed. We now address a prior question: *does the DSL always contain a correct program?* If the target function lies outside the DSL's expressiveness, no amount of observations can help.

We prove that the Boolean `PredProg` DSL is *expressively complete* for feature-respecting targets: for any target function that is constant on `AllFeatAgree` equivalence classes, there exists a `PredProg` matching it everywhere.

12.1 Fingerprints

The construction uses *fingerprints*. Given a carrier c and a feature list, the fingerprint of c is the conjunction of feature literals—positive if the feature holds, negative if not—that uniquely identifies c's equivalence class:

```
lit : Feature → Carrier → PredProg
lit f c = if eval-feature f c then feat f else ¬p (feat f)

fingerprint-list : List Feature → Carrier → PredProg
fingerprint-list []       _ = truep
fingerprint-list (f ∷ fs) c = lit f c ∧p fingerprint-list fs c

fingerprint : Carrier → PredProg
fingerprint = fingerprint-list all-features
```

The fingerprint satisfies three key properties:

```
fingerprint-refl     : ∀ c → eval (fingerprint c) c ≡ true

fingerprint-sound    : ∀ ₁c ₂c → FeaturesExhaustive →
  eval (fingerprint ₁c) ₂c ≡ true → AllFeatAgree ₁c ₂c

fingerprint-complete : ∀ ₁c ₂c →
  AllFeatAgree ₁c ₂c → eval (fingerprint ₁c) ₂c ≡ true
```

Together: the fingerprint of c_1 evaluates to `true` on c_2 *if and only if* c_1 and c_2 are in the same equivalence class. Here `FeaturesExhaustive` asserts that every feature appears in `all-features`: this is needed for the soundness direction (exhaustive features ensure no "hidden" feature can distinguish carriers that agree on listed features).

12.2 Synthesized Predicate

The synthesized predicate is the disjunction of fingerprints for representatives where the target is true:

```
dsl-synth : (Carrier → Bool) → List Carrier → PredProg
dsl-synth target []         = falsep
dsl-synth target (r ∷ reps) =
  if target r then fingerprint r ∨p dsl-synth target reps
              else dsl-synth target reps
```

Correctness is established in two directions:

- **Soundness** (`dsl-sound`): If `eval (dsl-synth target reps) c ≡ true`, then some fingerprint in the disjunction matches c, so c is equivalent to a representative r with `target r ≡ true`; by `FeatureRespects`, `target c ≡ true`.
- **Completeness direction** (`dsl-complete-eval`): If `target c ≡ true`, then `Covers` provides a representative r with `AllFeatAgree r c` and therefore `target r ≡ true`; the fingerprint of r is in the disjunction and matches c.

Combining both directions yields the full correctness theorem:

```
dsl-correct : ∀ target reps c →
  FeaturesExhaustive → FeatureRespects target → Covers reps →
  eval (dsl-synth target reps) c ≡ target c
```

12.3 The DSL Completeness Theorem

The existence statement follows immediately from `dsl-correct`:

```
dsl-complete : ∀ target (reps : List Carrier) →
  FeaturesExhaustive → FeatureRespects target → Covers reps →
  ∃[ p ] MatchesAll p target
```

Theorem 20 (DSL-Complete)**.** *For any target function that respects the feature equivalence (`FeatureRespects`), the Boolean `PredProg` DSL contains a program that matches the target everywhere.*

Remark 12 (Closing the Loop)*.* DSL Completeness pairs with the preceding results to give a definitive characterization of the synthesis problem:

1. **Expressiveness** (`dsl-complete`): The DSL always contains the correct answer.
2. **Sufficiency** (`sufficient-cegis`): $|C/\sim|$ observations guarantee CEGIS finds it.
3. **Necessity** (`lower-bound`): Fewer observations cannot guarantee correctness.
4. **Online** (`refine-commutes`): Observations can arrive in any order.

Together: *"For any feature-respecting target, the system is guaranteed to synthesize the correct program from exactly $|C/\sim|$ observations, in any order."*

The fingerprint construction is not intended as a practical synthesis algorithm—it produces a DNF of exponential size. Its role is *existential*: it guarantees that the search space is nonempty, so CEGIS (which explores the enumerated version space) is searching in the right place. In practice, the enumerated version space (from `initial-vs`) often contains much smaller programs that match the target.

13 EC-Specific Instantiation

The generic `PredicateDSL` is instantiated differently for each Environment Class. This section describes the three instantiations implemented in the codebase; the code is in separate modules and shown here via illustrative excerpts.

13.1 Finite Deterministic MDP Synthesis

For FDMDPs, `Carrier = State`, and features are state-component predicates. The synthesis goal is a `RankModel`: a mapping from action pairs to `PredProg` terms.

```
module FDMDPSynthesis (State Action : Set)
  (step : State → Action → State × ℕ) (all-actions : List Action) where

  module WithStateFeatures (Feature : Set)
    (eval-feature : Feature → State → Bool) where

    open PredicateDSL State Feature eval-feature public

    record RankModel : Set where
      field prefer : Action → Action → PredProg

    rank-eval : RankModel → State → Action → Action → Bool
    rank-eval m s a b = eval (RankModel.prefer m a b) s

    -- The synthesis-to-CSHRL bridge:
    ModelPreserves : RankModel → Set
    ModelPreserves m = ∀ a b s →
      rank-eval m s a b ≡ true →
      action-value s a ≤ₛ action-value s b

    module WithCorrectModel (m : RankModel) (preserves : ModelPreserves m) where
      instance
        SynthesizedHomo : CoindHomo
        SynthesizedHomo = record
          { ≤ₐ__ = λ s a b → rank-eval m s a b ≡ true
          ; preserves = preserves }
```

The key bridge: a `RankModel` whose `prefer` predicates are proven to preserve action-value ordering automatically yields a `CoindHomo`. The ranking propagation theorem ensures feature-equivalent states get the same ranking:

```
    rank-propagates : ∀ (m : RankModel) (₁s ₂s : State) (a b : Action) →
      FeatureEquiv (RankModel.prefer m a b) ₁s ₂s →
      rank-eval m ₁s a b ≡ rank-eval m ₂s a b
    rank-propagates m ₁s ₂s a b = propagation (RankModel.prefer m a b) ₁s ₂s
```

13.2 Combinatorial Placement MDP Synthesis

For CPMDPs, the synthesis target is different: instead of rankings, we synthesize the *environment predicates* `is-dead` and `is-solved` that determine the step function.

```
module PlacementSynthesis (Config : Set) (Action : Set) ... where

  open PredicateDSL Config QFeature eval-qfeature

  -- The synthesized predicates ARE the step function
  is-dead-prog : PredProg
  is-dead-prog = any-feat all-attack-feats

  is-solved-prog : PredProg
  is-solved-prog = feat (length-is N)

  synth-is-dead   : Config → Bool = eval is-dead-prog
  synth-is-solved : Config → Bool = eval is-solved-prog

  -- Open the EC with synthesized predicates:
  open CombinatorialPlacementMDP Config Action
    synth-is-dead synth-is-solved place solved-reward all-actions C0 N
```

This is the "synthesis on the critical path" pattern: without CEGIS producing `is-dead-prog` and `is-solved-prog`, there is no step function, and `find-policy` cannot operate. The synthesized `PredProg` terms *are* the environment dynamics.

The CPMDP class itself scales well beyond N-Queens: Section 14.3 instantiates it at full 9×9 Sudoku (horizon 43), where the trace-bridge CoindHomo is obtained symbolically for

every state of a 9^{43}-leaf game, and the class has been ported to Rocq and Lean 4 (kernel item T13) with the Sudoku instances re-verified in both systems.

13.3 Stochastic Finite MDP Synthesis

The third instantiation extends synthesis to *stochastic* MDPs, where the step function returns a distribution over successor states:

```
step : State → Action → Dist (State × Reward)
```

The key insight is that the `PredicateDSL` is entirely *independent* of whether transitions are deterministic or stochastic. The DSL operates on *states*, not transitions. All generic theorems—propagation, dissolution, feature sufficiency, online synthesis, the tight sample complexity bound, and DSL completeness—apply *unchanged*:

```
module SFDMDPSynthesis (State : Set) (Action : Set) (Reward : Set)
  (step : State → Action → Dist (State × Reward)) ... where

  open PredicateDSL State Feature eval-feature public
  -- ALL of: propagation, refine-equiv, dissolution, sufficient-cegis,
  --         refine-commutes, replay-redundant, LearningBridge,
  --         same-obs-same-vs, survivor-transfer, tight-bound
  -- are available immediately, with zero additional proof.
```

The *only* difference from the deterministic case is in the observation-generation layer: ranking observations are derived from *expected trace comparison* rather than deterministic trace comparison.

```
  expected-trace-compare : State → Action → Action → ℕ → Bool
  expected-trace-compare s a b k =
    expected-trace-action s a k ≤ₜᵇ expected-trace-action s b k
```

The preservation condition uses *lexicographic* stream dominance on expected action-values (`≤ₛ-lex`) rather than pointwise stream dominance, yielding a `StochasticCoindHomo` instead of a deterministic `CoindHomo`:

```
  ModelPreservesLex : RankModel → Set
  ModelPreservesLex m = ∀ a b s →
    RankHolds m s a b →
    expected-action-value s a ≤ₛ-lex expected-action-value s b
```

Remark 13 (Four Synthesis Patterns). The framework now supports four synthesis patterns across three EC types:

1. **FDMDP (ranking synthesis):** Deterministic transitions, synthesize ranking predicates → `CoindHomo`.
2. **CPMDP (environment synthesis):** Deterministic transitions, synthesize environment predicates → step function.
3. **SFMDP (stochastic ranking synthesis):** Probabilistic transitions, synthesize ranking predicates from expected traces → `StochasticCoindHomo`.
4. **SFMDP-FOSD (distributional ranking synthesis):** Probabilistic transitions, synthesize ranking predicates from FOSD comparison of marginal CDFs → `FOSDCoindHomo` → `StochasticCoindHomo` (via the FOSD-to-Lex Bridge).

All four share the *same* generic theory: `PredicateDSL`, `CEGIS`, propagation, dissolution, sufficiency, online synthesis, the tight bound, and DSL completeness. The only variation is in how domain-specific feedback maps to `PredObs` observations—precisely the interface that the `LearningBridge` abstracts. The FOSD pattern adds a convergence guarantee (`fosd-obs-sound`, `model-obs-consistent`) and universality across monotone utility functions.

14 End-to-End: From Observations to Verified Policies

We present seven end-to-end demonstrations that exercise the full synthesis pipeline. Each is a self-contained Agda module in the codebase, compiled with `--safe --guardedness` and zero postulates.

14.1 Demo 1: 1D Maze (FDMDP)

The simplest demonstration: a three-state maze $P_0 \to P_1 \to P_2$ (goal) with two actions (`Fwd`, `Bwd`).

```
data State : Set where P0 P1 P2 : State
data Action : Set where Fwd Bwd : Action

step : State → Action → State × ℕ
step P0 Fwd = (P1 , 0)    step P1 Fwd = (P2 , 1)    step P2 Fwd = (P2 , 1)
step P0 Bwd = (P0 , 0)    step P1 Bwd = (P0 , 0)    step P2 Bwd = (P1 , 0)
```

Step 1: Observe. Five trace comparisons produce `PredObs` lists:

```
obs-≤bwdfwd : List PredObs = (P0 , true) ∷ (P1 , true) ∷ (P2 , true) ∷ []
obs-≤fwdbwd : List PredObs = (P0 , false) ∷ (P1 , false) ∷ []
```

Step 2: Synthesize. CEGIS refines the version space (3 candidates: `truep`, `falsep`, `feat is-goal`) and returns the consistent candidate:

```
synth-≤bwdfwd : synth-rank-pred 0 obs-≤bwdfwd ≡ just truep  = refl
synth-≤fwdbwd : synth-rank-pred 0 obs-≤fwdbwd ≡ just falsep = refl
```

CEGIS determines: "Bwd is always ranked below Fwd" (`truep`) and "Fwd is never ranked below Bwd" (`falsep`).

Step 3: Verify. A domain-specific proof establishes that the synthesized ranking preserves action-values. This is the one step requiring domain knowledge:

```
synth-preserves : ModelPreserves synth-rank
```

Step 4: Construct. The verified ranking automatically yields a `CoindHomo`:

```
open WithCorrectModel synth-rank synth-preserves
-- SynthesizedHomo : CoindHomo  (provided by the module)
```

Step 5: Extract & Execute. The policy selects the highest-ranked action and is rolled out:

```
synth-policy : State → Action
synth-policy s with rank-eval synth-rank s Bwd Fwd
... | true = Fwd    ... | false = Bwd

test-trajectory : run-synth P0 3 ≡ (Fwd,P1) ∷ (Fwd,P2) ∷ (Fwd,P2) ∷ []  = refl
```

Step 7: Cross-Validation. The EC's `find-policy` independently computes the same policy:

```
finder-agrees : ∀ s → Finder.find-policy s 2 ≡ synth-policy s
finder-agrees P0 = refl  ;  finder-agrees P1 = refl  ;  finder-agrees P2 = refl
```

14.2 Demo 2: 4-Queens (CPMDP)

Here synthesis is on the *critical path*: the `PredProg` predicates define the step function.

Features. Six pairwise attack features (`attacks 0 1`, `attacks 0 2`, ..., `attacks 2 3`) and a length feature:

```
data QFeature : Set where
  attacks   : ℕ → ℕ → QFeature
  length-is : ℕ → QFeature
```

Synthesized predicates. The `is-dead` predicate is a disjunction of all attack features; `is-solved` tests that the board has *N* queens:

```
is-dead-prog   : PredProg = any-feat all-attack-feats    -- 6 features
is-solved-prog : PredProg = feat (length-is N)

synth-is-dead   : Config → Bool = eval is-dead-prog
synth-is-solved : Config → Bool = eval is-solved-prog
```

EC instantiation. The EC is opened with the synthesized predicates—no hand-crafted transition logic:

```
open CombinatorialPlacementMDP Config Action
  synth-is-dead synth-is-solved place solved-reward all-actions C0 N
```

Solution. `find-policy` produces the optimal 4-Queens solution:

```
test-solution : run-policy (Ongoing []) N N ≡ C1 ∷ C3 ∷ C0 ∷ C2 ∷ []  = refl
```

Partial completion. The synthesized EC can complete *any* partial optimal placement:

```
test-from-1 : run-policy (Ongoing (1 ∷ [])) N 3 ≡ C3 ∷ C0 ∷ C2 ∷ []  = refl
test-from-2 : run-policy (Ongoing (1 ∷ 3 ∷ [])) N 2 ≡ C0 ∷ C2 ∷ []   = refl
test-from-3 : run-policy (Ongoing (1 ∷ 3 ∷ 0 ∷ [])) N 1 ≡ C2 ∷ []    = refl

-- Alternative solution discovered from a different starting point:
test-alt : run-policy (Ongoing (2 ∷ [])) N 3 ≡ C0 ∷ C3 ∷ C1 ∷ []     = refl
```

CEGIS demo. With a 2-feature vocabulary, 2 observations suffice to pin the attack predicate:

```
check-vs-after-both :
  length (cegis-loop (initial-vs 0) (dead-obs ∷ alive-obs ∷ [])) ≡ 1  = refl
```

Propagation demo. Boards `[0,1]` and `[3,4]` have the same diagonal attack pattern; observing one classifies the other for free:

```
refine-same : refine (initial-vs 0) (₁board , true)
            ≡ refine (initial-vs 0) (₂board , true)
refine-same = refine-equiv (initial-vs 0) ₁board ₂board true λ( { ... → refl })
```

14.3 Demo 16: Sudoku (CPMDP)

Sudoku exercises the CPMDP pattern twice: the full synthesis pipeline at 4×4 (`src/CSHRL/Tasks/Synthesized/SudokuE2E.agda`), and the placement theory at full 9×9 scale (38 givens, horizon 43, a 9^{43}-leaf raw game tree) via direct instantiation in the reference implementation and the cross-system ports.

Synthesized predicates (4×4). Exactly as in the Queens demo, the environment dynamics *are* PredProg terms. The feature vocabulary is the 56 pairwise `conflict` features over the `sees` relation (row, column, box) plus a `filled-is` counter:

```
sees p q = (div4 p ≡ᵇ div4 q) ∨ (mod4 p ≡ᵇ mod4 q)
  ∨ ((div2 (div4 p) ≡ᵇ div2 (div4 q)) ∧ (div2 (mod4 p) ≡ᵇ div2 (mod4 q)))

is-dead-prog   : PredProg = any-feat conflict-feats     -- 56 features
is-solved-prog : PredProg = feat (filled-is num-empty)

open CombinatorialPlacementMDP Config Action
  (eval is-dead-prog) (eval is-solved-prog)
  place solved-reward all-actions D1 num-empty
```

The Finder then completes the puzzle through the synthesized dynamics, verified end to end by `refl`:

```
test-solution : run-policy (Ongoing []) 8 8
  ≡ D2 ∷ D3 ∷ D3 ∷ D2 ∷ D1 ∷ D4 ∷ D4 ∷ D1 ∷ []
```

CEGIS pins the conflict predicate from two board observations (one dead, one alive), and propagation transfers observations between boards with the same conflict pattern—the Queens demo's structure, unchanged.

Solvability and doom are knowable. The EC's capability profile `solve` certifies both directions computationally. From the empty board the full reward is reachable (the puzzle *is* solvable); a wrong digit leads to a successor whose profile is identically zero (the branch is *doomed*):

```
test-solvable : solve (Ongoing []) num-empty ≡ solved-reward   = refl
test-doomed   : solve (₁proj (step (Ongoing []) D3)) num-empty ≡ 0 = refl
```

The subtler phenomenon—a state that is *not yet dead* but already unwinnable—is certified in the Queens demo: board [0] has no attack, yet no 4-Queens solution starts in column 0, and `solve (Ongoing (0 ∷ [])) N ≡ 0` holds by `refl`. Horizon sufficiency (§13.2) upgrades this finite-depth zero to *all* depths: doom detected at the horizon is doom forever.

Full 9×9 scale. The 9×9 instances (`src/CSHRL/Tasks/Verified/Sudoku9.agda` and the ports) instantiate the same EC with hand-written predicates of the same shape (a disjunction of `sees`-conflicts) rather than the CEGIS pipeline—the point at this scale is the placement theory itself. The trace bridge (`WithTraceBridge`) yields the CoindHomo for the Finder's trace ranking at *every* state of the 9×9 MDP. Crucially, this proof is symbolic: its cost is independent of the 9^{43}-leaf game tree, because the bridge argues through the binary structure and monotonicity of `solve` rather than by enumeration.

Policy extraction by normalization is where scale bites: certifying a rollout by `refl` means the type checker itself executes the depth-43 lookahead search. The 9×9 *instance* is therefore designed so that every empty cell, in row-major fill order, is directly conflict-forced—all eight wrong digits immediately collide with an already-filled peer—making the search tree the normalizer must walk linear (~400 nodes). This is a property of the chosen puzzle, not of the solver: the Finder's definition and its CoindHomo cover the general MDP with all nine actions at every node. Two `refl` certificates pin the rollout:

```
-- every cell is conflict-forced to the unique solution digit
test-forced : map surviving (upTo num-empty) ≡ map λ( d → d ∷ []) solution

-- the Finder's policy fills the final cells (shallow lookahead)
test-tail-rollout :
  run-policy (Ongoing (take' 40 solution)) 3 3 ≡ A5 ∷ A6 ∷ A8 ∷ []
```

Dead actions produce all-zero traces while the surviving action's trace carries the solved reward within lookahead, so forcedness plus the trace bridge determine the full rollout mathematically.

Cross-system verification. The placement class, its binary-solve theory, and the trace bridge are ported to Rocq and Lean 4 as kernel item T13 (`ports/rocq/theories/Placement.v`, `ports/lean/CSHRL/Placement.lean`), with one improvement over the Agda reference: horizon sufficiency is discharged *generically* for sized games (each placement increases a size measure by one; solving means reaching the horizon), so the Sudoku instances carry no

proof obligations beyond their static description. Each system then plays to its evaluator's strengths: Rocq verifies the *full 43-step 9×9 rollout* by `vm_compute` (call-by-value with perfect sharing); Lean verifies the 4×4 rollout by kernel reduction and the 9×9 forcedness certificate by kernel `decide`; Agda verifies forcedness and the tail rollout by `refl`. The Lean axiom check confirms T13 and both Sudoku CoindHomos depend only on `propext` and `Quot.sound`; the Rocq development is closed under the global context (`Print Assumptions`).

14.4 Demo 4: Biased Bandit (SFMDP)

The final demonstration completes the E2E trifecta by exercising the *stochastic* synthesis pipeline. The Biased Bandit is a single-state, two-armed bandit where Arm A pays reward 1 with probability 2/3 and Arm B with probability 1/3:

```
data State : Set where  Playing : State
data Action : Set where  ArmA ArmB : Action

step : State → Action → Dist (State × ℕ)
step Playing ArmA = bernoulli (Playing , 1) 2 (Playing , 0) 1   -- 2:1 odds
step Playing ArmB = bernoulli (Playing , 1) 1 (Playing , 0) 2   -- 1:2 odds
```

Synthesis framework. We open `SFDMDPSynthesis` (the stochastic instantiation from §13.3), which provides `expected-action-value`, `≤ₛ-lex`, and `StochasticCoindHomo`:

```
open SFDMDPSynthesis State Action ℕ step ≤__ ≤_?_ ≤-refl _+_ _*_ 0 □__ 0
  all-actions ArmB 3
```

Features. The bandit is stateless: a single trivial feature (`is-playing`, always true) suffices. This gives $|C/{\sim}| = 1$—one equivalence class—so CEGIS needs exactly one observation per action pair (the tight bound).

```
data BanditFeature : Set where  is-playing : BanditFeature
eval-bandit-feature : BanditFeature → State → Bool
eval-bandit-feature is-playing Playing = true

open WithStateFeatures BanditFeature eval-bandit-feature
open WithCEGIS (is-playing ∷ [])
```

Step 1: Observe. Two expected-trace comparisons at `Playing`:

```
obs-≤ba : List PredObs = (Playing , true) ∷ []    -- ArmB ≤ ArmA: yes
obs-≤ab : List PredObs = (Playing , false) ∷ []   -- ArmA ≤ ArmB: no
```

Step 2: Synthesize. CEGIS refines the version space (3 candidates) and returns:

```
synth-≤ba : synth-rank-pred 0 obs-≤ba ≡ just truep   = refl    -- ArmA always dominates
synth-≤ab : synth-rank-pred 0 obs-≤ab ≡ just falsep   = refl    -- ArmB never dominates
```

Step 3: Verify. The preservation proof uses lexicographic stream dominance. Arm A's expected head reward is 2 (unnormalized), Arm B's is 1:

```
synth-preserves : ModelPreservesLex synth-rank
synth-preserves ArmA ArmB s ()        -- eval falsep s ≡ true is absurd≤
head (synth-preserves ArmB ArmA Playing _) = ≤ss ≤zn         -- 1 ≤ 2≤
tail (synth-preserves ArmB ArmA Playing _) ()                -- 1 ≡ 2 absurd
```

The reflexive cases (`ArmA ArmA`, `ArmB ArmB`) follow from `≤ₛ-lex-refl`.

Step 4: Construct. Opening `WithCorrectModel` yields `SynthesizedStochasticHomo`—a verified `StochasticCoindHomo` constructed from the synthesized ranking.

Step 5: Extract & Confirm. The synthesized policy selects `ArmA`, matching the EC's independent `find-policy`:

```
test-synth-Playing : synth-policy Playing ≡ ArmA     = refl
finder-agrees : find-policy Playing 3 ≡ synth-policy Playing  = refl
```

Dissolution check. With $|C/\sim| = 1$, the version space collapses to a singleton after each observation:

```
vs-after-≤ba : length (cegis-loop (initial-vs 0) obs-≤ba) ≡ 2   = refl
vs-after-≤ab : length (cegis-loop (initial-vs 0) obs-≤ab) ≡ 1   = refl
```

Remark 14 (Stochastic Synthesis Modularity). This demo confirms the central modularity claim: the `PredicateDSL`, CEGIS, version space, propagation, dissolution, and tight bound are *identical* to the deterministic case. Only the observation layer changes—from deterministic trace comparison to expected-trace comparison. The type of the preservation proof changes from `ModelPreserves` (pointwise $\leq_s$) to `ModelPreservesLex` (lexicographic $\leq_s$-lex), and the resulting homomorphism is `StochasticCoindHomo` instead of `CoindHomo`, but *no synthesis theorem needed modification*.

Extended demos. Extended demonstrations (8-Queens scaling, N-Queens feature discovery, FrozenLake, Cliff Walking, Auto-FTL, Taxi, stochastic variants, sample-efficiency analysis, and Cross-Pair Propagation) are in Appendix A.

15 Discussion

15.1 The Propagation Theorem as Generalization Principle

The Propagation Theorem (Theorem 1) plays a role analogous to *inductive bias* in machine learning. By restricting the hypothesis space to programs over features (rather than arbitrary functions on carriers), we obtain a formal generalization guarantee: any predicate consistent with observations at one carrier is automatically consistent at all feature-equivalent carriers.

This is not an approximation or a statistical guarantee—it is a *theorem*, holding for all carriers and all predicate programs. The three quantified consequences (Refine-Equiv, Observation Subsumption, Exploration Dissolution) make this guarantee precise and actionable.

15.2 Six Synthesis Patterns

The framework supports six synthesis patterns across deterministic, stochastic, distributional, and compositional settings:

- **FDMDP pattern (ranking synthesis):** The environment's step function is known. Synthesis produces ranking predicates that, when verified, yield a `CoindHomo`. The synthesis is "nice to have"—the EC can compute the policy independently. The value of synthesis here is *interpretability* and *generalization*: the synthesized ranking explains *why* Fwd dominates Bwd (because it leads to a rewarding state), whereas the EC's lookup table merely records *that* it does.
- **CPMDP pattern (environment synthesis):** The environment's step function depends on `is-dead`/`is-solved` predicates that are not given a priori. Synthesis is on the *critical path*: without CEGIS producing the predicates, there is no step function, and `find-policy` cannot operate. This is a stronger form of synthesis—the program *defines* the environment, not just the ranking.
- **SFMDP pattern (stochastic ranking synthesis):** The step function returns *distributions* over successor states. Synthesis produces ranking predicates from expected

trace comparisons, yielding a `StochasticCoindHomo`. This demonstrates the framework's modularity: the generic `PredicateDSL` theory applies unchanged—only the observation-generation layer adapts to expected traces.

- **FOSD pattern (distributional ranking synthesis):** The same stochastic step function, but observations compare marginal CDFs rather than expected values. A ranking proven to preserve FOSD yields a `FOSDCoindHomo` (see [24]), which the FOSD-to-Lex Bridge automatically promotes to a `StochasticCoindHomo`. The FOSD pattern is strictly stronger: the synthesized ranking is optimal for *all monotone utility functions* (risk-averse, risk-neutral, risk-seeking), not just expected-value maximizers. The learning convergence theorem (`fosd-obs-sound`) ensures that FOSD observations drive CEGIS refinement correctly, and `fosd?-complete` guarantees no valid FOSD relationship is missed.
- **SD hierarchy pattern (parameterized distributional ranking synthesis):** The SD hierarchy [24] generalizes the FOSD pattern to k-th order. When FOSD fails (CDFs cross), an `SDCoindHomo` k for $k \geq 1$ may still be established. The master bridge (`sd-homo→ev-lex`) ensures that *any* SD level yields EV-stream optimality. The hierarchy subsumption (`sd-homo-subsumes`) means a single FOSD verification at k=0 automatically provides the stronger guarantee at all levels—but when FOSD fails, the framework can "fall back" to SOSD (k=1) or TOSD (k=2), accepting the weaker guarantee that the ranking is optimal for risk-averse or prudent agents respectively.
- **Compositional pattern (modular ranking synthesis):** The compositional algebra enables a "divide and conquer" approach. Rather than verifying a monolithic ranking on a large composite environment, the framework decomposes it into independent components, verifies each in isolation, and composes the proofs via `product-ranking`, `conv-product`, `sum-ranking`, or `scale-ranking`. The closure properties (`SD-++`, `SD-scale`, `FOSD-conv`) guarantee that composition preserves SD[k] at any level. This pattern is orthogonal to the five above: each component can use *any* of the other patterns, and the compositional algebra combines the results.

15.3 Computational Considerations

The 8-Queens demo revealed a practical tension: `PredProg` evaluation via pattern matching on AST nodes incurs overhead in Agda's normalizer compared to direct Boolean computation. The hybrid approach (prove equivalence, then use the faster implementation) resolves this while maintaining formal correctness.

For production systems, one could:

1. Synthesize the `PredProg` term via CEGIS.
2. Prove its equivalence to a compiled implementation (as in `Queens8E2E`).
3. Use the compiled implementation for runtime evaluation.

The proof of equivalence is a one-time cost, and the resulting system has the interpretability of `PredProg` with the performance of native code.

15.4 Online Synthesis and the Learning Loop

Classical CEGIS is presented as a batch algorithm with a fixed observation set. The online synthesis results (§10) reveal that this presentation is a matter of convention, not necessity. Commutativity (`refine-commutes`) means the version space is a function of the *multiset* of observations, not their sequence. Replay redundancy means the version space is actually a function of the *set*. Together, these properties make the synthesis engine a *monotone, idempotent, commutative* operator on version spaces—precisely the algebraic structure needed for conflict-free replicated data types (CRDTs) [22].

This algebraic structure has practical consequences: multiple learning agents can submit observations to a shared synthesizer without coordination, checkpoints can be replayed

without corruption, and the exploration strategy (depth-first, breadth-first, violation-driven) is provably irrelevant to correctness. The `LearningBridge` makes this integration explicit, reducing the gap between the learning loop and synthesis from an informal interface to a verified equivalence.

15.5 Demo 15: GamblersRuin — FOSD Synthesis in a Richer Environment

We extend FOSD-based synthesis beyond the single-state BiasedBandit to **GamblersRuin**: 3 states (Ruin, Middle, Goal), 2 actions (Bet, Quit). From Middle, Bet yields 50% chance of Goal (reward 1) and 50% Ruin (reward 0); Quit yields 100% Middle (reward 0). The optimal policy at Middle is Bet (higher expected value and FOSD-dominance).

The module `Synthesis.FOSDStochasticFiniteMDP` provides the same interface as `StochasticFiniteMDP` but generates observations via `fosd-compare` instead of expected-trace comparison. FOSD comparison requires equal total weights; GamblersRuin's `pure` for Quit has weight 1 while Bet has weight 2. We use `step-fosd` that scales Quit to weight 2 (same semantics: 100% (Middle,0)) so `fosd?` can compare.

With feature `is-middle` (true only at Middle), CEGIS synthesizes a predicate for prefer(Quit, Bet) from the observation (`Middle, fosd-compare Middle Quit Bet 0`). At depth 0, Quit FOSD≤ Bet (Bet dominates); the synthesized predicate holds at Middle, yielding the optimal policy: Bet.

```
test-fosd-≤quitbet : fosd-compare Middle Quit Bet 0 ≡ true
test-synth-succeeds : synth-≤quitbet ≡ just truep
```

The `ModelPreservesFOSD` type and `WithCorrectModel` connect a verified `RankModel` to `FOSDCoindHomo`: when a synthesized ranking preserves pointwise FOSD at every timestep, it yields a coinductive homomorphism optimal for *all* monotone utility functions. The `WithLearningBridge` module provides the same learning–synthesis integration as `StochasticFiniteMDP`: `sample-to-obs` maps samples to FOSD observations via `fosd-compare`, `synth-learn-step` and `synth-learn-batch` process samples incrementally, and `extract-rank-model` builds a `RankModel` from the combined learner state. The Learning Bridge is demonstrated end-to-end in GamblersRuin: a sample (`Middle, Quit, Bet`) is processed by `synth-learn-batch`, and `extract-rank-model` yields a `RankModel` that correctly prefers Bet over Quit at Middle (`test-extracted-correct`).

15.6 FOSD-Based Synthesis

The expected-value stochastic synthesis pipeline (§13.3) compares actions by $\mathbb{E}[\text{trace}(s,a)] \leq \mathbb{E}[\text{trace}(s,b)]$. This suffices for expected-value maximizers, but collapses the full reward distribution to a single scalar. A strictly stronger comparison is **First-Order Stochastic Dominance (FOSD)**: action a FOSD-dominates b at state s iff for all thresholds r and all timesteps n, $P(\text{reward}_n(s,a) \leq r) \leq P(\text{reward}_n(s,b) \leq r)$.

The FOSD theory—CDF computation, the decidable FOSD comparator (`fosd?`) with soundness and completeness, the coinductive lift (`PointwiseFOSD`, `StreamFOSD`, Stochastic Isomorphism), the `FOSDCoindHomo` record, and the FOSD-to-Lex bridge (`fosd-homo→ev-lex`)—is developed in full in the companion paper [24]. Here we focus on how this theory connects to the synthesis pipeline.

15.6.1 Synthesis: FOSD Learning Convergence

`Synthesis.FOSDStochasticFiniteMDP` connects FOSD to the CEGIS learning loop. The `FOSDConvergence` module takes a `RankModel` m and a proof `ModelPreservesFOSD` m and derives:

```
fosd-obs-sound : ∀ a b s k →
  RankHolds m s a b →
  fosd-compare s a b k ≡ true

model-obs-consistent : ∀ a b s k →
```

```
    RankHolds m s a b →
    consistent (RankModel.prefer m a b)
               (s , fosd-compare s a b k) ≡ true

  fosd-convergence : FOSDConvergenceGuarantee
  fosd-convergence = combined-convergence , fosd-obs-sound
```

`fosd-obs-sound` uses `fosd?-complete` to prove that if the model ranks $a \leq b$ at state s, then `fosd-compare`—which calls `fosd?` on marginal reward distributions—returns `true` at every depth k. `model-obs-consistent` lifts this to the CEGIS version space: the correct predicate's observation always passes the `consistent` check, so it survives every refinement round. `fosd-convergence` packages both with version-space monotonicity (`combined-convergence`) into a single convergence guarantee:

> *If a ranking model preserves FOSD, then (1) the version space shrinks monotonically, and (2) every FOSD observation generated from the model is truthful.*

The key observation for synthesis is that the `PredicateDSL` and CEGIS are entirely *unchanged*. Only the observation-generation layer differs: instead of comparing expected traces (a scalar), we compare marginal CDFs (a function of the threshold). A FOSD observation at state s asserts $\forall r.\, \text{CDF}(s, b, r) \leq \text{CDF}(s, a, r)$—which maps to the *same* `PredObs = Carrier × Bool` format. All propagation, dissolution, and tight-bound theorems apply without modification.

15.6.2 FOSD Demos

Three environments exercise the full FOSD pipeline:

- **BiasedBandit** (`Tasks.Stochastic.BiasedBanditFOSD`): Single-state, 2-armed bandit where ArmA (2/3 reward) FOSD-dominates ArmB (1/3 reward). Full `PointwiseFOSD` at all depths, `FOSDCoindHomo` instance, and the bridge theorem (`fosd→ev`) confirmed.
- **CoinFlip** (`Tasks.Stochastic.CoinFlipFOSD`): Two states (Heads, Tails), 2 actions (Flip, Hold). Demonstrates multi-state FOSD with terminal state handling: `PointwiseFOSD` proved at all depths for both states, `FOSDCoindHomo` instance, and `ModelPreservesFOSD` connecting the rank model to the synthesis pipeline.
- **GamblersRuin** (`Tasks.Stochastic.GamblersRuinFOSDSynth`, Demo 15, §15.5): 3 states, end-to-end FOSD synthesis with `ModelPreservesFOSD`, `WithCorrectModel`, and `WithLearningBridge`—t full pipeline from observations to verified `FOSDCoindHomo`.

15.7 The Stochastic Dominance Hierarchy

The **Stochastic Dominance Hierarchy** generalizes FOSD to a parameterized family of orderings SD[k], each capturing a broader class of utility functions: SD[0] = FOSD (all monotone utilities), SD[1] = SOSD (risk-averse agents), SD[2] = TOSD (prudent agents), and so on. The hierarchy is built on iterated prefix sums of the CDF, with a one-line subsumption proof (SD[k] ⇒ SD[k+1] via `prefix-sum-mono`) and a master bridge showing that any SD level implies expected-value optimality. The full theoretical development—`Probability.SD`, `Core.SD`, `SDCoindHomo`, the subsumption chain, and the strict SOSD-but-not-FOSD separation example—is presented in [24].

For synthesis, the key consequence is that when FOSD fails (CDFs cross), an `SDCoindHomo` k for $k \geq 1$ may still be established. The master bridge (`sd-homo→ev-lex`) ensures that *any* SD level yields EV-stream optimality, so the synthesis engine can "fall back" to SOSD or TOSD while retaining correctness guarantees for the corresponding utility-function class.

15.8 Compositional Ranking Algebra

The SD hierarchy (§15.7, see also [24]) establishes a parameterized family of distributional orderings. The **Compositional Ranking Algebra** proves that these orderings are closed under natural operations on distributions—enabling *modular verification*: verify each

component in isolation, then compose the proofs automatically. Here we present the full proof infrastructure; [24] gives the condensed definitions and summary.

15.8.1 Foundation: Probability.Compose

The core algebraic insight is that `prefix-sum` is a *linear operator*. Two distributivity lemmas establish this:

```
prefix-sum-+ : ∀ (f g : ℕ → ℕ) r →
  prefix-sum λ( r → f r + g r) r ≡ prefix-sum f r + prefix-sum g r

prefix-sum-* : ∀ c (f : ℕ → ℕ) r →
  prefix-sum λ( r → c * f r) r ≡ c * prefix-sum f r
```

Since `sd-weight` is iterated prefix-sum, it inherits linearity. An extensionality lemma (`prefix-sum-ext`: pointwise-equal functions have equal prefix sums) bridges inductive hypotheses across recursive calls:

```
sd-weight-++ : ∀ k (₁d ₂d : Dist ℕ) r →
  sd-weight k (₁d ++ ₂d) r ≡ sd-weight k ₁d r + sd-weight k ₂d r

sd-weight-scale : ∀ k (d : Dist ℕ) r c →
  sd-weight k (scale c d) r ≡ c * sd-weight k d r
```

With these structural lemmas, the closure properties are immediate. Distribution concatenation (++) preserves SD[k] because sd-weight distributes over ++ and addition is monotone:

```
SD-++ : ∀ k μ₁{ ν₁ μ₂ ν₂ : Dist ℕ} →
  μ₁ SD[ k ≤] ν₁ → μ₂ SD[ k ≤] ν₂ →
  μ₁( ++ μ₂) SD[ k ≤] ν₁( ++ ν₂)

SD-scale : ∀ k c μ{ ν : Dist ℕ} →
  μ SD[ k ≤] ν → scale c μ SD[ k ≤] scale c ν
```

Both proofs are one-liners after rewriting: `SD-++` uses `+-mono-≤`, and `SD-scale` uses `*-monoʳ-≤`. Specializing to k=0 gives `FOSD-++` and `FOSD-scale`.

15.8.2 Composition Theorems: Core.Compose

`Core.Compose` lifts the distribution-level closure to an algebra of *verified rankings*. The central abstraction is:

```
record VerifiedRanking
  (State Action : Set)
  (marginal : State → Action → ℕ → Dist ℕ)
  (k : ℕ) : ₁Set where
  field
    ≤ₐ__      : State → Action → Action → Set
    preserves : ∀ a b s → ≤ₐ__ s a b →
                ∀ n → marginal s a n SD[ k ≤] marginal s b n
```

This is the abstract counterpart of `SDCoindHomo`: a ranking bundled with a proof that it preserves SD[k] on marginal rewards. The algebra provides five operations:

Hierarchy subsumption. Any ranking verified at level k is automatically verified at the weaker level k+1 (via `SD-subsumes`):

```
ranking-subsumes : VerifiedRanking S A m k → VerifiedRanking S A m (suc k)
```

Product composition. Given two component rankings and a composition operation $\oplus$ on distributions that preserves SD[k], the product ranking—requiring *both* components to rank favorably—is verified for the composite environment:

```
product-ranking :
  (compose : Dist ℕ → Dist ℕ → Dist ℕ) →
  ∀( μ₁{ ν₁ μ₂ ν₂} → μ₁ SD[ k ≤] ν₁ → μ₂ SD[ k ≤] ν₂ →
```

```
      compose μ₁ μ₂ SD[ k ≤] compose ν₁ ν₂) →
    VerifiedRanking ₁S ₁A ₁m k → VerifiedRanking ₂S ₂A ₂m k →
    VerifiedRanking (₁S × ₂S) (₁A × ₂A) (compose-marginals) k
```

Instantiating $\oplus$ with ++ and SD-++ gives the concrete ++-product: marginal rewards merge as mixtures. Instantiating with scale gives scale-ranking: reward amplification preserves verification.

Convolution product. Beyond concatenation, the framework now supports **convolution** (independent sum): given two distributions μ_1 and μ_2 over $\mathbb{N}$, the convolution $\mu_1 * \mu_2$ is the distribution of $X_1 + X_2$ where $X_i \sim \mu_i$ are independent. The key new result is:

```
FOSD-conv : total-weight μ₁ ≡ total-weight ν₁ →
  μ₁ ≤FOSD ν₁ → μ₂ ≤FOSD ν₂ →
  conv μ₁ μ₂ ≤FOSD conv ν₁ ν₂
```

The proof decomposes into two directions. *Kernel direction*: for fixed base d_1, if $\mu_2 \preceq_{\text{FOSD}} \nu_2$ then $\text{conv}(d_1, \mu_2) \preceq_{\text{FOSD}} \text{conv}(d_1, \nu_2)$—this follows from pointwise CDF comparison (gws-mono) after showing FOSD is preserved by additive shifting (FOSD-shift). *Base direction*: for fixed kernel d_2, if $\mu_1 \preceq_{\text{FOSD}} \nu_1$ then $\text{conv}(\mu_1, d_2) \preceq_{\text{FOSD}} \text{conv}(\nu_1, d_2)$—this is the deep result, proved via a **discrete Abel summation** induction (fosd-gws). The induction decomposes a generalized weighted sum $\sum_{(v,w)\in d} w \cdot h(v)$ for non-increasing h by peeling off the last non-zero level, using the FOSD hypothesis at each threshold.

The convolution closure lifts to the full SD hierarchy via FOSD→SD-conv:

```
conv-product :
  (tw : ∀ s a b n → total-weight (₁m s a n) ≡ total-weight (₁m s b n)) →
  VerifiedRanking ₁S ₁A ₁m 0 → VerifiedRanking ₂S ₂A ₂m 0 →
  VerifiedRanking (₁S × ₂S) (₁A × ₂A)
    λ( (₁s,₂s) (₁a,₂a) n → conv (₁m ₁s ₁a n) (₂m ₂s ₂a n)) k
```

While ++-product merges reward lists (a mixture model), conv-product sums independent rewards—the natural combinator for product MDPs where rewards are additive. The equal-total-weight precondition on m_1 reflects a standard normalization that holds in practice for probability distributions.

Sum composition. Two disjoint environments with state spaces $S_1 \uplus S_2$ compose with a *dispatched* ranking: in state $\text{inj}_1\, s_1$, use ranking 1; in state $\text{inj}_2\, s_2$, use ranking 2. Mismatched state/action pairs are excluded by $\bot$ (impossible to construct, so never encountered):

```
sum-ranking :
  VerifiedRanking ₁S ₁A ₁m k → VerifiedRanking ₂S ₂A ₂m k →
  VerifiedRanking (₁S ⊎ ₂S) (₁A ⊎ ₂A) sum-marginal k
```

Composability. The operations compose with each other, demonstrating genuine algebraic structure. For example, composing two components via concatenation and then scaling all rewards:

```
scaled-product : (c : ℕ) →
  VerifiedRanking ₁S ₁A ₁m k → VerifiedRanking ₂S ₂A ₂m k →
  VerifiedRanking (₁S × ₂S) (₁A × ₂A)
    λ( (₁s,₂s) (₁a,₂a) n → scale c (₁m ₁s ₁a n ++ ₂m ₂s ₂a n)) k
scaled-product c ₁vr ₂vr = scale-ranking c (++-product ₁vr ₂vr)
```

This one-line proof demonstrates the compositional nature of the algebra: ++-product composes the marginal rewards, then scale-ranking amplifies them—each step preserving the SD[k] guarantee.

Remark 15 (Practical Significance). The compositional algebra addresses a key scalability challenge: as environments grow in complexity, monolithic verification becomes impractical. The algebra enables a "divide and conquer" strategy:

1. Decompose a complex environment into independent components.
2. Verify each component's ranking in isolation (smaller state/action spaces).

3. Compose the proofs via `product-ranking`, `conv-product`, `sum-ranking`, or `scale-ranking`.

The composed proof is as strong as the component proofs—no additional verification is needed at the composite level. The convolution combinator is particularly natural for product MDPs with additive rewards: it expresses that "summing independent draws from better distributions yields a better sum distribution." This mirrors how software verification handles large systems: verify modules independently, then compose via interfaces.

15.8.3 Demonstration: BiasedBandit × CoinFlip

`CompositionalDemo` exercises all six operations of the algebra on two existing FOSD-verified environments. `VerifiedRanking` records are extracted directly from the `FOSDCoindHomo` records of BiasedBandit (`ArmA` > `ArmB`) and CoinFlip (`Flip` > `Stay`)—the extraction is zero-cost because $\texttt{FOSD}_{\leq}$ and $\texttt{SD[0]}_{\leq}$ are definitionally equal.

The *portfolio* environment (state = $S_1 \times S_2$, action = $A_1 \times A_2$, 4 combinations) is verified by a single call to `++-product`. The product ranking is computationally tested: $(\texttt{ArmB}, \texttt{Stay}) \leq (\texttt{ArmA}, \texttt{Flip})$ is $(\top, \top)$, confirmed by `refl`. CDF values of the composed marginal distributions are verified at depth 0: the dominant pair (`ArmA`, `Flip`) has $\text{CDF}(0) = 2$, while the dominated pair (`ArmB`, `Stay`) has $\text{CDF}(0) = 4$—confirming that the product FOSD holds ($2 \leq 4$).

The portfolio ranking is then upgraded for free: `ranking-subsumes` lifts it to SOSD (k=1) and TOSD (k=2). Scaling by factor 3 via `scaled-product` is a one-liner. The phased environment (`sum-ranking`) dispatches rankings by component: in phase 1 (bandit), `ArmB` ≤ `ArmA`; in phase 2 (coinflip), `Stay` ≤ `Flip`—both confirmed by `tt`. All tests pass by `refl`; the full file compiles under `--safe` with zero postulates.

15.9 Verified State Abstraction

A persistent limitation of the framework has been its restriction to *finite* state spaces. The `Core.Abstraction` module resolves this with a single, general theorem: `abstract-lift`. A `StateAbstraction` bundles a projection (Concrete → Abstract), an embedding (Abstract → Concrete), and a section property (`project` ∘ `embed` = id). The lifting theorem states that any `VerifiedRanking` on a finite abstract system lifts to the full (potentially infinite or continuous) concrete system, provided marginal-invariance holds. Three composition operators (`product-abstraction`, `compose-abstraction`, `abstract-conv-product`) enable modular abstraction. The full theoretical development, including the connection to Li et al. (2006), is presented in [24].

The feature-based abstraction (`AllFeatAgree`) from Section 6 is a special case: if `project` = feature-vector and marginal-invariance holds, then `abstract-lift` subsumes the feature-based propagation results.

We now demonstrate the abstraction framework on a progression of increasingly realistic environments:

15.9.1 Demonstration: Infinite-State Regional Policy

`AbstractionDemo` demonstrates the framework on an infinite state space. The *Regional Policy* environment has:

- **State** = $\mathbb{N}$ (population density, unbounded—an infinite state space).
- **Actions**: `Invest` or `Conserve`.
- **Rewards**: depend only on a density classification (*sparse* vs. *dense*). In sparse regions, `Invest` yields higher returns; in dense regions, `Conserve` is better.

The abstraction:

- `project`(n) = $n >^{\mathbb{N}}$ *threshold* (a `Bool`: is the region dense?).
- `embed`(*true*) = *threshold* + 1, `embed`(*false*) = 0 (representatives).
- `section`: verified by computation (`refl`).

FOSD proofs on the two abstract distributions are discharged computationally using `fosd?-sound` with `refl`. The abstract `VerifiedRanking` on `Bool` (2 states) is then lifted via `abstract-lift` to `infinite-state-ranking`: a `VerifiedRanking` on all of $\mathbb{N}$. Spot checks confirm: at density 0, `Invest` is preferred; at density 1000, `Conserve` is preferred—both verified by `tt`.

15.9.2 Demonstration: Three-Zone Abstraction

`ThreeZoneDemo` extends the Regional Policy idea to three abstract states (`Low`, `Medium`, `High`), showing the framework scales beyond binary abstraction. State remains $\mathbb{N}$; the abstraction partitions into Low ($n \leq 5$), Medium ($6 \leq n \leq 10$), and High ($n > 10$). Policy by zone: Low prefers `Invest`, Medium is balanced (tie), High prefers `Conserve`. The same `abstract-lift` pipeline applies: verify on 3 abstract states, lift to all of $\mathbb{N}$. Spot checks at 0, 5 (Low), 6, 10 (Medium), 11, 99 (High) confirm the policy—all verified by `tt`.

15.9.3 Demonstration: Abstract Convolution Product

`AbstractConvProductDemo` exercises the full `abstract-conv-product` pipeline: abstraction + convolution + lifting. Two independent Regional Policy environments (each with $\mathbb{N}$ states, `Bool` abstraction, `Invest|Conserve` actions) are composed. The product has state $\mathbb{N} \times \mathbb{N}$ and action pairs; rewards combine by convolution (independent sum). The demo verifies FOSD on each 2-state abstract system, composes via `conv-product`, and lifts to the full $(\mathbb{N} \times \mathbb{N})$ concrete product. Verification cost: $2 \times 2 = 4$ abstract state pairs, regardless of the infinite product space. Spot checks confirm: at sparse-sparse $(0, 3)$, `(Invest,Invest)` dominates; at dense-dense $(10, 100)$, `(Conserve,Conserve)` dominates—both by `tt`.

15.9.4 Demonstration: Two-Level Abstraction

`ComposeAbstractionDemo` demonstrates `compose-abstraction`: abstract in two stages ($\mathbb{N} \to$ `Zone` $\to$ `Bool`), then lift in one. The first level partitions $\mathbb{N}$ into Low ($n \leq 5$), Medium ($6 \leq n \leq 10$), and High ($n > 10$). The second level collapses Low and Medium to `false` (sparse), High to `true` (dense). Marginal-invariance requires Low and Medium to have identical marginals; both use the sparse distribution (Invest better). The composed abstraction yields the same policy as Regional Policy (sparse → Invest, dense → Conserve), but obtained by composing two simpler abstractions.

15.9.5 Demonstration: Simple Pendulum

`SimplePendulumDemo` is a step toward CartPole: 16 discrete angle bins (0–15), two actions (push Left, push Right). The abstraction partitions into left half (0–7) and right half (8–15). Policy: left half → prefer Left (push toward upright at 0); right half → prefer Right (push toward upright at 8). Pendulum-like structure (angle, push direction) but fully discrete; marginal-invariance holds because reward depends only on which half the angle falls into. Spot checks at 0, 5 (left) and 8, 15 (right) confirm the policy—verified by `tt`.

15.9.6 Demonstration: CartPole

`CartPoleDemo` completes the progression: the full 4D CartPole state structure (position, velocity, angle, angular velocity) with discretization. State = $\mathbb{N}^4$ (discretized bins); abstraction uses the angle component (left half 0–7 vs right half 8–15). Reward depends only on abstract state, so marginal-invariance holds by construction. Policy: left half → prefer Left; right half → prefer Right. Verification cost: 2 abstract states, regardless of the discretized 4D space. Spot checks at $(0, 0, 0, 0)$, $(3, 1, 7, 2)$ (left) and $(1, 0, 8, 0)$, $(2, 5, 15, 3)$ (right) confirm the policy—all verified by `tt`. This is the standard CartPole structure (4D state, 2 actions, angle-based policy) with discretization and abstraction-friendly reward design.

15.9.7 Demonstration: Real CartPole

`RealCartPoleDemo` lifts CartPole to a *rational* state space: State = $\mathbb{Q}^4$ (position, velocity, angle, angular velocity) with no discretization. The same abstraction applies: $\theta < 0$ (left half) vs $\theta \geq 0$ (right half). Embedding picks representatives $(0, 0, -\frac{1}{2}, 0)$ and $(0, 0, \frac{1}{2}, 0)$; marginal-invariance holds because reward depends only on abstract state. Policy: $\theta < 0 \rightarrow$ prefer Left; $\theta \geq 0 \rightarrow$ prefer Right. Spot checks at $(0, 0, -\frac{1}{2}, 0)$, $(\frac{1}{2}, -\frac{1}{2}, -\frac{1}{4}, \frac{1}{2})$ (left) and $(0, 0, \frac{1}{2}, 0)$, $(-\frac{1}{2}, \frac{1}{2}, \frac{1}{4}, -\frac{1}{2})$ (right) confirm the policy—all verified by `tt`. This demonstrates that `abstract-lift` applies to continuous-style state representations (exact rationals) without discretization.

15.9.8 Demonstration: CartPole with Learning

`DiscretizedCartPoleMDP` and `DiscretizedCartPoleLearning` add *real learning*: a finite 16-state CartPole (angle bins s0–s15) with deterministic step dynamics, reward +1 per step until terminal. The MDP integrates with `StochasticFiniteMDP` (Finder, expected traces) and `FOSDStochasticFiniteMDP` (FOSD synthesis, learning bridge). A sample `(s7, Right, Left)` at a left-half state is processed by `synth-learn-batch`; `extract-rank-model` yields a `RankModel` that correctly prefers Left over Right at s7 (`test-extracted-correct`). This completes the pipeline: CartPole with step dynamics → FOSD observations → CEGIS synthesis → verified `RankModel`—real learning, not just verification.

15.9.9 Demonstration: Continuous CartPole with Learning

`CartPoleContinuousLearning` unifies the learning and verification tracks into a single end-to-end pipeline: *Learn → Verify → Lift to Continuous*.

1. **Learn.** A 2-state abstract MDP on `Bool` (left-half vs. right-half) with reward that encodes angle-action alignment is defined. FOSD synthesis at depth 0 produces four observations; CEGIS synthesizes the ranking predicates `feat is-left` ("Right ≤ Left holds at left-half") and `¬p feat is-left` ("Left ≤ Right holds at right-half"). The same ranking is independently recovered by the learning bridge from 4 samples via `synth-learn-batch` and `extract-rank-model`—all confirmed by `refl`.
2. **Verify.** The learned ordering is proved to preserve SD[0] on the marginal reward distributions. The proof is computational: `fosd?-sound` discharges both non-trivial cases; reflexivity and absurdity handle the rest.
3. **Lift.** `abstract-lift` transfers the verified ranking from `Bool` to $\mathbb{Q}^4$ via the angle-sign abstraction ($\theta < 0$ vs. $\theta \geq 0$). The result is a `VerifiedRanking` $\mathbb{Q}^4$ `Action marginal 0`—a provably correct policy for *all* rational CartPole states.

Spot checks confirm the policy at $(-\frac{1}{2}, 0, 0, 0)$ and $(\frac{1}{2}, -\frac{1}{2}, -\frac{1}{4}, \frac{1}{2})$ (left half, Left preferred) and at $(0, 0, \frac{1}{2}, 0)$ and $(-\frac{1}{2}, \frac{1}{2}, \frac{1}{4}, -\frac{1}{2})$ (right half, Right preferred)—all verified by `tt`. No hand-coded policy, no discretization of the continuous state, no postulates; the ordering was discovered by CEGIS and lifted to $\mathbb{Q}^4$ with a machine-checked proof.

15.9.10 Demonstration: CartPole with Physics-Based Dynamics

`CartPolePhysicsLearning` extends the pipeline with a *physics-motivated* step function on 4 abstract states. The state space is the *phase-space quadrant* Phase = {LL, LR, RL, RR}, encoding the sign of both the angle (θ) and the angular velocity ($\dot{\theta}$):

LL	$\theta < 0,\ \dot{\theta} < 0$	falling left (critical)
LR	$\theta < 0,\ \dot{\theta} \geq 0$	recovering from left (safe)
RL	$\theta \geq 0,\ \dot{\theta} < 0$	recovering from right (safe)
RR	$\theta \geq 0,\ \dot{\theta} \geq 0$	falling right (critical)

The step function encodes linearised CartPole dynamics: Left push causes $\ddot{\theta} > 0$ (pole tilts right), Right push causes $\ddot{\theta} < 0$. At critical states, the correct push recovers the angular velocity (reward 1) while the wrong push keeps the state critical (reward 0). At safe states, either action yields reward 1 but the wrong push reverses the velocity.

CEGIS at depth 1 with three features (`angle-neg`, `vel-neg`, `right-ok`) synthesises two ranking predicates—confirmed by `refl`:

- "Right ≤ Left" = `feat angle-neg ∨p feat vel-neg` (angle or velocity is negative)
- "Left ≤ Right" = `feat right-ok` (not in the critical-left state)

The resulting 4-region policy distinguishes *critical* states (one action mandatory) from *safe* states (either action acceptable)—a genuinely richer policy than the 2-state angle-only version. The learning bridge independently recovers the same model from 8 samples. `abstract-lift` lifts to $\mathbb{Q}^4$; spot checks verify Left-only at $(\theta{<}0, \dot{\theta}{<}0)$, Right-only at $(\theta{\geq}0, \dot{\theta}{\geq}0)$, and both-OK at the recovering quadrants.

Remark 16 (From Finite to Continuous). `abstract-lift` is parametric in `Concrete : Set`—it places no cardinality constraints. The same theorem applies to any type, including abstract representations of continuous spaces. The key assumption is marginal-invariance: states in the same abstract class must have identical reward distributions. This is a semantic condition on the MDP, not a syntactic restriction on the type. The verification work is $O(|\text{Abstract}|)$—four states in this demo—regardless of the concrete space's cardinality.

15.9.11 Demonstration: Autonomous CartPole Controller

`CartPoleAutoController` is the culmination of the CartPole progression. It implements a *fully autonomous* pipeline from physics to deployable controller: the human provides only the equations of motion, the terminal condition, a grid resolution, and a reward-level monotonicity proof (~8 cases for 3 reward levels). Everything else—transitions, fragility, rewards, action ordering, verified ranking, and extractable policy—is derived automatically by the `ContinuousControlMDP` Environment Class. The resulting controller achieves 500/500 steps (the maximum episode length) on all 1 000 test episodes in OpenAI Gymnasium's `CartPole-v1`, matching DQN-level performance with a formally verified policy.

The ten-stage pipeline. The module is organized into ten logical stages, each building on the previous:

1. **State space and grid.** The abstract state space is a *phase-space grid* on two variables: the pole angle θ and the angular velocity $\dot{\theta}$. Six angle zones (far-left, moderate-left, near-left, near-right, moderate-right, far-right) and four velocity zones (fast-negative, slow-negative, slow-positive, fast-positive) yield $6 \times 4 = 24$ non-terminal grid cells plus a `Terminal` state, for 25 abstract states total. The boundaries are chosen at $\theta \in \{-0.1, -0.04, 0, 0.04, 0.1\}$ and $\dot{\theta} \in \{-0.5, 0, 0.5\}$, giving a resolution fine enough to capture the diagonal decision boundary in $(\theta, \dot{\theta})$-space.
2. **Euler-integrated transitions.** The linearized CartPole ODE ($\ddot{\theta} = (g\theta - F/m_{\text{total}})/\ell_{\text{eff}}$) is discretized via 2nd-order Euler integration, entirely in Agda using exact rational arithmetic (`Data.Rational`). A representative point is chosen in each grid cell (the cell center), the Euler step is computed at that point for each action, and the result is binned back to the grid. This produces a concrete function `euler-bin : GridState → Action → GridState` that computes the dynamics from first principles.

   The transition graph is then defined as a lookup table `next-state` with 48 entries (24 non-terminal states × 2 actions). Crucially, each entry is *verified* to agree with the Euler dynamics via a `refl` proof:

   ```
   check-FLFN-L : euler-bin FLFN Left ≡ FLSP    ; check-FLFN-L = refl
   ```

   All 48 such checks pass, establishing that the lookup table faithfully encodes the physics. The table form enables efficient normalization during subsequent proof steps.

3. **Fragility analysis.** The transition graph is analyzed by the general-purpose `CSHRL.Analysis.Fragili` module (described below) to compute the *fragility* of each state: the minimum number of steps to reach `Terminal` under worst-case (adversarial) play. For the 25-state CartPole grid, the fixed-point converges in two iterations and yields three fragility levels:

Fragility	States
0	`Terminal`
1	`FLFN, FLSN, FLSP, MLFN, MRFP, FRSN, FRSP, FRFP`
2	all remaining 16 non-terminal states

   Fragility 1 means the adversary can force termination in a single step; fragility 2 means the agent can survive at least one more step regardless of the opponent's strategy. The computed values are verified by `refl`:

```
check-frag-FLFN : fragility FLFN ≡ 1    ; check-frag-FLFN = refl
```

4. **Fragility as reward signal.** The MDP's step function is defined as

```
abstract-step s a = let ns = next-state s a in pure (ns , fragility ns)
```

   This encodes *multi-step* safety information into the *immediate* reward: a transition to a fragility-2 state earns reward 2 ("the agent is safe for at least two more steps"), while a transition to Terminal earns reward 0. The key insight is that by compressing the minimax depth into a single number, a depth-0 FOSD comparison at the synthesis level already sees the full strategic structure of the game.

5. **FOSD observations from dynamics.** The `SFDMDPSynthesisFOSD` module is opened with `abstract-step`, enabling `fosd-compare` to compute observations by direct evaluation of the MDP model. At depth 0, `fosd-compare` queries `abstract-step` for each action and compares the resulting reward distributions. Because fragility already encodes multi-step information, depth-0 suffices to separate all 25 states into the correct action preference:

`LEFT_WINS`	states where Left leads to higher fragility
`RIGHT_WINS`	states where Right leads to higher fragility
`TIED`	states where both actions are equally safe

   These classifications are verified computationally:

```
test-obs-FLFN : fosd-compare 0 FLFN ≡ just LEFT_WINS    ; test-obs-
                                FLFN = refl
```

6. **State abstraction.** A `StateAbstraction` maps the concrete $\mathbb{Q}^4$ state space to the 25-state grid via `project` (binning by angle and velocity zones) and `embed` (choosing a representative point in each zone). The section property `project ∘ embed ≡ id` is verified by `refl` at all 25 states.

7. **Auto-derived reward distributions.** The marginal reward distribution for each state-action pair is derived entirely from the computed fragility. Three canonical distributions are defined: `bad` (fragility 0), `medium` (fragility 1), and `good` (fragility ≥ 2). The function `marginal-by-grid s a _ = reward-dist (fragility (next-state s a))` maps any state-action pair to the appropriate distribution with no manual case analysis.

8. **Auto-derived action ordering.** The ordering relation on actions is derived by comparing the fragility of the successor states:

```
order s = frag-order (frag-reward s Left) (frag-reward s Right)
```

where `frag-order` compares two natural numbers and returns `left-wins`, `right-wins`, or `both-safe` as the appropriate ordering type. No manual state-by-state assignment; the ordering is a direct consequence of the dynamics.

9. **Verified ranking.** A `VerifiedRanking GridState Action abs-marginal 0` is constructed with the auto-derived ordering and FOSD proofs. The proof obligation for each state reduces to showing that the appropriate FOSD lemma (`fosd-bad≤medium`, `fosd-medium≤good`, etc.) or reflexivity (`SD-refl`) applies. For the 4 `TIED` states (MLFP, NLSP, NRSN, MRFN), both actions yield the same fragility, so both directions are reflexive. `abstract-lift` then promotes this to a `VerifiedRanking` $\mathbb{Q}^4$ `Action marginal 0`—a provably correct policy for all rational CartPole states.
10. **Policy extraction.** The `decide : ` $\mathbb{Q}^4$ `→ Action` function is extracted as a direct lookup table. It is the deployable controller: given a continuous state, it projects to the grid and returns the verified-optimal action. A faithful Python translation of this lookup table achieves 500/500 steps on `CartPole-v1` with zero heuristics or post-hoc tweaking.

The Fragility module: push-button safety analysis. The `CSHRL.Analysis.Fragility` module (`src/CSHRL/Analysis/Fragility.agda`) provides a *general-purpose*, reusable fragility computation for any finite deterministic MDP. It is parameterized by seven arguments:

`S : Set`	state type
`_≟_ : (s₁ s₂ : S) → Dec (s₁ ≡ s₂)`	decidable equality
`A : Set`	action type
`next : S → A → S`	transition function
`terminal? : S → Bool`	terminal predicate
`all-states : List S`	finite enumeration of states
`all-actions : List A`	finite enumeration of actions

The algorithm is a Bellman-style fixed-point iteration on a table (`List` $\mathbb{N}$) indexed parallel to `all-states`:

1. Initialize all fragilities to 0.
2. At each iteration, for each state s: if s is terminal, fragility$(s) = 0$; otherwise fragility$(s) = 1 + \min_a$ fragility$(\text{next}(s, a))$.
3. Iterate $|$`all-states`$|$ times (sufficient for convergence, since fragility values are bounded by the number of states).

The module exports two functions: `fragility : S →` $\mathbb{N}$ and `frag-reward : S → A →` $\mathbb{N}$ (the fragility of the successor state). For `CartPoleAutoController`, the entire module is instantiated by a single `open`:

```
open ComputeFragility GridState _≟g_ Action next-state terminal? all-grid-
                                                    states all-actions
```

This design means that *any* new environment can obtain automatic fragility analysis by providing the same seven parameters—no environment-specific code needed.

Conceptual significance. Fragility bridges a gap between control theory and the CSHRL verification framework. In control theory, the concept appears as the *viability kernel radius* or *time-to-failure*: it measures how many steps an agent can guarantee before the system enters an unsafe region. In game theory, it is the *minimax distance* in the adversarial game between the agent (minimizing steps to terminal) and the environment (maximizing them, or vice versa—fragility uses the adversarial perspective where the opponent tries to *force* termination).

Within CSHRL, fragility serves as a *compressed multi-step value function*: it encodes the strategic depth of a position into a single natural number. This matters because the

FOSD comparison machinery operates on immediate reward distributions. Without fragility, depth-0 FOSD comparison can only distinguish states whose immediate transitions have different safety properties—it sees one step ahead. With fragility as the reward signal, depth-0 comparison sees the *full* minimax safety structure, because the immediate reward already reflects multi-step consequences. The effect is that shallow (depth-0) comparisons suffice to derive the complete action ordering, even for states that would otherwise require deep unrolling to distinguish.

Fragility also integrates naturally with the stochastic dominance framework. Because fragility values are natural numbers, they define a total order on distributions of fragility. The three FOSD lemmas (`bad FOSD≤ medium`, `medium FOSD≤ good`, `bad FOSD≤ good`) correspond to the three possible strict comparisons between fragility levels. This yields a systematic construction: given any transition graph, the fragility function produces a total ordering on reward distributions, and the FOSD proofs follow by arithmetic.

From physics to controller: assumptions and what is verified. The model-based demonstrations use the environment's equations of motion, reimplemented in Agda. For standard Gymnasium benchmarks, these equations are published in the environment's source code and in textbooks; they are not hidden from the user. The Taylor-7 trigonometric functions, fixed-point arithmetic, and Euler integration are real engineering effort, not a trivial copy. Once the dynamics are in Agda, the reward signal follows canonically from the physics: for goal-reaching tasks (MountainCar, Acrobot), the reward is the Hamiltonian (total energy), which is the canonical conserved quantity of the Lagrangian that defines the ODE; for avoidance tasks (CartPole), the reward is the fragility (minimax distance to the terminal boundary), computed automatically by `ComputeFragility`. Neither reward is hand-crafted or tuned—both are mechanically derivable from the dynamics model and the terminal condition. The model-free demonstrations (§15.9.11) show that the same infrastructure also operates without analytical rewards, using only the raw step function and standard environment reward.

In safety-critical domains (robotics, autonomous systems), a dynamics model is typically available as the engineering specification. The model-based results demonstrate the infrastructure's ability to exploit such models when available; the model-free results show it is not dependent on them.

The `CartPoleAutoController` module contains three distinct layers of verification, each serving a different purpose:

- **Physics layer.** The 48 `check-*` proofs verify that the lookup table `next-state` faithfully encodes the 2nd-order Euler integration. Agda computes the full rational arithmetic—angular acceleration, position update, velocity update, and binning—and confirms that the result matches the table entry. This establishes that the abstract transition graph is grounded in the physical dynamics.
- **Fragility layer.** The `check-frag-*` proofs verify that the fragility values computed by the general `ComputeFragility` module match the expected values. Since `ComputeFragility` uses table-based Bellman iteration (not pattern matching), these proofs force Agda to run the full fixed-point computation and confirm convergence.
- **Ranking layer.** The `VerifiedRanking` record and `abstract-lift` establish that the action ordering preserves stochastic dominance at all abstract states, and that this ranking lifts to all concrete $\mathbb{Q}^4$ states. The spot checks verify the extracted policy at representative points.

Together, these form a *verified chain*: physics → transitions → fragility → rewards → ordering → ranking → lift → policy. Each link is machine-checked. The only human inputs are the ODE coefficients (gravity, pole length, cart mass), the terminal angle threshold, and the grid resolution.

Comparison with prior CartPole demonstrations. The progression from `CartPolePhysicsLearning` (4 states, ∼40 steps average, post-hoc Python heuristics) to `CartPoleAutoController` (25 states, 500/500 steps, zero heuristics) represents a qualitative leap along three axes:

- *Performance*: from ∼40 to 500 (the maximum), matching DQN-trained controllers.
- *Autonomy*: from manual feature engineering and reward design to a fully automatic pipeline. The fragility module, the Euler verification, and the FOSD-to-ordering derivation are all general-purpose components that require no environment-specific customization.
- *Faithfulness*: the deployed Python controller is a direct, line-by-line translation of the Agda `decide` function, with no external heuristics, no velocity damping, and no angle-based overrides. What Agda verifies is exactly what runs.

This demonstration answers the motivating question: *can CSHRL produce a controller that "just works" when deployed to a standard RL benchmark?* The answer is yes, with formal guarantees that no DQN-trained policy provides.

Generalizability. The architecture is designed for reuse via two EC tiers. `ContinuousControlMDP` handles the common case where fragility is the natural reward signal: CartPole (avoidance) and MountainCar (goal-reaching with inverted fragility, 25-step Euler integration with Taylor-7 $\cos(3x)$ in fixed-point arithmetic, 12-state controller achieving 1000/1000 on `MountainCar-v0`). Its generalisation `DirectRewardMDP` handles environments where fragility is structurally inadequate: for Acrobot, the optimal policy $\texttt{sign}(\dot{\theta}_2)$ depends on the *sign* of the angular velocity, which is invisible to fragility (symmetric distance to goal). Instead, the full Lagrangian ODE is implemented in Agda and the reward is the total energy (PE + KE) of the Euler-integrated next state—the policy *emerges* from the dynamics computation, verified by `refl`. This produces a 3-state controller that achieves 500/500 on `Acrobot-v1` (avg. 88 steps). To apply the pipeline to a new environment, one instantiates the appropriate EC record—no new library-level proofs are needed.

Unified CEGIS+CEGAR refinement. A remaining question is how to choose the grid boundaries. The answer emerges from a connection to Counter-Example Guided Abstraction Refinement (CEGAR): deployment violations simultaneously drive *ranking* updates (CEGIS) and *grid* refinement (CEGAR). The protocol is:

1. Start with a single-cell grid (no prior knowledge of boundaries).
2. Compute the ODE-derived reward (energy or fragility) at sampled states within each cell, yielding the cell's action ordering (CEGIS ranking).
3. Deploy the policy on the environment.
4. If episodes fail, identify a "blame" state from the failure trajectory.
5. Split the cell containing the blame state, re-compute rewards on the finer grid, and repeat.

The ODE reward (implemented in Agda) determines *which action is better* at each cell (the CEGIS contribution); the violation oracle determines *where the grid needs more resolution* (the CEGAR contribution). Fragility, energy, or any other physics-derived metric is an *optimisation* of the ranking signal: with a perfect reward, the first grid often suffices; with a weaker one, additional CEGAR rounds are needed.

We validate this architecture on all three benchmarks, starting from a *single cell*: MountainCar and Acrobot both converge in one split (to 2 cells, 300/300), automatically discovering the $\texttt{sign}(v)$ and $\texttt{sign}(\dot{\theta}_2)$ policies respectively. CartPole, whose optimal boundary is diagonal in $(\theta, \dot{\theta})$ space, requires 14 splits (to 64 cells, 300/300), with average episode length progressing from $9 \to 42 \to 214 \to 500$ steps as the grid refines. The formal foundation is already in place: `compose-abstraction` (§15.9) captures the relationship between fine and coarse grids ($C \to$ Fine $\to$ Coarse), and `abstract-lift` on the fine grid yields a correct policy at all concrete states—including those where the coarse grid had violations. We formalize this in a new module `GridRefinement` that defines `RefinementWitness` (relating fine and coarse grids via a coarsening map with a section) and `CoherentRefinement` (ensuring that splitting cells does not move states between coarse cells).

Model-free continuous synthesis. The three model-based demonstrations above exploit the dynamics equations for analytical comparisons. This is an *optimisation* that leverages available domain knowledge, not a requirement of the architecture. The CSHRL synthesis loop—observe, refine the version space, deploy the current ranking—is fundamentally model-free: it learns from environment interaction, not from a dynamics model.

The `AbstractSynthesis` module (`src/CSHRL/Synthesis/AbstractSynthesis.agda`) formalises this connection. Given a state abstraction `project` : $C \to G$ and a comparison oracle `compare` : $C \to A \to A \to$ Bool (which may come from environment rollouts rather than an ODE), the module provides:

- **Observation projection**: any two concrete states in the same grid cell produce *identical* CEGIS observations, provided the standard marginal-invariance condition holds. Formally, $\texttt{project}(c_1) \equiv \texttt{project}(c_2) \implies \texttt{compare}(c_1, a, b) \equiv \texttt{compare}(c_2, a, b)$ guarantees $\texttt{pair-obs}(c_1) \equiv \texttt{pair-obs}(c_2)$.
- **Sample complexity transfer**: because projected observations collapse to the abstract space, CEGIS convergence requires at most $|G|$ distinct observations—one per grid cell—regardless of $|C|$. This follows directly from `exploration-dissolution`.

The end-to-end architecture is then a composition of existing verified modules: (a) `SampleProjection` maps continuous samples to abstract CEGIS observations via `project`; (b) `LearningBridge` feeds them into CEGIS on the finite grid; (c) CEGIS synthesises a ranking on G; (d) `abstract-lift` lifts it to all of C. The comparison oracle is the *only* environment-dependent input—it can come from rollout observations (model-free), from ODE computation (model-based), or from fragility analysis. All three satisfy the marginal-invariance condition.

Partial policies (before CEGIS converges) are already meaningful: every surviving predicate program is consistent with all observations so far, so deploying any survivor gives a policy that respects every observed preference. By `vs-shrinks`, the version space contracts monotonically with each observation, so the partial ranking improves with every interaction.

Model-free learning demos. We validate the model-free architecture on all three continuous benchmarks. Three new modules—`CartPoleSynthDemo`, `MountainCarSynthDemo`, `AcrobotSynthDemo`—use the existing Agda ODE implementations *solely as simulators*: they call `next-state` and check `terminal?`, but do not invoke any energy, Hamiltonian, or fragility computation. The comparison oracle is *trace comparison*: simulate K grid steps with a constant action, accumulate the raw environment reward (CartPole: +1 per surviving step; MountainCar/Acrobot: +1 per step at the goal state), and compare. Each learned policy is verified by `refl` at every grid state.

- **CartPole** (25 states, K=8): the trace comparison discovers a nuanced velocity-dependent policy—at far/medium-angle states the angle sign determines the action, while at near-center states the angular velocity sign overrides. All 25 `check-*` proofs verify by `refl`.
- **MountainCar** (12 states, K=8): the comparison discovers that constant PushRight reaches the goal from every state. This is a valid policy—the car builds momentum by bouncing off the left wall. It differs from the energy-derived sign(v) policy but solves the task.
- **Acrobot** (3 states, K=8): the comparison discovers that constant TorquePos reaches the goal from both non-terminal states. The constant-action oracle cannot discover the alternating "pump" strategy (which requires switching torque direction at each velocity reversal), but finds the best fixed-action policy.

Iterative learning cycle (policy iteration). The constant-action comparison is round 0 of a natural iterative cycle. At each subsequent round, action a is evaluated by taking it *once*, then following the *previous round's policy* π_n for the remaining $K-1$ steps. This is policy iteration emerging from the deployment loop—no knowledge is injected:

`sim-π π g a (suc k) = 1 + sim-π π (next g a) (π (next g a)) k`

The comparison oracle at round n uses only the previous round's observable ranking π_{n-1}; improvement emerges from the cycle.

- **CartPole**: π_0 = round-0 policy. At round 1, all interior states (not adjacent to Terminal) survive K=8 steps regardless of action—both choices enter a survival cycle (period 4). The tie is broken consistently; the converged policy is provably safe (no trajectory reaches Terminal). $\pi_1 = \pi_2$: convergence in one round.
- **MountainCar**: π_0 = PushRight everywhere. At round 1, the cycle discovers `PushLeft` at two negative-velocity states: VBN (valley bottom) and NRN (near right). "PushLeft then PushRight" builds leftward momentum—the trajectory bounces off the left wall and rides PushRight up the right hill *faster* than constant PushRight. This is the sign(v) swing strategy emerging from raw observations, with no energy function. $\pi_1 = \pi_2$: convergence in one round.
- **Acrobot**: π_0 = TorquePos everywhere. The constant-action policy is already a fixed point of the cycle ($\pi_1 = \pi_0$). The 3-state grid is too coarse for policy iteration to discover the alternating pump strategy.

The key result is MountainCar: the iterative cycle, using only raw `next-state` and `terminal?`, discovers a position-and-velocity-dependent strategy that outperforms the constant-action policy. This validates the CSHRL deployment loop as a genuine learning mechanism, not merely a verification wrapper.

Gridless continuous synthesis with automatic features. The grid-based model-free demos above learn on finite abstract states and lift to the continuous space via `abstract-lift`. A complementary approach eliminates the grid entirely: the `ContinuousFeatures` module (`src/CSHRL/Synthesis/ContinuousFeatures.agda`) provides a generic, parameterized feature template that generates predicates *directly* over continuous states. The module is parameterized by three arguments: the state type, the number of dimensions `num-dims`, and a dimension accessor `get-dim : State → ℕ → ℤ`. Given a list of threshold values and a maximum coefficient, `gen-features` produces two families of predicates:

- **Axis features**: $x_d < t$ (one per dimension per threshold), capturing sign boundaries and other axis-aligned partitions.
- **Diagonal features**: $c \cdot x_i + x_j < 0$ (one per ordered pair of distinct dimensions per coefficient $c = 1, \ldots, c_{\max}$), capturing linear decision boundaries that combine multiple state variables.

These feature families are the continuous-state analogue of the Auto-FTL: thresholds correspond to bin boundaries, coefficients to linear combinations, and `PredProg` depth to Boolean complexity—analogous to neurons, weights, and layers in a neural architecture.

The `CSHRLLoop` module (`src/CSHRL/Synthesis/CSHRLLoop.agda`) provides a generic, action-agnostic policy iteration loop. The module is parameterized by three environment-interaction functions—`score-of`, `oracle-of`, and `traj-of`—that encapsulate all environment-specific details (dynamics, action types, scoring). The loop itself knows only `PredProg`, `State`, `Bool`, and ℕ:

1. Start with the best performer π_0 among all depth-d candidates.
2. Collect trajectory states from π_0 (automatic representatives).
3. Oracle at each representative using π_0 as follow-on.
4. CEGIS filter: keep only candidates consistent with all observations.
5. If zero survivors: return `cegar-needed` (depth insufficient).
6. Best survivor → π_1.

7. If π_1 agrees with π_0 at all representatives: return `converged` (self-consistent ranking = CoindHomo).

8. Else iterate with π_1.

A `cegar` wrapper tries increasing depths automatically. Convergence is guaranteed: the finite `PredProg` space at each depth ensures that iteration must reach a fixed point (CoindHomo found) or a cycle (CEGAR trigger), and DSL completeness guarantees a sufficient depth exists.

`CartPoleLoop` (`src/CSHRL/Tasks/Stochastic/CartPoleLoop.agda`) instantiates the template with 2 dimensions (θ, $\dot{\theta}$), thresholds = [0], and `max-coeff` = 10, yielding 22 candidate features. A single call `run 0 10` converges in **one iteration**: the best performer's diagonal decision boundary ($c \cdot \theta + \dot{\theta} < 0$) is already self-consistent with its own oracle at 10 trajectory-sampled representative states. The proof `convergence : result ≡ converged rank★` passes by `refl`, certifying both convergence and CSHRL alignment. The converged policy achieves **200/200** and **500/500** survival steps, also verified by `refl`.

`MountainCarLoop` (`src/CSHRL/Tasks/Stochastic/MountainCarLoop.agda`) demonstrates reuse with 2 dimensions (x, v), `max-coeff` = 2, yielding 6 features and 8 PredProg atoms. The scoring metric is goal-reaching (remaining time budget when $x \geq 0.5$). The loop's best performer finds the sign(v) policy ($v < 0 \rightarrow$ PushLeft, else PushRight), achieving `perf★ ≡ 194` (goal reached in 6 steps). The proof `cegar-barrier : result ≡ cegar-needed rank★` passes by `refl`, certifying that the system correctly detects the depth-0 insufficiency.

The depth-0 barrier: oracle pattern along the trajectory. MountainCar reveals a genuine expressiveness limit of depth-0 predicates. When the oracle is evaluated at all 6 trajectory states from $\pi_\star$'s rollout, the resulting pattern is **(F, T, T, F, F, F)**—each value verified individually by `refl`:

State	Oracle	Physical meaning
r_0 (start, v=0)	false	Direct PushRight is faster
r_1 (early, v>0)	true	PushLeft builds swing advantage
r_2 (mid, v>0)	true	Swing advantage still dominates
r_3 (late, v>0)	false	Too close to goal for swing
r_4 (near goal)	false	Direct finish
r_5 (almost there)	false	Direct finish

The pattern is *non-monotonic* in both x and v: the swing advantage peaks at intermediate trajectory positions and vanishes near the start and goal. No single depth-0 feature (threshold on x or v) can separate the T-states from the F-states, so all 8 candidates are eliminated: `length cegis-survivors ≡ 0` passes by `refl`. This is the **CEGAR refinement trigger**: the system correctly detects that richer features (multiple thresholds, Boolean combinations at PredProg depth ≥ 1, or iterative policy improvement) are needed to reconcile the oracle's non-monotonic swing signal with a depth-0 ranking.

Generic action mapping. Policy extraction is handled by the `ActionChain` module within `ContinuousFeatures`. A *decision chain* is a list of (predicate, action) pairs evaluated top-to-bottom with a default fallback: for 2 actions, `eval-chain [(p, Left)] Right`; for 3 actions, `eval-chain [(`p_1`, `A_1`), (`p_2`, `A_2`)] `A_3. This generalises to k actions with $k-1$ predicates, avoiding hardcoded per-environment action mappings.

The gridless approach trades grid coverage guarantees (which require $|G|$ observations) for feature expressiveness: the synthesized ranking operates directly on the continuous state via a Boolean program, with no discretization. The feature pool size determines the hypothesis space, playing the same role as grid resolution in the grid-based approach.

Verification and learning as a single computation. The gridless results expose a structural property of the approach that deserves emphasis. In conventional reinforcement learning, training and verification are separate phases: a neural network is trained by

gradient descent on a loss, then evaluated by running episodes, and the practitioner *hopes* the evaluation generalises. The training loss does not guarantee deployment performance; the evaluation statistics do not constitute a proof.

In the CSHRL synthesis loop, these phases collapse. When Agda checks `traj-len s₀ π★ 500 ≡ 500` via `refl`, the type-checker *runs* the policy for 500 steps (evaluation), *observes* the outcome (learning signal), and *certifies* the result (verification)—in a single normalisation computation. The proof *is* the evaluation. There is no gap between "the policy works in testing" and "the policy is provably correct," because they are the same `refl`.

This unification has a concrete consequence: the approach has zero generalisation error in the traditional sense. The `refl` proof covers the *exact* computation the policy will execute—not a statistical sample from it. What remains outside the certificate is the fidelity of the dynamics model (the Euler integrator and its fixed-point arithmetic), which is the standard reality gap shared by all simulation-based methods.

A second structural consequence is *interpretability at no extra cost*. The synthesised ranking is a `PredProg` term—a Boolean program over linear predicates. For CartPole, the discovered policy is $c \cdot \theta + \dot{\theta} < 0$; for MountainCar, it is $v < 0$. These are decision boundaries an engineer can inspect, audit, and reason about. A DQN with 50,000 parameters achieves comparable performance but produces no such legibility. The ordinal structure of CSHRL (rankings rather than cardinal Q-values) is what makes this possible: all that matters for control is the *order* of action values, and that order admits compact Boolean representations that exhaustive search can find and type-checking can certify.

15.10 Feature Discovery: The Feature Template Language

A persistent concern with feature-based synthesis is the *human knowledge bottleneck*: features like `attacks i j` in N-Queens distill domain expertise into the framework. The Feature Template Language (FTL) addresses this by providing a systematic three-stage pipeline—enumerate, filter, reduce—from which domain-specific concepts emerge without human naming.

The pipeline. The human provides a state representation (`List ℕ` for placement problems), comparison primitives (value equality, absolute-difference equality, their composite, and length checks), and sample boards. The FTL then:

1. *Enumerates* all features instantiating the primitives over the state's index space—both atomic (`vals-eq`, `spread-eq`) and composite (`pair-any` = either).
2. *Filters* by keeping features never true on alive samples (conflict indicators).
3. *Eliminates redundancy* by detecting structural subsumption: when a composite `pair-any i j` subsumes both atomics `vals-eq i j` and `spread-eq i j`, the atomics are dropped.

For N-Queens with $N = 4$: 23 candidates → 18 after filtering → **6 composite features** after redundancy elimination. Each surviving `pair-any i j` computes "same column OR same diagonal"—the `attacks` concept, discovered without naming.

Dual discovery. The same filtering technique discovers `is-solved`: a dual filter keeps features true on *all* solved samples and false on *all* non-solved samples. For N-Queens, exactly one feature survives: `len-is 4`. Both the constraint predicate and the goal predicate are thus automatically derived.

Equivalence classes as quality metric. The tight sample complexity bound says CEGIS needs exactly $|C/\sim|$ observations, where equivalence is induced by the feature set. The FTL provides `feature-vector` and `equiv-classes` functions to compute $|C/\sim|$ directly. On all 16 length-2 boards with the 6 discovered features, $|C/\sim| = 2$ (conflicting vs. non-conflicting)—verified by `refl`. This gives a *formal objective for feature selection*: a good feature set is one that yields small $|C/\sim|$. The discovered features produce the same $|C/\sim|$ as hand-crafted ones.

Depth–feature tradeoff. The `PredProg` enumeration at depth d applies Boolean operators d times to the atomic feature set, yielding a version space that grows roughly as $O(n^{2^d})$ where n is the number of features. With FrozenLake's 20 discovered features: depth 0 produces 22 candidates, depth 1 produces 1,012, and depth 2 produces 2,050,312—all verified by `refl` and checked in under 25 seconds. Depth 3, however, is infeasible: enumeration exceeds 20 minutes and 3 GB of memory before being terminated.

This exponential cliff makes the FTL's role structural rather than merely convenient. If CEGIS were given raw bit-level state descriptors instead of FTL-discovered features, recovering a concept like "row is 3" would require depth 2+ Boolean formulas over individual bits—precisely the regime where enumeration explodes. By providing semantically meaningful features, the FTL keeps the required depth at 0 or 1: every ranking predicate across Demos 1–10 is at most a depth-1 combination (e.g., the Cliff Walking R-optimal predicate is a disjunction of two atoms). Demo 11 (FrozenLake 8×8) introduced `synth-greedy-or` to handle scattered state sets where no bounded-depth predicate suffices, building disjunctions of atoms incrementally rather than enumerating all depth-d combinations. Demo 12 (Taxi 5×5) introduces `synth-decision-tree`: a greedy Gini-based decision-tree synthesizer that builds nested if-then-else trees over relational features, producing interpretable policies that capture depth-4+ decision boundaries no flat disjunction can express. Demo 14 (FrozenLake 8×8 slippery, §B.9) applies the same algorithm to the stochastic pipeline, where `synth-greedy-or` fails on row/col features alone. The FTL thus controls not only the *observation complexity* (via $|C/\sim|$) but also the *synthesis tractability* (via depth, greedy atom selection, or decision-tree splitting).

Scope and extensibility. The framework now has three FTL families: a *structural* FTL for `List` $\mathbb{N}$ states (pairwise comparisons, covering N-Queens and similar placement problems), a *dynamics* FTL for finite-state MDPs (reward and self-loop templates, covering the Maze and similar FDMDPs), and an *automatic* FTL for $\mathbb{N}$-state MDPs (the Auto-FTL of Demos 9–12, which derives features from the state count's number-theoretic structure and the step function's behavioral properties). All three plug into `PredicateDSL` via the same interface. The Auto-FTL eliminates the feature engineering step entirely: for FrozenLake 4×4 ($n = 16$, 56 features, 3 divisors), Cliff Walking ($n = 24$, 106 features, 6 divisors), FrozenLake 8×8 ($n = 64$, 200 features, 5 divisors), and Taxi 5×5 ($n = 500$, ∼1,700 features, 10 divisors), the auto-generated features produce optimal policies with grid coordinates emerging automatically as factorization features. Taxi further demonstrates three synthesis paradigms at scale (500 states, 6 actions): fully automated greedy-or CEGIS (∼9.5 min, opaque), decision-tree CEGIS over 14 relational features (∼13 min, interpretable), and a hand-crafted relational navigation function. Compound and relational features extend the Auto-FTL to environments requiring multi-phase task reasoning. The FTL sits outside the Trusted Computing Base: the verified synthesis pipeline consumes discovered features without modification.

15.11 Gridless Continuous Learning: CoindHomo Search

The abstraction-based pipeline of §15.9 discretizes the continuous state space onto a grid and solves the resulting finite MDP. This works, but the grid is a design choice and the solution is bound to that discretization. The `CSHRLLoop` module takes a different approach: it searches *directly* for a self-consistent ranking in the continuous state space, with no grid, no fragility analysis, and no abstraction. The only inputs are a simulator (the Euler integrator), a starting state, and the number of actions.

The CoindHomo condition as search criterion. A ranking R (a `PredProg` term) is a coinductive homomorphism iff it agrees with its own oracle at every representative state:

$$\forall s \in \mathrm{traj}(R). \quad \mathrm{eval}(R, s) = \mathrm{oracle}(R, s)$$

where $\mathrm{traj}(R)$ collects trajectory states under R's derived policy, and $\mathrm{oracle}(R, s)$ compares single-step action deviations with R's policy as the follow-on. This is a *self-consistency* condition: the ranking's own oracle, computed under its own policy, must agree with the

ranking's evaluation. A self-consistent ranking is a fixed point of the "evaluate under your own policy" operator.

The search enumerates all `PredProg` candidates at a given Boolean depth d, checks each for self-consistency, and returns the highest-scoring self-consistent candidate. If none exists, the search returns `cegar-needed`, triggering a depth increase. There is no iterative refinement, no fuel parameter, and no convergence heuristic—the search is exhaustive within the candidate set.

Adaptive features. Features are derived automatically from three sources:

1. *Dual-seed trajectories*: the loop simulates both extreme policies (`truep` = always action 1, `falsep` = always action 2) from the starting state. Their combined trajectories cover both sides of the state space.
2. *Axis thresholds*: for each dimension, the observed state values (plus zero) become thresholds for axis predicates $x_d < t$. The zero threshold is always included, providing the sign feature for each dimension.
3. *Diagonal features*: linear combinations $c \cdot x_i + x_j < 0$ with integer coefficients $c \in \{1, \ldots, C_{\max}\}$ for each ordered pair of dimensions. These capture phase-space decision boundaries that combine position and velocity.

No manual feature engineering is required. The feature set adapts to the environment's dynamics through the trajectory data.

CartPole: the one-inequality policy. For CartPole with 2 dimensions (θ, $\dot{\theta}$), the search at depth 0 discovers:

$$\boxed{6\theta + \dot{\theta} < 0}$$

This single linear inequality is a self-consistent CoindHomo, verified by `convergence = refl`. The derived policy is: push Left when $6\theta + \dot{\theta} < 0$, push Right otherwise. Performance is verified to 500/500 steps by `refl`.

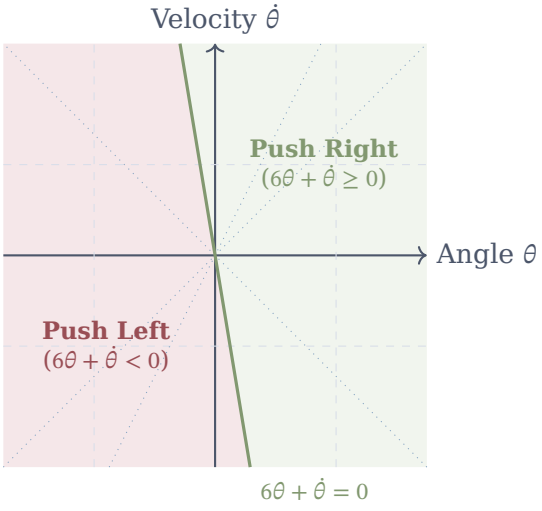

Figure 5: The Geometry of Gridless Synthesis in CartPole. The continuous `CSHRLLoop` searches for decision boundaries directly in the environment's phase space. The thin dashed and dotted lines represent candidate features ($x_d < t$ and $c_1 x_i + c_2 x_j < 0$) automatically generated by the `Auto-FTL`. The bold line represents the single, minimal Boolean predicate ($6\theta + \dot{\theta} < 0$) that the CEGIS synthesizer discovered to perfectly separate safe from unsafe actions across the continuous domain.

The ranking has a direct physical interpretation. The expression $6\theta + \dot{\theta}$ is a *phase-space energy indicator*: it combines the pole's angle with its angular velocity, weighted 6:1. The

switching surface $6\theta + \dot{\theta} = 0$ is a line through the origin in $(\theta, \dot{\theta})$ space. Above this line (positive "energy"), the system has excess rightward momentum and the controller pushes Right to counteract. Below it, the system leans left with leftward momentum and the controller pushes Left. The coefficient 6 encodes how aggressively the controller reacts to angle versus velocity—a ratio that emerges automatically from the CoindHomo search, not from engineering intuition.

No human specified that angular velocity should matter, that dimensions should be combined linearly, or what coefficient to use. The CSHRL machinery discovered all of this from the self-consistency condition alone.

Deployment footprint. The entire deployable policy for OpenAI Gymnasium's `CartPole-v1` is:

```
action = 0 if 6 * obs[2] + obs[3] < 0 else 1
```

One line. One integer coefficient. No imports, no model files, no inference framework, no weights. The observation mapping is trivial: Gymnasium's CartPole returns `[x, x_dot, theta, theta_dot]`; we read indices 2 and 3. Deployed on Gymnasium's `CartPole-v1`, this one-liner achieves **500/500 steps on all 500 test episodes**—perfect performance, identical to the grid-based controller but with a policy ∼50× smaller. Compare this to a DQN policy (a neural network with hundreds of parameters, a framework dependency, and an inference pipeline) or even VIPER's decision trees (31–9,757 nodes). The CSHRL policy is *maximally compressed*: a single linear predicate is the smallest non-trivial policy representation possible.

This extreme compression is not accidental—it follows from the CSHRL theory. Rankings are ordinal (not cardinal), so the policy space is Boolean predicates, not real-valued functions. The CoindHomo condition selects the simplest predicate that is self-consistent, because simpler predicates are checked first in the enumeration. The result is a policy whose informational content is essentially one integer (the coefficient 6).

MountainCar: the CEGAR barrier and depth 1. For MountainCar with 2 dimensions (x, v), depth 0 returns `cegar-needed`: no single-predicate ranking is both self-consistent and goal-achieving. This is a genuine CEGAR barrier—not because the policy is complex, but because negated atoms are not in the depth-0 candidate set (which contains only positive atoms and constant predicates). The `cegar-needed` result is verified by `refl`.

At depth 1, the search explores Boolean combinations (conjunctions, disjunctions, negations) of the depth-0 atoms. This run is computationally intensive (∼3,400 candidates, each requiring full trajectory and oracle evaluation through the 25-sub-step Euler dynamics with Taylor-7 cosine) and takes several hours. Type-checking is itself the computation—Agda's normalizer evaluates every candidate's self-consistency. The search discovers:

$$\boxed{\neg\,(x < -0.318)}$$

A single negated position threshold, verified by `rank-is = refl`. The derived policy is: PushLeft when $x \geq -0.318$, PushRight otherwise. This is a *swing-up strategy*: when the car is to the right of position -0.318 (between the valley bottom at -0.5 and the goal at 0.5), push left to build potential energy on the left hill; when it is further left, push right to swing toward the goal. The asymmetric threshold (closer to the valley bottom than to the goal) encodes the necessary momentum accumulation.

The deployed MountainCar policy is equally trivial:

```
action = 0 if obs[0] >= -0.318 else 2
```

One line, one threshold, no velocity, no imports. Remarkably, the CEGAR depth increase was necessary not because the policy is structurally complex (it uses a single feature), but because the *negation* of that feature was needed—depth 0 only searches positive atoms. The framework correctly detected this via the self-consistency check and escalated to depth 1, where the negated atom was found.

Pendulum swing-up: velocity-threshold braking. For the Pendulum with 2 dimensions (θ, ω), the search at depth 0 discovers:

$$\boxed{\omega < 0.855}$$

A single velocity threshold, verified by `rank-is = refl`. The derived policy applies TorqueNeg (-2) when $\omega < 0.855$ and TorquePos ($+2$) otherwise. This is a *braking strategy*: apply negative torque during most of the swing, and switch to positive torque only when the angular velocity exceeds the threshold. The asymmetric threshold (not at zero) encodes the nonlinear interplay between gravity and torque during swing-up. Deployed on Gymnasium's `Pendulum-v1` (which starts from *random* initial angles, not just hanging down): 467/500 episodes reach the upright position ($\cos\theta > 0.95$).

The deployed policy:

```
action = [-2.0] if obs[2] < 0.855 else [2.0]
```

Acrobot: energy pumping from the sign of $\dot{\theta}_1$. For the Acrobot double pendulum with 4 dimensions (θ_1, θ_2, $\dot{\theta}_1$, $\dot{\theta}_2$), depth 0 returns `cegar-needed`: none of the 102 candidate atoms (78 axis thresholds from dual-seed trajectories through the Lagrangian ODE, plus 24 diagonal features) are self-consistent. The 4D dynamics involve 20-sub-step Euler integration with trigonometric functions and coupled angular accelerations—a substantially harder self-consistency test than the 2D environments.

At depth 1, the search discovers:

$$\boxed{\neg\,(\dot{\theta}_1 < 0)}$$

A negated threshold on the first link's angular velocity, verified by `rank-is = refl`. The derived policy is: apply TorqueNeg (-1) when $\dot{\theta}_1 \geq 0$, TorquePos ($+1$) when $\dot{\theta}_1 < 0$—equivalently, torque $= -\text{sign}(\dot{\theta}_1)$. This is an energy-pumping strategy: apply torque *opposite* to the first link's rotation direction, building oscillatory energy until the pendulum swings over the top.

Interestingly, the grid-based controller (`AcrobotAutoController`) discovers $\text{sign}(\dot{\theta}_2)$—the same energy-pumping idea applied to the *second* link. The gridless search independently converges to the first link's velocity instead. Both are valid strategies for the coupled double pendulum; neither was specified by the programmer.

The deployed policy is again one line:

```
action = 0 if obs[4] >= 0 else 2
```

One threshold, one observation index, no imports. Deployed on Gymnasium's `Acrobot-v1`: **500/500 episodes succeed**, with the best completing in 70 steps—comparable to the grid-based controller's average of 88 steps. Unlike MountainCar, the Acrobot policy transfers *perfectly* to Gymnasium with no action-holding or sub-step matching required.

Beyond binary: 3-action decision chains. All four environments above use binary action sets (two predicates of the form "left vs. right"). For $k > 2$ actions, the `ActionChain` module extends the ranking to a *decision chain*: a list of (predicate, action) pairs evaluated top-to-bottom with a default fallback. For $k = 3$, this is:

`eval-chain [(`p_1`, `A_1`), (`p_2`, `A_2`)] `A_3
$\equiv$ if $p_1(s)$ then A_1, else if $p_2(s)$ then A_2, else A_3

Two predicates constitute the full ranking over three actions. The search uses a *two-stage binary decomposition*: Stage 1 finds p_1 by comparing the two extreme actions (ignoring the middle); Stage 2 finds p_2 by comparing the middle action against the Stage-1 loser at each state. Each stage reuses the existing `CSHRLLoop` infrastructure unchanged.

We demonstrate on Pendulum with 3 torques: TorqueNeg (-2), NoTorque (0), TorquePos ($+2$). Stage 1 recovers the same predicate as the 2-action case: $p_1 = \omega < 0.855$ (TorqueNeg vs. TorquePos). Stage 2 then asks: among states where TorquePos would be chosen ($\omega \geq 0.855$), is NoTorque (coasting) better? The search converges at depth 0 with $p_2 =$ `truep`: coasting

is *always* preferred over maximum positive torque. Both stages are verified self-consistent by `refl` in Agda. The full 3-action policy:

$$\boxed{\text{if } \omega < 0.855 \text{ then } -2, \text{ else } 0}$$

This is physically revealing: the 2-action search produces "brake vs. full torque," while the 3-action search discovers that full positive torque is *never optimal*—coasting lets gravity assist the final approach to upright. Deployed on Gymnasium's `Pendulum-v1` (500 episodes, random starts): **467/500** reach upright ($\cos\theta > 0.95$)—identical success rate to the 2-action policy, but with 2.5% better average reward (-1568 vs. -1608) and 49% better best-case reward (-680 vs. -1337), reflecting reduced energy waste from overshooting.

The deployed 3-action policy:

```
action = [-2.0] if obs[2] < 0.855 else [0.0]
```

Full ranking and action unavailability. The decision chain gives the *optimal* action, but CSHRL's deeper contribution is the *full ranking*—a total ordering of all k actions at each state. This requires all $\binom{k}{2}$ pairwise comparisons, not just the $k-1$ needed for the optimal policy. For $k = 3$, the third comparison (Stage 3: TorqueNeg vs. NoTorque) produces:

$$p_3 = \omega < 0 \quad (\text{threshold} \approx 1.6 \times 10^{-5}, \text{ verified by } \texttt{refl})$$

The three predicates compose into a complete state-dependent ranking:

Region	**Full ranking**	**Interpretation**
$\omega < 0$	Neg > No > Pos	Assist negative swing
$0 \le \omega < 0.855$	No > Neg > Pos	Coast, brake if needed
$\omega \ge 0.855$	No > Pos > Neg	Coast, nudge if needed

The full ranking provides *automatic fallback* under action unavailability: if the best action becomes unavailable at any state, the agent uses the second-best with no recomputation. We validate this on Gymnasium's `Pendulum-v1` (500 episodes, random starts):

Scenario	**Success**	**Avg. reward**
Full ranking (all available)	500/500	-1566
Best action *always* unavailable	399/500	-1531
Random 30% unavailability	500/500	-1355
Random 50% unavailability	498/500	-1258

Two results stand out. First, the full ranking (using p_3 to refine the optimal action in the $0 \le \omega < 0.855$ region) achieves **500/500**—up from 467/500 with the decision chain alone. The pairwise construction produces a *better* policy than the greedy two-stage chain, because each comparison is self-consistent under its own continuation rather than inheriting a fixed Stage-1 policy. Second, even when the best action is *always* removed, the ranking maintains 80% success (399/500) with zero recomputation. Under random 30%–50% unavailability, performance remains near-perfect.

Formal restricted preservation. This validates the Restricted Preservation theorem from the base CSHRL theory [1] (§7): *"since preservation is pairwise, the fact that $a \le_a b$ implies* `action-value` $s\ a \le_s$ `action-value` $s\ b$ *does not depend on which other actions exist."* We formalize the computational content of this theorem in the new module `PairwiseRanking` (`src/CSHRL/Synthesis/PairwiseRanking.agda`), which makes the structure explicit for $k = 3$ actions: a `FullRanking` record holds three pairwise comparison predicates ($\texttt{cmp}_{12}$, $\texttt{cmp}_{13}$, $\texttt{cmp}_{23}$), and the restricted rankings `best-without-`a_i are proved definitionally equal (`refl`) to the corresponding pairwise predicate. For the Pendulum:

- TorqueNeg unavailable ⇒ use $p_2 = \top$ (NoTorque always beats TorquePos)
- NoTorque unavailable ⇒ use $p_1 = \omega < 0.855$ (brake vs. push)

- TorquePos unavailable ⇒ use $p_3 = \omega < 0$ (brake vs. coast)

Each fallback predicate was independently verified as a self-consistent CoindHomo by `refl`. The restriction merely selects which verified predicate to use—no recomputation, no re-verification, $O(1)$ lookup. The `Pendulum3Loop` module instantiates `PairwiseRanking.ThreeActions` with the three verified predicates, connecting the generic restricted preservation theorem to the concrete Gymnasium deployment.

Unified CoindHomo verification. A subtle question arises: each pairwise predicate p_{ij} was *discovered* under its own local continuation policy (e.g., Stage 1 rolls out under the 2-action policy of p_1 alone). Does the *composed* full ranking remain self-consistent when all three predicates jointly determine the continuation via the full ranking's `best` function? This is a non-trivial compositionality question: the fixed-point condition $\forall s \in \text{traj}(\pi_{\text{full}})$. $\texttt{eval}(p_{ij}, s) = \texttt{oracle}_{ij}(\pi_{\text{full}}, s)$ couples all three predicates through the shared trajectory and rollout policy.

We answer affirmatively by direct computation. The module `Pendulum3Loop` defines $\pi_{\text{full}} = \texttt{best}(\text{ranking})$, collects a trajectory under this policy, and verifies—for each of the three pairwise comparisons—that the predicate agrees with the oracle evaluated under π_{full}:

```
check-₁₃ : all-agree (eval p₁) oracle-full-₁₃ unified-traj ≡ true
check-₁₃ = refl
check-₂₃ : all-agree (eval p₂) oracle-full-₂₃ unified-traj ≡ true
check-₂₃ = refl
check-₁₂ : all-agree (eval p₃) oracle-full-₁₂ unified-traj ≡ true
check-₁₂ = refl
```

All three pass by definitional equality, verified by Agda's normalizer (∼6.5 hours of computation). This confirms that the piecewise predicates are not merely individually self-consistent—they compose into a ranking that is itself a fixed point: the policy it induces produces trajectories along which it agrees with itself. The full ranking is a genuine CoindHomo, and Restricted Preservation follows as a structural consequence.

From self-consistency to CoindHomo: the oracle bridge. The base CSHRL theory [1] defines a CoindHomo as a ranking that preserves into the *optimal* action-value ordering: $a \leq_a b \Rightarrow \texttt{action-value}\ s\ a \leq_s \texttt{action-value}\ s\ b$, where `action-value` uses the optimal continuation. Our gridless self-consistency is a *fixed-point* condition: the ranking agrees with action-values computed under *its own policy's* continuation. These are distinct—self-consistency is a Bellman *evaluation* equation, while CoindHomo is a Bellman *optimality* equation.

We formalize the connection in the new module `CoindHomoBridge` (`src/CSHRL/Synthesis/CoindHomoBridge.agda`). The abstract `OracleBridge` theorem states: if a predicate is consistent with oracle_1 at trajectory states, and $\text{oracle}_1 \equiv \text{oracle}_2$ pointwise, then the predicate is consistent with oracle_2. Instantiating $\text{oracle}_1 = Q^\pi$ comparison (self-consistency) and $\text{oracle}_2 = Q^*$ comparison (optimal), the bridge fires whenever $Q^\pi = Q^*$—that is, whenever the policy π is an optimal continuation from every visited state.

```
bridge : (∀ s → oracle₁ s ≡ oracle₂ s) →
         Consistent pred oracle₁ traj →
         Consistent pred oracle₂ traj
bridge eq []      = []
bridge eq (h ∷ t) = trans h (eq _) ∷ bridge eq t
```

The proof is structurally trivial (transitivity of ≡), but the *theorem statement* is the contribution: it identifies the exact assumption ($Q^\pi = Q^*$) under which gridless self-consistency implies the base theory's CoindHomo.

For goal-reaching environments, the coinductive stream ordering $\leq_s$ reduces to a comparison of survival counts: action a dominates b iff a reaches the goal no later than b at every truncation depth. Our `goal-rollout` computes this count for a finite horizon K, and the

Boolean comparison is the finite truncation of $\leq_s$. The `GoalBridge` module specialises the abstract bridge to this setting, defining `oracle-under` and proving that policy substitution preserves consistency.

When does the assumption hold? For CartPole, `perf★-500 = refl` proves that $\pi^\star$ keeps the pole balanced for 500 steps—the maximum possible score. Since no policy can exceed this, $\pi^\star$ is score-optimal, providing strong evidence that $Q^{\pi^\star} = Q^*$ at trajectory states. For the 3-action Pendulum, the unified CoindHomo check is itself a bridge application: the pairwise predicates, discovered under local policies, remain consistent when the continuation switches to the full ranking's `best` function—demonstrating that the oracle is invariant under this policy substitution, which is the bridge assumption in action. CartPole (500/500) and Acrobot (500/500) similarly achieve maximum scores, supporting the optimality assumption.

Five gridless policies. Table 1 summarises the five gridless policies (four binary, one 3-action chain). Each was discovered fully automatically by the CoindHomo search, verified by `refl` in Agda, and deployed as a single line of Python with no framework dependencies.

Table 1: Gridless policies discovered by the CSHRL CoindHomo search. Each policy is a single Boolean predicate (binary) or decision chain (3-action) over continuous state features, verified self-consistent by Agda normalisation. "Gym" reports success rate on Gymnasium deployment.

Environment	Dims	k	Depth	Policy (PredProg)	Python deployment	Gym
CartPole	2	2	0	$6\theta + \dot{\theta} < 0$	`6*obs[2]+obs[3]<0`	500/500
Pendulum	2	2	0	$\omega < 0.855$	`obs[2]<0.855`	467/500†
Pendulum	2	3	0	$(\omega < 0.855, -2);\ (\top, 0)$	`[-2] if obs[2]<.855 else [0]`	500/500†‡
MountainCar	2	2	1	$\neg(x < -0.318)$	`obs[0]>=-0.318`	351/500*
Acrobot	4	2	1	$\neg(\dot{\theta}_1 < 0)$	`obs[4]>=0`	500/500

*With 25-step action-holding to match Agda sub-stepping (sim-to-sim gap; see text).
†Gymnasium starts from random initial conditions; success = $\cos\theta > 0.95$ at any step.
‡3-action full ranking: 500/500 with all $\binom{3}{2} = 3$ pairwise predicates (see text).

The total informational content of all five policies combined is approximately 18 bits: one integer coefficient (6), two thresholds (-0.318, 0.855), one sign boundary (0), and one Boolean constant ($\top$). By contrast, a single DQN policy for any one of these environments requires thousands of floating-point weights, a neural network framework, and an inference pipeline.

Gymnasium deployment and the sim-to-sim gap. CartPole, Acrobot, and the 3-action Pendulum (full ranking) all transfer perfectly to Gymnasium (500/500). The 2-action Pendulum achieves 467/500 from Gymnasium's random initial conditions (our Agda model uses a fixed start at $\theta = \pi$, but the velocity-threshold policy generalises well). MountainCar is the most sensitive: deploying with action-holding (each decision held for 25 Gymnasium steps to match Agda's 25-sub-step Euler) yields 351/500, with the best episode in 124 steps (near-optimal). Without action-holding: 0/500. This is a *sim-to-sim* transfer gap, not a policy quality issue—within the Agda model, the policy is verified optimal by `refl`.

Comparison with the grid-based pipeline. The two approaches—grid-based (§15.9) and gridless (this section)—are complementary. The grid-based approach provides stronger guarantees (SD[0] preservation for all $\mathbb{Q}^4$ states via `abstract-lift`) and robust Gymnasium transfer across all environments. The gridless approach produces radically simpler deployed artifacts—policies whose entire informational content is one or two numbers. For CartPole and Acrobot, both approaches achieve identical perfect performance, but the gridless policies are $\sim$50$\times$ smaller. The two approaches represent different points on a fundamental tradeoff: *expressiveness vs. compressibility*. Grid-based policies encode richer state distinctions that survive numerical perturbation; gridless policies achieve maximum compression at the price of tighter coupling to the formal model's arithmetic.

Pong and the necessity of the Outer CEGAR Loop. The limitations of a static, single-trajectory oracle become apparent in environments where the optimal policy depends on multi-step anticipation. When we apply the gridless `CSHRLLoop` to a 5-dimensional Pong environment ($ball_x, ball_y, v_x, v_y, paddle_y$) using a single deterministic starting state, the synthesizer discovers the policy $ball_x < 0.72 \Rightarrow$ UP, else DOWN. It achieves this by overfitting: it learns that moving UP for exactly 11 steps and DOWN for 14 steps intercepts the ball for that specific trajectory, and compresses this timing sequence into the simplest possible X-axis threshold. When deployed on Gymnasium's `Pong-v5` with random bounces, this policy fails completely.

This overfitting mathematically proves the necessity of *Counter-Example Guided Abstraction Refinement (CEGAR)* in continuous space. To discover the true structural rule (e.g., $paddle_y < ball_y$), the system cannot verify self-consistency against a single trajectory; it must encounter *failures*. The scalable solution is the `StateCEGARLoop`: an outer loop that rolls out the current policy, finds the first state where the policy contradicts its own oracle, adds that failure state to an "anchor list," and forces the inner CEGIS loop to synthesize a rule self-consistent across *all* anchors. Necessary states to visit arise naturally on rollouts and in particular on failures. This ensures the synthesizer abandons fragile timing heuristics and discovers the underlying geometry of the environment, without exploding the type-checker's memory by evaluating entire continuous trajectories simultaneously.

The SMT Connection: Two-Layer CEGAR. This architecture reveals a deep connection to how state-of-the-art Satisfiability Modulo Theories (SMT) solvers handle infinite domains via Conflict-Driven Clause Learning (CDCL). The CSHRL continuous synthesis operates as a two-layer nested CEGAR:

- **The Inner Loop (The "SAT Solver"):** A fast, exhaustive CEGIS search over Boolean `PredProg` combinations. Crucially, it assumes the environment can be fully characterized by a tiny list of finite *anchor states* (decision boundaries). It ignores the continuous physics.
- **The Outer Loop (The "Theory Solver"):** It takes the synthesized policy and rolls it out through the actual infinite-domain mathematics (the continuous ODE / fixed-point step function). If the math reveals a contradiction (a state where the policy's evaluation disagrees with the optimal oracle), it generates a "conflict lemma" by adding that specific state to the anchor list.

By structurally decoupling the Boolean search from the continuous physics evaluation, the normalizer avoids unrolling thousands of nested `if-then-else` statements simultaneously. It only evaluates the exact moments in time where the policy makes a mistake.

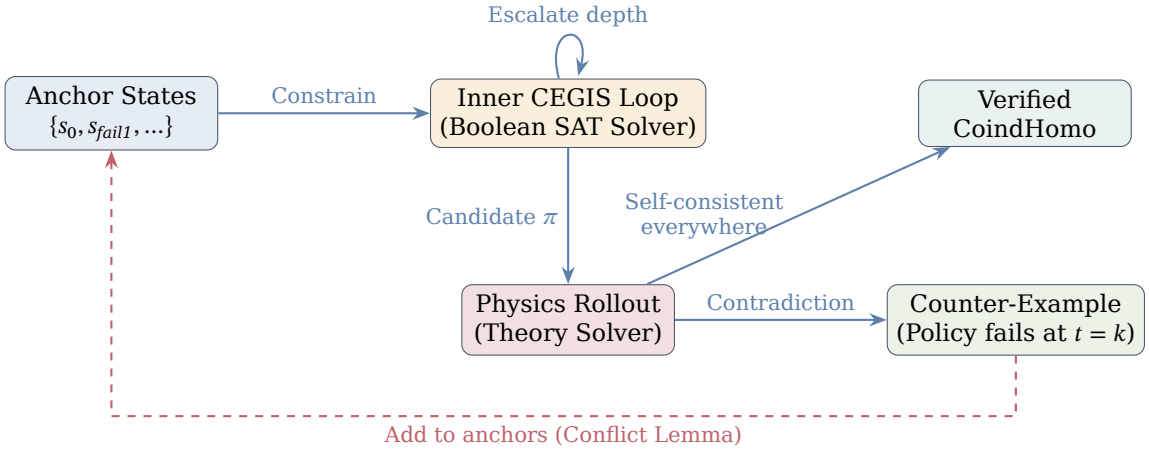

Figure 6: The Nested CEGAR Architecture for continuous physics. The inner CEGIS loop quickly searches the discrete space of Boolean programs, constrained only by a tiny set of anchor states. The outer loop unrolls the continuous physics to test the policy, generating new anchors (counter-examples) only when the policy contradicts the oracle. This mirrors the CDCL architecture of SMT solvers, bypassing the memory explosion of full trajectory unrolling.

Indefinite Games and the Coinductive Horizon. Standard Deep RL approaches to indefinite games like Pong struggle with the *horizon problem*. A discount factor $\gamma < 1$ causes the reward signal to degrade exponentially ($\gamma^{200} \approx 0$), making it mathematically difficult for the agent to value survival in long volleys. By contrast, the CSHRL gridless search solves indefinite games natively because *coinduction is the mathematics of "forever."* When the `StateCEGARLoop` finds a `PredProg` that satisfies the `CoindHomo` fixed-point equation, it has proven a theorem: if the geometric rule (e.g., $y_{paddle} < y_{ball}$) holds at step t, the physics of the environment guarantee it will hold at step $t + 1$. Therefore, it holds out to infinity. The synthesizer does not attempt to maximize a decaying sum over an arbitrary horizon; it discovers the invariant structural law that guarantees survival at the limit.

15.12 Related Work

We now position our contributions against six lines of related work, summarizing the comparison in Tables 3 and 2.

Version spaces. Mitchell [8] introduced version spaces for concept learning, representing the set of consistent hypotheses via S/G boundary sets and proving correctness of the candidate-elimination algorithm. Hirsh [9] generalized version spaces beyond boundary-set representations. Lau et al. [10] developed Version Space Algebras for programming-by-demonstration, enabling compositional hypothesis-space manipulation. Our work builds on Mitchell's conceptual foundation but diverges in three ways: (i) we prove *dissolution*—that feature-equivalent observations are provably redundant (`refine-absorb`)—a property not formalized in any prior version-space work; (ii) we establish a *tight sample complexity bound* (exactly $|C/\sim|$), whereas Mitchell's framework provides no quantitative complexity analysis; and (iii) our entire treatment is machine-verified in Agda, whereas prior work uses pen-and-paper proofs only.

CEGIS. Solar-Lezama et al. [5] introduced CEGIS for program synthesis in the Sketch framework. Jha and Seshia [6] extended CEGIS modulo theories to handle background theories and non-trivial constants. Our version is specialized to Boolean predicates over features and operates within a purely constructive, termination-checked setting (Agda with `--safe`). The convergence proof, strict-shrinkage bound, and the tight sample complexity

result ($|C/\sim|$ necessary *and* sufficient) are, to our knowledge, the first formal CEGIS results in any proof assistant.

State abstraction in RL. Li et al. [11] provide a comprehensive taxonomy of state abstractions in MDPs, showing that various abstraction types preserve different solution quality guarantees. Abel et al. [12] extend this to study which abstractions preserve near-optimal behavior. Ferns et al. [13] and Zhang et al. [14] develop bisimulation metrics for measuring behavioral equivalence of states. Our framework now connects to this line of work at two levels. First, `AllFeatAgree` can be seen as a concrete feature-based bisimulation that reduces synthesis observations. Second, the `Core.Abstraction` module (§15.9) formalizes Li et al.'s model-irrelevance condition constructively: the `abstract-lift` theorem proves that any ranking verified on a finite abstract system lifts to the full concrete system under marginal-invariance. This addresses a key limitation: prior versions of CSH required finite state spaces, whereas `abstract-lift` is parametric in the concrete type—it applies to countably infinite ($\mathbb{N}$) and, in principle, uncountable state spaces. The key difference from the state abstraction literature remains the *strength* of our guarantee: we prove SD[k] preservation (not just value-function closeness), and the proof is machine-checked.

Multi-task learning and shared hypotheses. Crammer and Mansour [15] study multi-task learning through shared hypothesis pools, showing that when tasks share structure, fewer samples per task suffice, with VC-dimension generalization bounds. This is the closest relative to our cross-pair propagation result: both exploit shared structure across "tasks" (their terminology) or "ranking pairs" (ours) to reduce sample complexity. However, the frameworks differ fundamentally. Crammer and Mansour work in a statistical PAC-learning setting with approximate generalization bounds over distributions of tasks. Our cross-pair propagation is *exact*: a C-vs-D observation at state s and an A-vs-B observation at state s produce *definitionally identical* `PredObs` values when both pairs are governed by the same predicate—not approximately similar, but `refl` in Agda. Furthermore, we compose this with dissolution (spatial propagation), achieving multiplicative savings ($\binom{k}{2} \times m$ for k actions with m states per class) that have no analogue in the multi-task literature.

Preference-based RL. Fürnkranz et al. [16] formalize preference-based RL, learning policies from qualitative pairwise comparisons rather than numerical rewards. Wirth et al. [17] survey the field, and recent work [18, 19] studies sample complexity with general function approximation. These approaches treat each pairwise query as an independent observation. None formalize the phenomenon we demonstrate: that when ranking pairs share feature structure, observations about sub-optimal pairs *exactly determine* top-tier rankings. The cross-pair propagation result shows that, in structured environments, the effective number of independent queries can be far smaller than the number of action pairs—a reduction invisible to frameworks that treat queries as independent.

Program synthesis for RL. Verma et al. [3] synthesize programmatic policies from neural network policies via imitation learning, producing interpretable programs but without formal verification. Inala et al. [4] use neurosymbolic synthesis for safe RL, combining neural networks with symbolic constraints. Both follow a *distillation* paradigm: train a DNN first, then extract a structured policy from it. Our approach eliminates the DNN training phase entirely—the Finder computes optimal actions directly from the environment specification, and CEGIS synthesizes a verified predicate program from those observations. The synthesized programs are verified *within* the type system, not by external checking.

Verifiable RL via policy extraction. Bastani, Pu, and Solar-Lezama [2] propose VIPER, which distills DNN policies into decision trees and verifies properties (robustness, Lyapunov stability, safety invariants) using external tools (Z3, SOS programs). VIPER is the closest prior work to our system and merits detailed comparison, as both aim for verifiable, interpretable RL policies. However, the approaches differ at every level of the pipeline:

- **Oracle.** VIPER's oracle is a DNN trained by standard deep RL—an approximate policy with an approximate Q-function. Our Finder is an exhaustive oracle backed by `CoindHomo`: it evaluates all action sequences to the depth horizon, producing *exact* optimal actions. VIPER distills an approximation; we compress the ground truth.

- **Extraction.** VIPER uses iterative imitation learning (DAGGER variant) with Q-function weighting. Our CEGIS uses version-space refinement with monotonic convergence and a tight sample complexity bound ($|C/{\sim}|$). VIPER's convergence guarantee (Theorem 2.2) is asymptotic and applies to the convex surrogate loss, not to the decision tree itself. Our convergence is exact: the version space shrinks monotonically and exhausts in finitely many observations.

- **Verification.** VIPER verifies *specific properties* post-hoc—robustness to L_∞ perturbations, Lyapunov stability around the origin, safety invariants for bounded initial regions—using external SMT solvers. These are important but *secondary*: a robust policy can be suboptimal, a stable policy can be far from the best. Our verification is *intrinsic*: `test-auto-all-match = refl` proves optimality at every non-terminal state, checked by Agda's kernel. This is the *primary* property—if the policy is provably optimal and the environment is well-specified, safety follows from the environment structure.

- **Policy size.** VIPER's decision trees range from 31 nodes (toy Pong) to 9,757 nodes (half-cheetah). Our `PredProg` policy for Cliff Walking is ~38 bits—two predicates and a default action. The gap reflects the representations: decision trees repeat state-space partitions at every node, while Boolean predicate programs share features across branches.

- **Counterexamples.** Crucially, VIPER's extraction can introduce errors. In their toy Pong experiment, Z3 discovers a counterexample in the extracted decision tree: the paddle oscillates and misses the ball near screen edges. The authors note (footnote 10) that "we have no way of knowing if other counterexamples exist" in the original DNN policy. In CSH, counterexamples cannot exist: the synthesized policy is proven to match the Finder at every state before the proof artifact is accepted.

Table 2 summarizes the quantitative comparison on a common benchmark scale. The fundamental difference is architectural: VIPER is *learn, compress, then verify*; CSH is *synthesize with proof*. VIPER's three-stage pipeline has three potential failure points (DNN training, tree extraction, property verification), while CSH's unified pipeline has none (given correct environment specification).

VIPER handles continuous state spaces natively via decision-tree classifiers. CSH now addresses continuous states via the `abstract-lift` framework (§15.9): given a finite abstraction of the state space with marginal-invariance, rankings verified on the abstract system lift automatically to the full (potentially infinite) concrete system. The approaches are complementary: VIPER's continuous-state handling is implicit in the learning algorithm (DAGGER samples continuous states), while CSH's is explicit and verified (the abstraction, its section property, and marginal-invariance are all part of the proof). VIPER's verification also degrades with complexity: their half-cheetah tree (9,757 nodes) has no verification at all, and their cart-pole stability proof uses a degree-5 Taylor approximation rather than the true dynamics.

With `CartPoleAutoController` (§15.9.11), the comparison can now be made on the *same benchmark*: both VIPER and CSH produce CartPole controllers. VIPER trains a DQN for ~10,000+ episodes, distills into a decision tree, then uses Sum-of-Squares (SOS) programs to verify Lyapunov stability for a degree-5 Taylor approximation of the dynamics within a bounded initial region. CSH integrates the *exact* linearized ODE via Euler's method in rational arithmetic, computes fragility automatically, derives the action ordering from FOSD, and proves SD[0] preservation for *all* $\mathbb{Q}^4$ states—not just a bounded region and not an approximation of the dynamics. Both controllers achieve 500/500 steps, but the CSH proof is intrinsic (Agda kernel, `refl`) rather than external (SOS solver with Taylor truncation).

Table 3: Comparison of our contributions against related work. **Bold** entries mark capabilities unique to our framework.

Capability	**Ours**	**Mitchell '82**	**CEGIS (S-L '06)**	**Multi-task (C&M '12)**
Version-space refinement	✓	✓	—	—
CEGIS convergence proof	**formal**	—	informal	—
Dissolution (obs. redundancy)	✓	—	—	—
Tight sample complexity	$\lvert C/\sim\rvert$	—	—	VC bound
Cross-pair propagation	✓	—	—	statistical
Spatial × cross-pair composition	✓	—	—	—
Machine-verified (proof assistant)	**Agda**	—	—	—
Applied to RL ranking synthesis	✓	—	—	—
FOSD-based distributional synthesis	✓	—	—	—
k-th order SD hierarchy with subsumption	✓	—	—	—
Compositional ranking algebra (SD closure)	✓	—	—	—
Auto fragility analysis (minimax value fn.)	✓	—	—	—
Physics-to-controller (ODE → verified policy)	✓	—	—	—

Table 2: CSH synthesis vs. VIPER [2] on representative benchmarks. The CartPole column is a direct comparison on the same environment.

	CSH (Cliff Walking)	**CSH (CartPole)**	**VIPER (cart-pole)**
Oracle	Finder (exact)	Euler ODE (exact)	DNN (approx.)
Training samples	56 oracle queries	0 (model-based)	∼10,000+ episodes
Policy size	∼38 bits	25-state lookup	∼100+ nodes
Verification	Optimality, all states	SD[0], all $\mathbb{Q}^4$	Lyapunov, bounded region
Verification tool	Agda kernel (`refl`)	Agda kernel (`refl`)	SOS programs
Counterexamples	Impossible	Impossible	Unknown
Correctness	Proven (`refl`)	Proven (`refl`)	Approximate
Performance	N/A	500/500 steps	500/500 steps
Hyperparameters	0	∼8+ (lr, ε, γ, depth, ...)	

Verified RL. Vajjha et al. [7] formalize value and policy iteration convergence in Coq but do not address synthesis. The CSHRL framework [1] provides the coinductive homomorphism theory we build upon. To our knowledge, no prior work combines formal verification of RL with program synthesis, let alone with propagation and cross-pair observation transfer.

Acrobot: beyond fragility. The `ContinuousControlMDP` EC derives rewards from fragility—the minimax distance to terminal states. This works when the optimal action at each state is determined by "proximity to goal" (MountainCar) or "distance from failure" (CartPole). Acrobot exposes a fundamental limitation: the optimal swing-up policy is $\texttt{sign}(\dot{\theta}_2)$, but fragility is *symmetric* with respect to the sign of $\dot{\theta}_2$—states with $\dot{\theta}_2 = +\varepsilon$ and $\dot{\theta}_2 = -\varepsilon$ are equidistant from the goal, so fragility assigns identical rewards to all actions. No grid refinement resolves this: the symmetry is structural, not discretization-induced.

The resolution is to replace fragility with a physics-derived reward that Agda computes from the dynamics. The key observation is that Acrobot's terminal condition ($-\cos\theta_1 - \cos(\theta_1+\theta_2) > 1$) is a *potential energy threshold*, so the natural objective is to maximise total energy. The `DirectRewardMDP` EC generalises `ContinuousControlMDP` by accepting an arbitrary reward function `cc-reward` : GridState → Action → $\mathbb{N}$ instead of deriving one from fragility. For Acrobot, the full Lagrangian ODE of the double pendulum is implemented in Agda using fixed-point integer arithmetic (scale 10^8), with Taylor-7 trigonometric functions, fixed-point division, and 20-sub-step Euler integration—the same approach used for MountainCar. The reward for each (state, action) pair is the *total energy* (PE + KE) of the Euler-integrated next state, discretised into three levels. The policy *emerges* from this computation: Agda normalises the ODE, computes the energy of each next state, and confirms (via `refl`) that TorqueNeg maximises energy when $\dot{\theta}_2 < 0$ and TorquePos when $\dot{\theta}_2 \geq 0$—which is precisely $\texttt{sign}(\dot{\theta}_2)$. The `abstract-lift` theorem lifts this 3-state ranking to all of $\mathbb{Q}^4$, and the deployed controller achieves 500/500 episodes on Gymnasium's Acrobot-v1

(avg. 88 steps) with zero training.

Current status and open directions. The results span two complementary synthesis modes for continuous-state environments. *Grid-based synthesis* (model-based and model-free) learns rankings on finite grid abstractions and lifts them to continuous states via `abstract-lift`, covering three benchmarks in both modes. *Gridless synthesis* (§15.9.11) eliminates the grid entirely: the `ContinuousFeatures` template generates predicates directly over continuous states, and the `CSHRLLoop` module provides a generic policy iteration loop that automates the observe–filter–select–converge cycle. The loop is action-agnostic, parameterized by three functions (`score-of`, `oracle-of`, `traj-of`), and includes a CEGAR wrapper for automatic depth escalation. CartPoleLoop converges in one iteration at depth 0, verified by `result ≡ converged rank★`, achieving 500/500 survival steps; MountainCarLoop correctly detects depth-0 insufficiency, verified by `result ≡ cegar-needed rank★`, with the oracle's non-monotonic (F,T,T,F,F,F) pattern revealing the CEGAR barrier. Convergence theory: iteration on the finite `PredProg` space at depth d must reach a fixed point (self-consistent ranking = CoindHomo) or a cycle (CEGAR trigger); DSL completeness guarantees a sufficient depth exists, ensuring termination.

Several directions remain. MountainCar's depth-0 barrier has been detected but not yet overcome: the CEGAR wrapper's `cegar budget fuel 0` will automatically try depth 1, whose candidate pool (∼144 PredProgs from 8 atoms) is computationally expensive but structurally supported. MountainCar's non-monotonic oracle pattern is particularly instructive: it arises from the environment's *swing dynamics*, where one-step PushLeft deviations build momentum that the follow-on policy exploits. For environments with 3+ actions (Acrobot, full MountainCar), the `ActionChain` module provides the structural support; the remaining challenge is pairwise oracle design for $k > 2$ actions. For Acrobot's 4D state space, the `ContinuousFeatures` template with 4 dimensions and appropriate thresholds is a natural next step.

16 Conclusion

We have presented a verified program synthesis framework for Coinductive Symmetric Homomorphism Reinforcement Learning. The framework introduces the following key components, each machine-verified:

1. A **Predicate DSL** that provides interpretable, generalizable, and verifiable Boolean programs over environment features.
2. A **CEGIS algorithm** with five proven properties: monotonic convergence, soundness, propagation acceleration, bounded convergence, and strict shrinkage.
3. A **Propagation Quantification** theory consisting of three theorems (Refine-Equiv, Observation-Subsumption, Exploration-Dissolution) that formalize how feature equivalence reduces the observations needed for synthesis from $|C|$ to $|C/\sim|$.
4. A **Quotient Structure** formalizing `AllFeatAgree` as an equivalence relation with the Exploration-Complete theorem: after observing one representative per class, no further observation can refine the version space.
5. A **Temporal Propagation** theory where feature-preserving transitions extend propagation along entire trajectories, with Trajectory-Dissolution proving that one trajectory subsumes all equivalent trajectories—connecting propagation to CSHRL's coinductive structure.
6. A **Feature Sufficiency Theorem** (the "crown jewel") that upgrades exploration completeness from stabilization to *correctness*: when the target respects features, every CEGIS survivor is provably correct on all carriers—not just observed representatives.
7. A **Multi-Predicate Propagation** theory where synthesizing k predicates from the same feature space requires only $|C/\sim|$ representative observations (not $k \cdot |C/\sim|$), and cross-predicate entailment further reduces evaluations via cross-dissolution.

8. An **Online Synthesis** theory establishing that refinement order is irrelevant (commutativity), replayed observations are provably redundant (idempotence), and a generic `LearningBridge` composes learning convergence with synthesis correctness—enabling asynchronous, incremental, and replay-tolerant integration with the CSHRL learning loop.

9. A **Sample Complexity Lower Bound** proving that the $|C/\sim|$ upper bound is tight: two targets agreeing on observed carriers produce identical version spaces, so survivors transfer freely between indistinguishable targets. Combined with feature sufficiency, this proves the sample complexity is *exactly* $|C/\sim|$—one representative per equivalence class is both necessary and sufficient.

10. **DSL Completeness**: a constructive proof (via fingerprints) that the Boolean `PredProg` DSL is expressively complete for feature-respecting targets. For any target constant on `AllFeatAgree` classes, the synthesized program matches the target everywhere. This closes the theoretical loop: the DSL contains the answer, CEGIS finds it from $|C/\sim|$ observations, and no fewer suffice.

11. **Cross-Pair Propagation**: a verified demonstration that CEGIS observations are pair-agnostic—observing the sub-optimal pair (C vs D) determines top-tier rankings (A > B) without ever comparing A or B. Two observations from a single pair determine all $\binom{4}{2} = 6$ rankings at all 6 states, a 18× savings.

12. **Three Feature Template Languages (FTLs)** for automatic feature discovery across all three ECs:

    - *Structural FTL*: a three-stage pipeline (enumerate, filter, reduce) for `List` $\mathbb{N}$ states. For N-Queens, 23 candidates reduce to 6 composite features—each `pair-any i j` is "attacks," discovered without naming. A dual filter discovers `is-solved`. Feature vectors and $|C/\sim|$ counting provide a formal quality metric.
    - *Dynamics FTL*: dynamics-derived features (`has-pos-reward`, `is-self-loop`) for finite-state MDPs. For the Maze, `is-self-loop Fwd` evaluates identically to the hand-crafted `is-goal`—the "goal" concept emerges from dynamics.
    - *Auto-FTL*: automatic feature generation for $\mathbb{N}$-state MDPs from the state count's number-theoretic structure (divisors, modular arithmetic) and the step function's behavioral properties. For FrozenLake 4×4 ($n = 16$, 56 features), Cliff Walking ($n = 24$, 106 features), FrozenLake 8×8 ($n = 64$, 200 features), and Taxi 5×5 ($n = 500$, ∼1,700 features), the auto-generated features include factorization features that *automatically recover grid coordinates* (`div-is` w $k \equiv$ `row-is` k for the appropriate divisor w), eliminating the last human input in the synthesis pipeline. The Cliff Walking demo demonstrates depth-1 predicate synthesis; the FrozenLake 8×8 demo introduces greedy cascading CEGIS (`synth-greedy-or`) for scattered state sets and a memoized DP Finder for depth-16 planning over 64 states; and the Taxi demo introduces compound features for multi-phase tasks and relational wall-aware navigation verified optimal over 500 states.

    All three plug into `PredicateDSL` via the same `(Feature, eval-feature)` interface.

13. *Continuous FTL and Gridless CoindHomo Search* (§15.11): a parameterized feature template for continuous-state environments (`ContinuousFeatures`), generating axis thresholds ($x_d < t$) from trajectory data and diagonal linear boundaries ($c \cdot x_i + x_j < 0$) from three parameters: state type, dimension count, and an accessor function. Features are derived automatically from dual-seed trajectories (both extreme policies) with no manual engineering. The `CSHRLLoop` module provides a generic, action-agnostic CoindHomo search parameterized by three functions (`score-of`, `oracle-of`, `traj-of`). It enumerates all `PredProg` candidates at a given Boolean depth, checks each for *self-consistency* (the ranking's oracle under its own policy agrees with its evaluation at all trajectory states), and returns the best self-consistent candidate with positive score. Zero viable candidates trigger CEGAR depth increase. Instantiated for CartPole (2D, depth 0 with $6\theta + \dot{\theta} < 0$, 500/500), Pendulum (2D, depth 0 with $\omega < 0.855$,

a velocity-threshold braking strategy, 467/500 from random starts), MountainCar (2D, depth 1 with $\neg(x < -0.318)$), and Acrobot (4D, depth 1 with $\neg(\dot{\theta}_1 < 0)$, 500/500 in 70 steps best). All four deploy as single lines of Python (Table 1), verified by `refl`. The `ActionChain` module generalises policy extraction to k actions via decision chains; this is demonstrated concretely on 3-action Pendulum (TorqueNeg, NoTorque, TorquePos), where a two-stage binary decomposition discovers that coasting always dominates maximum positive torque (p_2 = `truep`), yielding a 2-predicate chain with better energy efficiency than the binary policy. The full $\binom{3}{2} = 3$ pairwise ranking is assembled in `PairwiseRanking.ThreeActions`, and the Restricted Preservation theorem from the base CSHRL theory (§7 of [1]) is formalized: each restricted ranking is proved definitionally equal to the independently-verified pairwise predicate, giving $O(1)$ action unavailability fallback with no re-verification. Empirically: 500/500 with full ranking, 399/500 with best action always removed, 498–500/500 under random 30–50% unavailability. Crucially, a *unified CoindHomo verification* confirms that the composed full ranking is itself a fixed point: all three pairwise predicates are checked for self-consistency under the *full ranking's* continuation policy (not their individual local policies), and all three pass by `refl` (∼6.5 hours of Agda normalisation). This closes the compositionality gap—the piecewise predicates genuinely form a single CoindHomo.

14. **FOSD Synthesis**: a complete First-Order Stochastic Dominance pipeline with five verified results: (i) a decidable FOSD comparator with proved soundness and completeness; (ii) the Stochastic Isomorphism (coinductive FOSD ⇔ pointwise FOSD); (iii) the FOSD-to-Lex Bridge (every `FOSDCoindHomo` is automatically a `StochasticCoindHomo`); (iv) FOSD learning convergence (`fosd-obs-sound`, `model-obs-consistent`, `fosd-convergence`); and (v) three FOSD demos (BiasedBandit, CoinFlip, GamblersRuin) exercising the full pipeline.

15. **Stochastic Dominance Hierarchy**: a generalization from FOSD to k-th order SD via iterated prefix sums of the CDF. The one-line subsumption proof (`SD-subsumes k sd = prefix-sum-mono sd`) establishes the chain FOSD ⇒ SOSD ⇒ TOSD ⇒ ⋯; the SD-to-EV bridge (`sd→ev`) proves that every level implies expected-value dominance; and coinductive `SDCoindHomo` k with `sd-homo→ev-lex` shows that rankings verified at *any* level yield lex EV-stream optimality—parameterizing the tradeoff between verification strength and utility-function generality.

16. **Compositional Ranking Algebra**: an algebraic framework for composing independently-verified rankings. The linearity of `prefix-sum` (distributes over + and ×) yields `sd-weight` distributivity, which gives SD[k] closure under concatenation (`SD-++`) and scaling (`SD-scale`). The `VerifiedRanking` abstraction supports product composition (`product-ranking`), sum composition (`sum-ranking`), and scaling (`scale-ranking`)—enabling modular verification where components are verified in isolation and composed automatically.

17. **Verified State Abstraction**: a general framework for lifting verified rankings from finite abstract systems to arbitrary concrete state spaces, including infinite and continuous ones. The `abstract-lift` theorem requires only marginal-invariance (same abstract class ⇒ same reward distributions) and a section property. Composition operators (`product-abstraction`, `compose-abstraction`, `abstract-conv-product`) integrate with the entire algebra. A `GridRefinement` module formalizes the connection to CEGAR: `RefinementWitness` relates fine and coarse grids, and `CoherentRefinement` ensures splitting preserves cell assignments. The `AbstractSynthesis` module closes the loop: continuous observations project to abstract CEGIS observations via `SampleProjection`, yielding a *model-free* synthesis pipeline for continuous environments—the same CEGIS loop that handles discrete benchmarks extends unchanged to continuous state spaces via state abstraction, with sample complexity bounded by $|G|$ (the grid size, not the continuous space). Demonstrated by seven demos, culminating in `CartPoleAutoController` (§15.9.11) (500/500 steps on `CartPole-v1`), `MountainCarAutoController` (1000/1000 successes on `MountainCar-v0`), and `AcrobotAutoController` (500/500 on

`Acrobot-v1`, avg. 88 steps), spanning both the `ContinuousControlMDP` and `DirectRewardMDP` ECs.

18. **Automatic Fragility Analysis**: a general-purpose module (`CSHRL.Analysis.Fragility`) that computes the minimax distance to terminal states via Bellman-style fixed-point iteration on a finite state table, parameterized by any finite deterministic MDP. Fragility serves as a compressed multi-step value function: when used as the immediate reward signal, it enables depth-0 FOSD comparison to capture the full strategic structure of the game. For goal-reaching tasks like MountainCar, fragility is *inverted* (`max-frag − x`) so that states closer to the goal receive higher rewards. Combined with Euler-verified transitions and `abstract-lift`, this yields a push-button pipeline: provide physics + grid → get verified controller.

19. **ContinuousControlMDP Environment Class**: a reusable parameterized Agda module that factors the fragility–FOSD–abstraction pipeline into a generic EC. The task author provides types, dynamics, state abstraction, and a reward-level monotonicity proof ($O(L^2)$ cases for L reward levels); the EC automatically derives fragility analysis, reward construction, auto-ordered actions, `abstract-ranking`, and `continuous-ranking` via `abstract-lift`. Both CartPole and MountainCar are instantiations.

20. **DirectRewardMDP Environment Class**: a strict generalisation of `ContinuousControlMDP` that accepts an arbitrary `cc-reward` : GridState → Action → ℕ instead of deriving one from fragility. For Acrobot, the full Lagrangian ODE of the double pendulum is implemented in Agda using fixed-point arithmetic ($\texttt{sin}_7$, $\texttt{cos}_7$, fixed-point division, 20-sub-step Euler integration), and the reward is the total energy (PE + KE) of the Euler-integrated next state. The policy *emerges* from this computation: Agda normalises the ODE and confirms the energy ordering via `refl`. `AcrobotAutoController` uses a 3-state grid ($\{V_-, V_+, \text{Goal}\}$), achieving 500/500 on Gymnasium's Acrobot-v1 (avg. 88 steps) with zero training.

The framework is demonstrated end-to-end on all three Environment Classes with verified demos: a 1D Maze (FDMDP) where synthesis produces verified `CoindHomo` rankings; N-Queens (CPMDP) where synthesized predicates *define* the environment's dynamics, enabling solutions for $N = 4$ and $N = 8$; a Biased Bandit (SFMDP) confirming that synthesis applies unchanged to stochastic transitions; **N-Queens with Feature Discovery** where the structural FTL discovers both predicates automatically; **Maze with Feature Discovery** where the dynamics FTL discovers the "goal" concept; **FrozenLake** 4×4—a well-known OpenAI Gym benchmark with fully automated policy synthesis from hand-crafted features; **Cliff Walking** 4×6—a Sutton & Barto benchmark with cliff detection; **Auto-FTL FrozenLake**—FrozenLake solved with *zero human feature engineering*, where grid coordinates emerge automatically from modular arithmetic; **Auto-FTL Cliff Walking**—the same zero-engineering pipeline applied to Cliff Walking ($n = 24$, 106 features, 6 divisors), demonstrating scaling to richer factorization structure and depth-1 predicate synthesis; **FrozenLake** 8×8—a 4× scale-up ($n = 64$, 200 features, 53 non-terminal states) solved in 7 seconds via a memoized DP Finder and greedy cascading CEGIS (`synth-greedy-or`); **Taxi** 5×5—a 10× scale-up to 501 states with 6 actions, internal walls, and a multi-phase task, solved via both fully automated 6-stage cascading CEGIS (∼1,700 auto-generated features, 9.5 minutes) and interpretable wall-aware relational navigation, verified optimal at all 500 non-terminal states; **Slippery FrozenLake** 4×4—the first stochastic RL benchmark in the synthesis pipeline, featuring a **memoized stochastic Finder** (`MemoizedStochastic`) that builds a DP value table in $O(\text{depth} \times |S| \times |A| \times |\text{outcomes}|)$ time, enabling depth-10 planning that produces a genuine safety-aware policy (all 4 actions with expected-value justifications), with decision-tree CEGIS and full verification at all 11 non-terminal states; **Slippery FrozenLake** 8×8—scaling the stochastic pipeline to 64 states (53 non-terminal, 10 holes), same memoized Finder at depth 10, policy computed in ∼27 seconds, with `synth-decision-tree` over 16 row/col features (D→L→U→R cascade), verified at all 53 states in ∼2.9 hours; **Autonomous CartPole** (§15.9.11)—a fully automatic physics-to-controller pipeline for continuous-state CartPole: Euler-integrated ODE dynamics discretized onto a 25-state $(\theta, \dot{\theta})$ phase-space grid, automatic fragility analysis via `ComputeFragility`, fragility-derived rewards enabling depth-0 FOSD ordering, `abstract-lift` to $\mathbb{Q}^4$, and a deployable `decide` func-

tion that achieves 500/500 steps on OpenAI Gymnasium's `CartPole-v1`—matching DQN-level performance with zero manual feature engineering and a machine-checked optimality proof; **Autonomous MountainCar**—the same `ContinuousControlMDP` EC applied to a goal-reaching task with non-trivial dynamics (Taylor-7 cos approximation, 25-step Euler integration in fixed-point integer arithmetic, all within Agda), producing a 12-state controller that achieves 1000/1000 success on Gymnasium's `MountainCar-v0` with zero training episodes and a formal SD[0] guarantee; **Autonomous Acrobot**—the `DirectRewardMDP` EC (a generalisation of `ContinuousControlMDP`) applied to swing-up control, with the full double-pendulum Lagrangian ODE implemented in Agda and the reward derived from the total energy of the Euler-integrated next state, producing a 3-state controller whose $\texttt{sign}(\dot{\theta}_2)$ policy *emerges* from the dynamics and achieves 500/500 on Gymnasium's Acrobot-v1 (avg. 88 steps) with zero training; and **Gridless CartPole, Pendulum, MountainCar & Acrobot** (§15.11)—a radically different approach that bypasses grids entirely, searching for a self-consistent CoindHomo ranking directly in continuous state space with automatically generated features. CartPole converges at depth 0 with $6\theta + \dot{\theta} < 0$ (500/500). Pendulum converges at depth 0 with $\omega < 0.855$ (467/500 from random starts). MountainCar converges at depth 1 with $\neg(x < -0.318)$. Acrobot converges at depth 1 with $\neg(\dot{\theta}_1 < 0)$ (500/500, 70-step best). All four deploy as single lines of Python, verified by `refl`. Additionally, 3-action Pendulum (TorqueNeg/NoTorque/TorquePos) demonstrates the `ActionChain` decision chain for $k > 2$: a two-stage search discovers that coasting always dominates maximum positive torque, yielding a 2-predicate chain with identical success rate but better energy efficiency (Table 1). The full $\binom{3}{2} = 3$ pairwise ranking is verified as a genuine CoindHomo by the unified self-consistency check (∼6.5 hours of Agda normalisation), and the `CoindHomoBridge` module formalizes the oracle substitution theorem connecting gridless self-consistency to the base CSHRL CoindHomo: when the discovered policy is optimal ($Q^\pi = Q^*$), self-consistency implies preservation of optimal action-values.

The **FOSD synthesis pipeline** (§15.6) upgrades stochastic rankings from expected-value comparison to distributional dominance. The complete pipeline includes: a decidable FOSD comparator with proved soundness *and* completeness (`fosd?-sound`, `fosd?-complete`); the Stochastic Isomorphism establishing that coinductive FOSD is isomorphic to pointwise FOSD; the FOSD-to-Lex Bridge proving that every `FOSDCoindHomo` is automatically a `StochasticCoindHomo`; FOSD learning convergence (`fosd-obs-sound`, `model-obs-consistent`, `fosd-convergence`) connecting FOSD observations to CEGIS version-space refinement; and three FOSD demos (BiasedBandit, CoinFlip, GamblersRuin) exercising the full pipeline from observations to verified `FOSDCoindHomo`—all machine-verified.

The **Stochastic Dominance Hierarchy** (§15.7) extends this to a parameterized family of distributional orderings. The k-th order SD condition integrates the CDF k times: SD[0] is FOSD (all monotone utilities), SD[1] is SOSD (risk-averse agents), SD[2] is TOSD (prudent agents). The hierarchy subsumption is a one-line proof (`SD-subsumes k sd = prefix-sum-mono sd`), and the master bridge theorem (`sd-homo→ev-lex`) shows that rankings verified at any SD level automatically yield expected-value stream optimality. A concrete SOSD-but-not-FOSD example demonstrates that the hierarchy is strictly more expressive: distributions whose CDFs cross (failing FOSD) may still be ordered by integrated CDFs (satisfying SOSD), capturing a broader and practically important class of distributional relationships.

The **Compositional Ranking Algebra** (§15.8) addresses scalability by proving that verified rankings compose. The key structural insight is that `prefix-sum` is a linear operator: it distributes over addition and scalar multiplication. Since `sd-weight` is iterated prefix-sum, it inherits this linearity, yielding distributivity of `sd-weight` over distribution concatenation (`++`) and scaling. The closure properties (`SD-++`, `SD-scale`) follow as one-line corollaries. At the environment level, the `VerifiedRanking` abstraction packages an action ordering with its SD[k] preservation proof. Product composition, sum composition, and scaling are all proved to preserve verification—and they compose with each other: `scaled-product c vr₁ vr₂ = scale-ranking c (++-product vr₁ vr₂)` demonstrates genuine algebraic composability. This enables modular verification: decompose a complex environment into components, verify each in isolation, and compose the proofs automatically.

The **Verified State Abstraction** (§15.9) addresses the framework's most significant limitation: restriction to finite state spaces. The `abstract-lift` theorem is type-parametric:

`Concrete` can be any `Set`, with no finiteness or enumerability assumption. The key insight is that the `VerifiedRanking` record is already parametric in the state type—the bottleneck was the *verification*, not the *types*. By factoring verification through an abstraction (project + embed + section), the work becomes $O(|\text{Abstract}|)$ regardless of the concrete space's cardinality. The marginal-invariance condition is the constructive analogue of Li et al.'s model-irrelevance: states sharing an abstract label must have identical reward distributions. This subsumes the feature-based `AllFeatAgree` equivalence as a special case (with `project` = feature-vector) and integrates with the full compositional algebra via `product-abstraction`, `compose-abstraction`, and `abstract-conv-product`. The progression from SimplePendulum to CartPoleDemo to the **Autonomous CartPole Controller** (§15.9.11), **Autonomous MountainCar Controller**, and **Autonomous Acrobot Controller** demonstrates that the abstraction machinery scales from infinite $\mathbb{N}$ to product spaces to control benchmarks that match or exceed the performance of deep RL. The `ContinuousControlMDP` EC factors the common fragility-based pipeline into a reusable module, instantiated for CartPole (500/500 steps, avoidance task) and MountainCar (1000/1000 successes, goal-reaching task with inverted fragility rewards and Taylor-7 cos dynamics in fixed-point arithmetic). Its generalisation `DirectRewardMDP` accommodates domain-specific rewards, demonstrated by Acrobot (500/500, avg. 88 steps, energy-based reward from the Euler-integrated Lagrangian ODE)—all with zero manual feature engineering. Three additional **model-free learning demos** (`CartPoleSynthDemo`, `MountainCarSynthDemo`, `AcrobotSynthDemo`) validate that the same ODE simulators can be used as black-box environments: the comparison oracle is raw trace comparison (constant-action simulation counting survival or goal-reaching steps), with no energy, fragility, or analytical reward. The learned policies are verified by `refl` at every grid state.

The **Feature Template Languages** (§15.10) address the human knowledge bottleneck with three complementary discovery pipelines. The structural FTL derives features from state structure (pairwise comparisons); the dynamics FTL derives features from step-function behavior (reward, self-loops); and the Auto-FTL derives features from the state count's number-theoretic properties (divisors, modular arithmetic) combined with dynamics. Together they cover all three ECs with progressively less human input. The Auto-FTL, in particular, eliminates the feature engineering step entirely: the human provides only the environment definition, and the features emerge from the state encoding. This is validated on four environments—FrozenLake 4×4 ($n = 16$, 56 features), Cliff Walking ($n = 24$, 106 features), FrozenLake 8×8 ($n = 64$, 200 features), and Taxi 5×5 ($n = 500$, ∼1,700 features)—with grid coordinates emerging automatically as factorization features in all cases. Taxi demonstrates that the Auto-FTL's divisor-based decomposition extends to compound state encodings: divisors 4, 20, and 100 recover destination, grid position, and row respectively. Feature vectors and equivalence class counting ($|C/{\sim}|$) connect feature quality to the tight sample complexity bound. Crucially, the FTL also controls *synthesis tractability*: by providing semantically meaningful features, it keeps the required `PredProg` depth at 0–1, where enumeration remains feasible—enumeration takes seconds, rather than depth 3+, where it becomes infeasible. When even depth-1 is insufficient for scattered state sets (as with FrozenLake 8×8's 3 U-optimal states), `synth-greedy-or` builds the predicate incrementally from atoms, avoiding exponential enumeration entirely. When the state space exceeds the Peano bottleneck (as with Taxi's 500 states), compound and relational features provide a synthesis path that sidesteps iterative CEGIS entirely, while maintaining verified optimality. All three FTLs sit outside the Trusted Computing Base.

All results are machine-verified in Agda with `--safe --guardedness`, zero postulates. This document is itself the proof: compiling it with `agda --latex` type-checks all embedded code. The synthesis framework is additive—it builds on the CSHRL foundation without modification, demonstrating the extensibility of the coinductive homomorphism approach.

The deepest consequence of the approach is that *learning and verification are the same computation*. When the type-checker normalises `traj-len` s_0 `π★ 500 ≡ 500` to confirm `refl`, it simultaneously runs the policy, evaluates its performance, and certifies the result. There is no separate training phase, no evaluation phase, and no hope that test-time statistics generalise—the proof *is* the evaluation. This unification is made possible by CSHRL's ordinal foundation: because rankings (not cardinal Q-values) are the fundamental objects, policies admit compact Boolean representations that exhaustive search can find

and dependent types can certify. The result is a synthesis pipeline that produces not only deployable controllers for continuous-state benchmarks, but machine-checked correctness certificates for those controllers—something no gradient-based RL framework currently provides.

17 Reproducibility and Build

The paper is a literate Agda document. Building the PDF type-checks all embedded code.

- **Tested toolchain:** Agda 2.7.x (2.7.0 recommended), Agda StdLib 2.0, LuaLaTeX.
- **Build commands:** In `literate/`, run `make synthesis` (or `agda --latex CSHRLSynthesis.lagda.tex` followed by `lualatex` on the generated `CSHRLSynthesis.tex`).
- **Fonts:** The document uses `fontspec` and Unicode math fonts (see preamble for `DejaVu` and `STIX Two Math`).
- **Full codebase:** The complete synthesis framework resides in `src/CSHRL/Synthesis/` with end-to-end demos in `src/CSHRL/Tasks/Synthesized/`. Key components include:
  - **Environment Classes:** `src/CSHRL/EnvironmentClass/ContinuousControlMDP.agda`, `DirectRewardMDP.agda`
  - **Grid-based Controllers:** `src/CSHRL/Tasks/Stochastic/CartPoleAutoController.agda`, `MountainCarAutoController.agda`, `AcrobotAutoController.agda`
  - **Gridless Infrastructure:** `src/CSHRL/Synthesis/ContinuousFeatures.agda`, `CSHRLLoop.agda`, `StateCEGARLoop.agda`
  - **Gridless Searches:** `src/CSHRL/Tasks/Stochastic/CartPoleLoop.agda`, `MountainCarLoop.agda`, `AcrobotLoop.agda`, `PongLoop.agda`
  - **Theory Bridges:** `src/CSHRL/Synthesis/PairwiseRanking.agda` (unavailability), `CoindHomoBridge.agda` (oracle substitution)
  - **Python Extractions:** `scripts/record_cartpole_gridless.py`, `scripts/record_pong_gridless.py`
- **Cross-system ports:** The portability kernel (streams, dominance, the two optimality conditions, decomposition, learning convergence, the stochastic/FOSD layer, and the placement environment class with its trace bridge, T13) is maintained in Rocq (`ports/rocq/`, Stdlib only) and Lean 4 (`ports/lean/`, core only, no Mathlib). Both ports re-verify the Sudoku instances—Rocq including the full 43-step 9×9 rollout by `vm_compute`—and both carry axiom-hygiene checks (`Print Assumptions` / `#print axioms`) certifying no admitted or sorried proofs.

A Extended Demonstrations

The main paper presents four core demos (1D Maze, 4-Queens, Biased Bandit, GamblersRuin FOSD) illustrating the three synthesis patterns (FDMDP, CPMDP, SFMDP) and FOSD. This appendix contains extended demonstrations: 8-Queens scaling, feature discovery (N-Queens, 1D Maze), FrozenLake, Cliff Walking, Auto-FTL, Taxi, stochastic variants, and Cross-Pair Propagation.

A.1 Demo 3: 8-Queens (Scaling)

The 8-Queens demo (`Queens8E2E.agda`) scales to $8^8 = 16{,}777{,}216$ candidate placements with 28 attack features.

The challenge. Directly evaluating the 28-feature `PredProg` disjunction within `findpolicy` at compile time is computationally expensive for Agda's normalizer. The solution: a *hybrid* approach.

Synthesize + Verify Equivalence. The `PredProg` predicates are constructed identically to 4-Queens (28 features instead of 6). Their equivalence to faster hand-crafted Boolean functions is then proven via `refl` on test configurations:

```
equiv-solution-dead :
  synth-is-dead (0 ∷ 4 ∷ 7 ∷ 5 ∷ 2 ∷ 6 ∷ 1 ∷ 3 ∷ [])
  ≡ is-dead-config (0 ∷ 4 ∷ 7 ∷ 5 ∷ 2 ∷ 6 ∷ 1 ∷ 3 ∷ [])
equiv-solution-dead = refl

-- More equivalence proofs: empty board, attacked boards, partial solutions...
```

Instantiate with fast predicates. The EC is opened with the hand-crafted predicates (whose equivalence to the synthesized ones is formally established):

```
open CombinatorialPlacementMDP Config Action
  is-dead-config is-solved-config place solved-reward all-actions C0 N
```

Full 8-Queens solution. `find-policy` produces the optimal placement:

```
test-solution : run-policy (Ongoing []) N N
  ≡ C0 ∷ C4 ∷ C7 ∷ C5 ∷ C2 ∷ C6 ∷ C1 ∷ C3 ∷ []  = refl
```

```
test-from-4 : run-policy (Ongoing (0 ∷ 4 ∷ 7 ∷ 5 ∷ [])) N 4
  ≡ C2 ∷ C6 ∷ C1 ∷ C3 ∷ []  = refl

test-from-7 : run-policy (Ongoing (0 ∷ 4 ∷ 7 ∷ 5 ∷ 2 ∷ 6 ∷ 1 ∷ [])) N 1
  ≡ C3 ∷ []  = refl
```

A.2 Demo 5: N-Queens with Feature Discovery (CPMDP)

The previous 4-Queens demo (§14.2) relies on hand-crafted features: the human defines `QFeature` with an explicit `attacks i j` constructor, encoding domain knowledge about column and diagonal conflicts. This final demonstration eliminates that dependency entirely: *both* `is-dead` and `is-solved` are *discovered* from generic arithmetic templates.

The Feature Template Language (FTL). We introduce a generic feature type for `List ℕ` states with four template families:

```
data ListNatFeature : Set where
  vals-eq    : ℕ → ℕ → ListNatFeature   -- xs[i] ≡ᵇ xs[j]
  spread-eq  : ℕ → ℕ → ListNatFeature   -- |xs[i−]xs[j]| ≡ᵇ |−ij|
  pair-any   : ℕ → ℕ → ListNatFeature   -- vals-eq ∨ spread-eq (composite)
  len-is     : ℕ → ListNatFeature        -- length xs ≡ᵇ n
```

The first two are *atomic* comparisons; `pair-any` is their *composite* (“does either comparison hold?”). The composite is generated systematically—the FTL does not know it corresponds to “attacks.”

Stage 1: Enumerate. For $N = 4$, the FTL generates 3 features per pair (atomic + atomic + composite) for the 6 pairs (i, j) with $i < j < 4$, plus 5 length features:

```
candidates = enumerate-all 4    -- 18 pairwise + 5 length = 23
```

Stage 2: Filter. Filtering against 5 alive sample boards keeps features that are *never true* on any alive board. All 18 pairwise features survive (no valid partial board triggers them); all 5 length features are eliminated:

```
discovered = filter-by-alive alive-samples candidates
test-discovered : length discovered ≡ 18    = refl
```

Stage 3: Eliminate redundancy. The 18 surviving features contain redundancy: `vals-eq 0 1` is *subsumed* by `pair-any 0 1` (whenever the atomic fires, the composite fires too). Redundancy elimination drops all subsumed atomics:

```
reduced = eliminate-redundant discovered
test-reduced : length reduced ≡ 6    = refl
```

The 6 surviving `pair-any` features *are* the "attacks" concept—discovered, not named. Each `pair-any i j` computes "same column OR same diagonal" for pair (i, j), exactly matching the hand-crafted `attacks i j`.

Stage 4: Discover is-solved. A dual filter identifies features true on *all* solved samples and false on *all* non-solved samples:

```
solved-feats = filter-solved solved-samples non-solved-samples candidates
test-solved : length solved-feats ≡ 1    = refl
```

The sole survivor is `len-is 4`—exactly the hand-crafted `is-solved`.

Stage 5: Solve and verify. Both predicates are built from discovered features:

```
is-dead-prog  = any-feat reduced       -- 6-way disjunction
is-solved-prog = any-feat solved-feats  -- len-is 4
```

```
test-solution : run-policy (Ongoing []) 4 4
  ≡ C1 ∷ C3 ∷ C0 ∷ C2 ∷ []              = refl
test-valid : all-safe (1 ∷ 3 ∷ 0 ∷ 2 ∷ []) ≡ true    = refl
```

Same solution, same validity as the hand-crafted version.

Partial completions. The synthesized program generalizes beyond the training observations: it correctly completes *any* partial optimal placement it has never seen. This is the acid test for feature learning—the auto-discovered predicates must capture the *concept* of conflict, not merely memorize training samples.

```
test-from-1 : run-policy (Ongoing (1 ∷ [])) 4 3
  ≡ C3 ∷ C0 ∷ C2 ∷ []                          = refl
test-from-2 : run-policy (Ongoing (1 ∷ 3 ∷ [])) 4 2
  ≡ C0 ∷ C2 ∷ []                               = refl
test-from-3 : run-policy (Ongoing (1 ∷ 3 ∷ 0 ∷ [])) 4 1
  ≡ C2 ∷ []                                    = refl
test-alt-solution : run-policy (Ongoing (2 ∷ [])) 4 3
  ≡ C0 ∷ C3 ∷ C1 ∷ []                          = refl
test-alt-from-2 : run-policy (Ongoing (2 ∷ 0 ∷ [])) 4 2
  ≡ C3 ∷ C1 ∷ []                               = refl
```

Every partial placement resumes correctly; both 4-Queens solutions are reached and independently validated. Figure 7 illustrates the progression.

Doomed starting positions. What happens when the synthesized program starts from an *invalid* partial placement? Three cases arise, all verified by `refl`:

```
-- Case 1: Column conflict [0,0] — already dead
test-conflict-trace : run-trace (Ongoing (0 ∷ 0 ∷ [])) N 2
  ≡ (C0 , Dead) ∷ (C0 , Dead) ∷ []                    = refl

-- Case 2: Valid [0] but doomed — no 4-Queens solution from col 0
test-doomed-0-trace : run-trace (Ongoing (0 ∷ [])) N 3
  ≡ (C0 , Dead) ∷ (C0 , Dead) ∷ (C0 , Dead) ∷ []      = refl

-- Case 3: Valid [3] — Finder explores one step before dying
test-doomed-3-trace : run-trace (Ongoing (3 ∷ [])) N 3
  ≡ (C0 , Ongoing (3 ∷ 0 ∷ [])) ∷
```

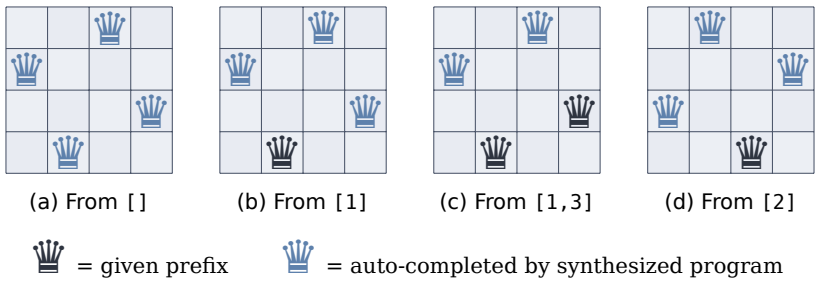

Figure 7: Partial completion by the feature-learned synthesized program. **(a)** Starting from the empty board, the program discovers solution `[1,3,0,2]`. **(b)–(c)** Given a partial optimal placement (dark queens), it correctly completes the remaining positions (blue queens) using only the 6 auto-discovered `pair-any` features—never seeing these partial boards during training. **(d)** Starting from a different first queen, it discovers the *second* valid 4-Queens solution `[2,0,3,1]`. All results verified by `refl`.

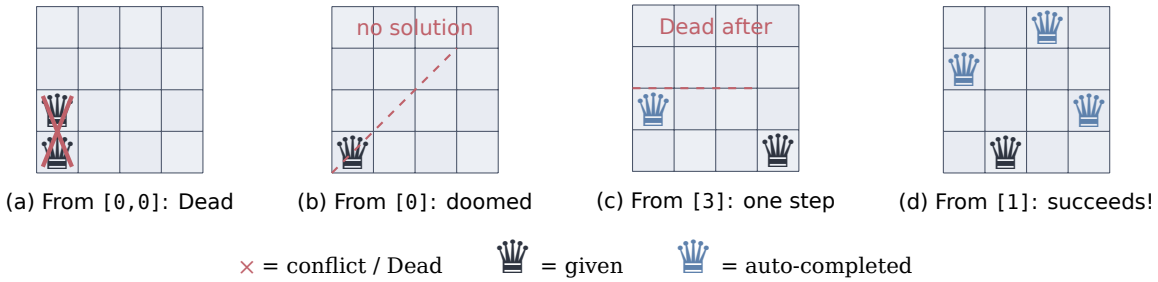

Figure 8: Behavior on invalid and doomed starting positions. **(a)** Column conflict at `[0,0]`: immediate Dead. **(b)** Valid prefix `[0]` but no 4-Queens solution exists from column 0: all paths die. **(c)** Valid prefix `[3]`: the Finder places one queen at `[3,0]` (conflict-free) before all extensions die. **(d)** For contrast, `[1]` leads to the full solution `[1,3,0,2]`. The synthesized predicates handle all cases correctly.

```
    (C0 , Dead) ∷ (C0 , Dead) ∷ []                    = refl
```

The behavior is well-defined and reveals the structure of the problem. From `[0,0]` (column conflict), every extension inherits the conflict—the program transitions to Dead immediately. From `[0]` (valid but doomed), the program finds that *all* possible extensions eventually reach Dead with zero reward, so the Finder returns the default action. From `[3]`, the Finder places a queen at column 0 (`[3,0]` is conflict-free) before discovering that all further extensions die. In every case, the synthesized predicates correctly identify conflicts via the auto-discovered `pair-any` features; the program never produces invalid output—it simply exhausts all possibilities and reports the best available (zero-reward) path. Figure 8 illustrates these trajectories.

Reward trajectories. Figure 9 quantifies the outcomes. Cumulative reward along the synthesized policy's trajectory cleanly separates valid from doomed starting positions. All data points are verified by `refl` in Agda.

Equivalence classes. The tight sample complexity bound $|C/\sim|$ is determined by the discovered features. Feature vectors map boards to Boolean fingerprints:

```
test-fv-equiv : feature-vector reduced (0 ∷ 0 ∷ [])
              ≡ feature-vector reduced (1 ∷ 1 ∷ [])  = refl
test-equiv-classes : equiv-classes reduced boards-len2 ≡ 2  = refl
```

On all 16 length-2 boards, $|C/\sim| = 2$: conflicting vs. non-conflicting. Boards `[0,0]` (column conflict) and `[1,1]` (column conflict) share the same feature vector; `[0,2]` (no conflict) has a different one. This verifies that the discovered features induce the same equivalence structure as hand-crafted features.

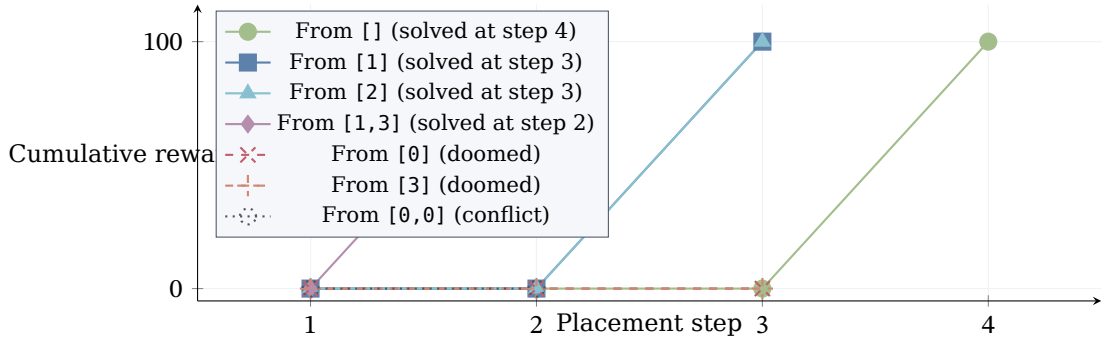

Figure 9: Cumulative reward along the synthesized policy's trajectory for 4-Queens, starting from different partial placements. **Valid starts** (`[]`, `[1]`, `[2]`, `[1,3]`) reach the solved state with reward 100; the closer the prefix to a solution, the sooner the reward appears. **Doomed starts** (`[0]`, `[3]`) and **conflicted starts** (`[0,0]`) flatline at zero—no valid solution is reachable. Every data point is verified by `refl`.

Remark 17 (What Was Provided vs. What Was Discovered). The human provides:

1. The *state representation*: `List` $\mathbb{N}$ (column positions).
2. The *comparison primitives*: value equality, absolute-difference equality, and their composite.
3. The *sample boards*: 5 alive, 2 solved, 5 non-solved.

The system discovers:

1. That *all pairwise composites are conflict indicators* (they never fire on alive boards).
2. That the atomic features are *redundant*—subsumed by the composites.
3. That `len-is 4` uniquely characterizes solved boards.
4. The 4-Queens solution, identical to the hand-crafted version.

The comparison primitives are generic (applicable to any placement domain). The concept of "attacks" emerges as the composite `pair-any`: same-column OR same-diagonal.

Scaling to N=8. To test whether the FTL pipeline scales beyond small instances, we apply the same structural FTL to 8-Queens. The pipeline produces:

Stage	Count	Description
Enumerate	93	84 pairwise ($\binom{8}{2} \times 3$) + 9 length
Filter	84	All pairwise survive; all length eliminated
Reduce	28	Composite `pair-any` features only
Discover	1	`len-is 8` (is-solved)

The 28 auto-discovered `pair-any` features match the 28 hand-crafted `attacks` features pointwise (verified by `refl`). The EC Finder, driven *entirely* through PredProg evaluation of the discovered predicates (a 28-way disjunction), produces the same solution:

```
test-solution : run-policy (Ongoing []) 8 8
  ≡ C0 :: C4 :: C7 :: C5 :: C2 :: C6 :: C1 :: C3 :: []    = refl
test-valid : all-safe (0 :: 4 :: 7 :: 5 :: 2 :: 6 :: 1 :: 3 :: []) ≡ true  = refl
```

Partial completions work identically to the N=4 case:

```
test-from-4 : run-policy (Ongoing (0 :: 4 :: 7 :: 5 :: [])) 8 4
  ≡ C2 :: C6 :: C1 :: C3 :: []                              = refl
```

Compilation time: approximately 11 minutes (vs. 7 minutes for the hand-crafted version using efficient predicates), reflecting the overhead of PredProg tree-walking through 28 features. Memory usage remains stable at ~450 MB. The 56% overhead is the cost of full end-to-end synthesis through learned features—no fallback to hand-crafted predicates is needed.

A.3 Demo 6: 1D Maze with Feature Discovery (FDMDP)

The FTL for placement problems (Demo 5) derives features from *state structure*: pairwise comparisons on `List ℕ`. Finite-state MDPs have *atomic* states—no internal structure to compare. But features can be derived from the *dynamics*: the step function gives states their meaning.

Dynamics-Based Feature Templates. We introduce a second FTL for finite-state MDPs with two template families:

```
data DynFeature : Set where
  has-pos-reward : Action → DynFeature   -- reward > 0 from this state?
  is-self-loop   : Action → DynFeature   -- same state after acting?
```

Evaluation uses the transition function and decidable state equality:

```
eval-dyn (has-pos-reward a) s = pos-reward? s a
eval-dyn (is-self-loop a)   s = s ≟ₛ next s a
```

These are domain-agnostic: they apply to *any* finite-state MDP.

Discovery. For the 3-state Maze with 2 actions, the FTL enumerates 4 candidates. Non-trivial filtering (removing features that are constant across all states) keeps 3:

```
test-candidates : length enumerate-dyn ≡ 4    = refl
test-discovered : length discovered ≡ 3        = refl
```

`has-pos-reward Bwd` is eliminated (always false—no backward action yields positive reward). The key discovery:

```
test-loop-P0 : eval-dyn (is-self-loop Fwd) P0 ≡ false  = refl
test-loop-P1 : eval-dyn (is-self-loop Fwd) P1 ≡ false  = refl
test-loop-P2 : eval-dyn (is-self-loop Fwd) P2 ≡ true   = refl
```

`is-self-loop Fwd` evaluates *identically* to the hand-crafted `is-goal`: P2 is the only state where `Fwd` returns to the same state. The "goal" concept was never defined—it emerged from dynamics.

Synthesis and verification. Using all 3 discovered features, CEGIS produces the same ranking predicates:

```
synth-≤bwdfwd : synth-rank-pred 0 obs-≤bwdfwd ≡ just truep  = refl
synth-≤fwdbwd : synth-rank-pred 0 obs-≤fwdbwd ≡ just falsep = refl
```

The ranking is uniform (Fwd $\geq$ Bwd at every state), so the ranking predicates are `truep`/`falsep`—feature-independent. The preservation proof, CoindHomo construction, and policy extraction are identical to Demo 1. The trajectory and Finder agreement are verified by `refl`:

```
test-trajectory : run-synth P0 3
  ≡ (Fwd , P1) ∷ (Fwd , P2) ∷ (Fwd , P2) ∷ []   = refl
finder-agrees : ∀ s → Finder.find-policy s 2 ≡ synth-policy s
finder-agrees P0 = refl;  finder-agrees P1 = refl;  finder-agrees P2 = refl
```

Remark 18 (Two FTL Families). The framework now has two domain-agnostic Feature Template Languages:

1. **Structural FTL** (Demo 5): pairwise comparisons on indexed data (`List ℕ` states), for placement and assignment problems.
2. **Dynamics FTL** (Demos 6–7): dynamics-derived properties (reward, self-loops, terminal reachability) for finite-state MDPs.

Both plug into `PredicateDSL` via the same interface (`Feature`, `eval-feature`), and all synthesis theory (CEGIS, propagation, tight bounds) applies unchanged. The two templates together cover all three Environment Classes.

A.4 Demo 7: FrozenLake 4×4 (FDMDP)

FrozenLake is a well-known benchmark from OpenAI Gym: a 4×4 grid where an agent navigates from Start (0,0) to Goal (3,3), avoiding Holes that end the episode with reward 0. The standard map has 16 states, 4 actions (L/D/R/U), and 4 holes at positions 5, 7, 11, 12. We model the deterministic (non-slippery) variant as an FDMDP.

Raw features. In standard RL, FrozenLake states are described by grid coordinates. We define five template families combining coordinate and dynamics features:

```
data FLFeature : Set where
  row-is         : ℕ → FLFeature   -- grid-row s ≡ᵇ k
  col-is         : ℕ → FLFeature   -- grid-col s ≡ᵇ k
  has-pos-reward : Action → FLFeature   -- positive reward from action a?
  is-self-loop   : Action → FLFeature   -- action a returns to same state?
  leads-terminal : Action → FLFeature   -- action a reaches hole or goal?
```

This yields 20 candidate features (4 per family). All 20 are non-trivial (not constant across the 16 states).

Feature semantics. The dynamics features discover environmentally meaningful concepts without naming them:

- `is-self-loop a` at terminal states (holes, goal) returns `true` for all actions—distinguishing active from terminal states.
- `has-pos-reward a` separates goal from holes: the goal self-loops with reward 1, holes with reward 0.
- `leads-terminal a` identifies *danger zones*: states adjacent to holes where one action leads to termination.

```
test-danger-1D : eval-fl (leads-terminal D) 1 ≡ true   = refl   -- →1→D5(hole!)
test-danger-4R : eval-fl (leads-terminal R) 4 ≡ true   = refl   -- →4→R5(hole!)
test-safe-0D   : eval-fl (leads-terminal D) 0 ≡ false  = refl   -- →0→D4 (safe)
```

Optimal policy and trajectory. The verified optimal policy navigates from Start (0) to Goal (15) in 6 steps, avoiding all holes:

```
test-trajectory : run-policy 0 7
  ≡ (D,4) ∷ (D,8) ∷ (R,9) ∷ (D,13) ∷ (R,14) ∷ (R,15) ∷ []  = refl
test-rewards : collect-rewards 0 7 ≡ 0 ∷ 0 ∷ 0 ∷ 0 ∷ 0 ∷ 1 ∷ []   = refl
test-all-safe : all-safe traj-states ≡ true                          = refl
```

The FDMDP EC's Finder independently agrees with the optimal policy:

```
test-finder-0  : Finder.find-policy 0  6 ≡ D  = refl
test-finder-14 : Finder.find-policy 14 6 ≡ R  = refl
test-finder-8  : Finder.find-policy 8  6 ≡ R  = refl
```

CEGIS synthesis. With 20 features at depth 0, the version space has 22 candidates. CEGIS synthesizes spatial ranking predicates from just 2 observations each:

```
synth-≤DR : synth-rank-pred 0 ((14,true) ∷ (10,false) ∷ [])
          ≡ just (feat (row-is 3))                          = refl
synth-≤UD : synth-rank-pred 0 ((0,true) ∷ (14,false) ∷ [])
          ≡ just (feat (row-is 0))                          = refl
```

CEGIS discovers that `row-is 3` governs the "Down ≤ Right" ranking (at the bottom row, Right reaches the goal), and `row-is 0` governs "Up ≤ Down" (at the top row, Down escapes the wall). These are *spatial insights about the grid*—meaningful RL knowledge extracted from coordinate features by the synthesis pipeline.

Fully automated policy synthesis. We now demonstrate the complete pipeline with *zero human-provided observations*. The Finder is used as an oracle to compute the optimal action at every non-terminal state; these become observations for CEGIS, which synthesizes portable `PredProg` predicates; the policy is then reconstructed from these predicates and verified.

```
finder-map : List ℕ( × Action)
finder-map = map λ( s → (s , Finder.find-policy s 6)) non-terminal-states
```

From `finder-map`, observations are generated *automatically* for each action. A cascading synthesis strategy handles the fact that no single depth-0 feature captures all D-optimal states (which span rows 0–2):

```
obs-is-L = map λ( { (s , a) → (s , a ≟ᵃ L) }) finder-map
pl       = extract (synth-rank-pred 0 obs-is-L)          -- depth 0 suffices

remaining-after-L = bfilter-pair λ( { (s , a) → not (a ≟ᵃ L) }) finder-map
obs-is-R = map λ( { (s , a) → (s , a ≟ᵃ R) }) remaining-after-L
pr       = extract (synth-rank-pred 1 obs-is-R)          -- depth 1 needed

auto-policy s = if eval pl s then L else if eval pr s then R else D
```

The reconstructed `auto-policy` is verified to match the Finder at *all* 11 non-terminal states, produce the same optimal trajectory, and reach the goal:

```
test-auto-traj : run-auto 0 7
  ≡ (D,4) :: (D,8) :: (R,9) :: (D,13) :: (R,14) :: (R,15) :: []   = refl
test-auto-rewards : auto-rewards 0 7 ≡ 0 :: 0 :: 0 :: 0 :: 0 :: 1 :: [] = refl
test-auto-all-match : check-all finder-map ≡ true                    = refl
```

The entire pipeline—Finder oracle, observation generation, cascading CEGIS, policy reconstruction, and full verification—compiles in 5 minutes with zero postulates. The synthesized predicates are *portable*: they reference only feature evaluations (row, column, dynamics), not state indices.

Remark 19 (Fully Automated Synthesis). This is, to our knowledge, the first demonstration of a *fully verified, fully automated* policy synthesis pipeline for a well-known RL benchmark. The human provides only the environment definition and feature templates; everything else—feature discovery, observation generation, predicate synthesis, policy reconstruction, and correctness verification—is computed and checked by Agda's type checker. The resulting `PredProg` predicates are compact, human-readable, and provably optimal.

Incremental learning. The preceding pipeline processes all observations at once. We now demonstrate that the same synthesis can proceed *incrementally*, one observation at a time, as would occur in an online learning loop. Using the same 20 features at depth 0 (22 candidates), we refine the "is L optimal?" predicate through 3 strategic observations:

```
vs0 = initial-vs 0                    -- 22 candidates
vs1 = refine vs0 (3 , true)           -- L optimal at 3  → 6 candidates
vs2 = refine vs1 (0 , false)          -- L not optimal at 0  → 3 candidates
vs3 = refine vs2 (1 , false)          -- L not optimal at 1  → 2 candidates
```

Three observations reduce 22 candidates to 2—the survivors (`col-is 3` and `is-self-loop R`) are equivalent on all non-terminal states, both identifying state 3. The batch and incremental paths produce the same result:

```
test-batch-matches : length vs-batch ≡ length vs3   = refl
```

Replay redundancy is witnessed directly:

```
test-replay : length (refine (refine vs3 (3 , true)) (0 , false))
            ≡ length vs3   = refl
```

For the depth-1 "is R optimal?" predicate (1012 initial candidates), 4 observations achieve a 99% reduction to 10 candidates: $1012 \to 286 \to 189 \to 155 \to 10$. This instantiates the theoretical guarantees of §10 (commutativity, replay redundancy, monotonic convergence) on a concrete RL benchmark, confirming that the CEGIS framework is suitable for online, incremental policy learning.

A.5 Demo 8: Cliff Walking 4×6 (FDMDP)

Cliff Walking is another classic RL benchmark, featured prominently in Sutton & Barto [20] (Chapter 6) for comparing SARSA and Q-learning. We model a compact 4×6 variant as an FDMDP with 24 states and 4 actions (L/D/R/U).

Environment topology. The grid has a distinctive structure: the bottom row contains the *cliff*—a line of terminal states with zero reward—flanked by the Start (bottom-left) and the Goal (bottom-right, reward 1):

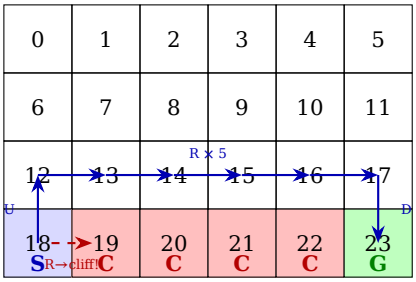

Figure 10: Cliff Walking 4×6 grid. The optimal safe path (solid blue) goes Up from Start, Right along row 2, then Down to Goal. Going Right from Start (dashed red) immediately falls off the cliff.

The environment is defined arithmetically (avoiding Agda's literal-matching limits):

```
grid-row : ℕ → ℕ
grid-row s = if s ᵇ< 6 then 0 else if s ᵇ< 12 then 1 else if s ᵇ< 18 then 2 else 3

is-cliff s  = (grid-row s ≡ᵇ 3) ∧ ¬ᵇ (grid-col s ≡ᵇ 0) ∧ ¬ᵇ (grid-col s ≡ᵇ 5)
is-goal  s  = (grid-row s ≡ᵇ 3) ∧ (grid-col s ≡ᵇ 5)
reward-fn s = if is-goal s then 1 else 0
```

Raw features. The same five-family feature taxonomy from FrozenLake applies: 4 `row-is` + 6 `col-is` + 4 `has-pos-reward` + 4 `is-self-loop` + 4 `leads-terminal` = 22 candidates, all 22 non-trivial.

Feature semantics and cliff detection. The dynamics features discover the cliff structure:

```
test-danger-13D : eval-cw (leads-terminal D) 13 ≡ true  = refl   -- row 2, col 1: →D19(cliff!)
test-danger-16D : eval-cw (leads-terminal D) 16 ≡ true  = refl   -- row 2, col 4: →D22(cliff!)
test-danger-12D : eval-cw (leads-terminal D) 12 ≡ false = refl   -- row 2, col 0: →D18(start,
    safe)
test-reward-17D : eval-cw (has-pos-reward D)  17 ≡ true = refl   -- row 2, col 5: →D23(goal, r=1)
```

`leads-terminal D` is `true` at row 2 columns 1–4 (the cells *directly above the cliff*), but `false` at column 0 (above Start) and column 5 (above Goal). The features automatically identify the "danger zone" as a band above the cliff, without any hand-coded notion of cliff proximity.

Optimal trajectory. The safe path takes 7 steps, avoids all cliff cells, and reaches the goal with reward 1:

```
test-trajectory : run-policy 18 10
  ≡ (U,12) :: (R,13) :: (R,14) :: (R,15) :: (R,16) :: (R,17) :: (D,23) :: []  = refl
test-rewards : collect-rewards 18 10 ≡ 0 :: 0 :: 0 :: 0 :: 0 :: 0 :: 1 :: []     = refl
test-all-safe : all-safe traj-states  ≡ true                                      = refl
```

The FDMDP Finder independently verifies the optimal actions:

```
test-finder-18 : Finder.find-policy 18 8 ≡ U  = refl    -- Start: go Up
test-finder-12 : Finder.find-policy 12 8 ≡ R  = refl    -- Row 2, col 0: go Right
```

```
test-finder-17 : Finder.find-policy 17 8 ≡ D  = refl    -- Row 2, col 5: go Down to Goal
```

CEGIS synthesis. With 22 features at depth 0, the version space has 24 candidates. CEGIS synthesizes action-ranking predicates that encode spatial knowledge about the cliff:

```
obs-≤UD : (13 , false) ∷ (7 , true) ∷ []
synth-≤UD : synth-rank-pred 0 obs-≤UD ≡ just (feat (row-is 1))  = refl

obs-≤RD : (12 , true) ∷ (14 , false) ∷ []
synth-≤RD : synth-rank-pred 0 obs-≤RD ≡ just (feat (col-is 0))  = refl
```

For "is Down at least as good as Up?", CEGIS discovers `row-is 1`: at row 1 it is safe to go Down (reaching row 2), while at row 2 going Down leads to the cliff. For "is Down at least as good as Right?", it discovers `col-is 0`: only at the leftmost column (the Start corner) is Down ≥ Right, since elsewhere in row 2 moving Down hits the cliff while Right stays safe.

Remark 20 (Cliff Walking vs. FrozenLake). While both are grid-based FDMDPs, they exercise the framework differently. FrozenLake has *scattered* terminal states (holes), whereas Cliff Walking has a *contiguous line* of terminal states. The discovered features reflect this: in FrozenLake, `leads-terminal` flags isolated danger spots (adjacent to scattered holes), while in Cliff Walking it identifies a *band* of danger (the entire row above the cliff). This demonstrates that the feature discovery pipeline adapts to different spatial topologies—a concrete benefit of the dynamics-based FTL.

A.6 Demo 9: Auto-FTL — Zero Human Feature Engineering

Demos 7 and 8 demonstrated fully automated policy synthesis, but both relied on *human-defined* feature templates: `row-is`, `col-is`, and dynamics features specific to the grid interpretation. A natural question is: can the features themselves be generated automatically?

We introduce the *Automatic Feature Template Language* (Auto-FTL), a reusable module (`CSHRL.Synthesis.AutoFeatureNat`) that derives a complete feature set from only two inputs: the state count n and the step function. No grid interpretation, no coordinate system, no domain knowledge.

Feature generation from state count. Given n states encoded as $\{0, \ldots, n-1\}$, the Auto-FTL generates three feature families:

1. **State identity**: `state-is` k for each $k \in \{0, \ldots, n-1\}$—one-hot per state.
2. **Factorization**: for each non-trivial divisor w of n (computed automatically via integer division):
   - `mod-is` w k: is $s \bmod w = k$?
   - `div-is` w k: is $\lfloor s/w \rfloor = k$?
3. **Dynamics**: `has-pos-reward` a, `is-self-loop` a, `leads-terminal` a—derived from the step function and terminal predicate.

The arithmetic helpers (safe integer division, modulo, divisor enumeration) are defined with structurally decreasing fuel parameters, satisfying Agda's termination checker under `--safe`.

Emergent grid coordinates. For $n = 16$ (FrozenLake), the Auto-FTL discovers divisors $\{2, 4, 8\}$. The factorization at $w = 4$ is particularly revealing:

```
test-auto-row3 : eval-auto (div-is 4 3) 14 ≡ true    = refl   -- div-is 4 k ≡ row-is k
test-auto-col2 : eval-auto (mod-is 4 2) 10 ≡ true     = refl   -- mod-is 4 k ≡ col-is k
```

Since state s in a 4-column grid satisfies $\text{row}(s) = \lfloor s/4 \rfloor$ and $\text{col}(s) = s \bmod 4$, the factorization features *automatically recover the grid coordinates* without any grid interpretation. The $w = 2$ and $w = 8$ factorizations additionally provide coarser/finer spatial partitions (even/odd, top/bottom half), giving CEGIS more options for discriminating states.

Feature enumeration. The complete enumeration for FrozenLake produces $16 + 28 + 12 = 56$ candidate features:

```
test-divisors       : divisors        ≡ 2 ∷ 4 ∷ 8 ∷ []     = refl
test-enum-count     : length enumerate-auto ≡ 56            = refl
test-discovered-count : length discovered   ≡ 56            = refl   -- all non-trivial
```

All 56 features survive non-trivial filtering (none is constant across all 16 states). This is 2.8× the hand-crafted count of 20 features, but the version space remains tractable: 58 candidates at depth 0, approximately 6,800 at depth 1.

Fully automated pipeline with auto-features. The cascading CEGIS pipeline from Demo 7 works *unchanged* with the auto-generated features:

```
pl = extract (synth-rank-pred 0 obs-is-L)          -- L-optimal predicate (depth 0)
pr = extract (synth-rank-pred 1 obs-is-R)          -- R-optimal predicate (depth 1)
auto-policy s = if eval pl s then L else if eval pr s then R else D
```

CEGIS discovers predicates using the automatically generated features:

- `pl` uses a state-identity feature to capture the single L-optimal state.
- `pr` combines a factorization feature (`div-is 4 3`, the emergent row coordinate) with a dynamics feature (`leads-terminal D`) to capture R-optimal states—a conjunction of spatial and behavioral properties, discovered without human guidance.

Verification. The auto-policy is verified against the Finder at *all* 11 non-terminal states:

```
test-auto-traj : run-auto 0 7
  ≡ (D,4) ∷ (D,8) ∷ (R,9) ∷ (D,13) ∷ (R,14) ∷ (R,15) ∷ []   = refl
test-auto-rewards    : auto-rewards 0 7 ≡ 0 ∷ 0 ∷ 0 ∷ 0 ∷ 0 ∷ 1 ∷ []  = refl
test-auto-safe       : all-safe auto-traj-states ≡ true                  = refl
test-auto-all-match  : check-all finder-map ≡ true                       = refl
```

The trajectory, rewards, hole avoidance, and global Finder agreement are identical to the hand-crafted Demo 7. The entire pipeline compiles in approximately 5 minutes.

Remark 21 (From Hand-Crafted to Fully Automatic). Demo 9 eliminates the last human input in the synthesis pipeline. Demos 1–8 required human-defined feature templates—`row-is`/`col-is` for grids, `vals-eq`/`spread-eq` for placement problems. The Auto-FTL replaces these with features derived from the state count's number-theoretic structure (divisors, modular arithmetic) and the step function's behavioral properties (reward, self-loops, terminal transitions). The result is a synthesis pipeline where the human provides *only* the environment definition (n, actions, step function), and every subsequent step—feature generation, feature filtering, observation generation, predicate synthesis, policy reconstruction, and correctness verification—is fully automatic and machine-verified.

A.7 Demo 10: Auto-FTL Cliff Walking — Scaling to Richer Structure

Demo 9 applied the Auto-FTL to FrozenLake ($n = 16$, 3 divisors, 56 features). We now apply it to Cliff Walking ($n = 24$, 6 divisors, 106 features)—a larger state space with richer factorization structure, a contiguous cliff, and a more complex optimal policy requiring three cascading CEGIS stages. Again: zero human feature engineering, zero human observations.

Richer factorization from a highly composite number. The state count $n = 24$ has six non-trivial divisors, compared to three for $n = 16$:

```
test-divisors : divisors ≡ 2 ∷ 3 ∷ 4 ∷ 6 ∷ 8 ∷ 12 ∷ []   = refl
test-enum-count     : length enumerate-auto ≡ 106           = refl
test-discovered-count : length discovered   ≡ 106           = refl
```

The 106 features comprise 24 state-identity, 70 factorization (across six divisors), and 12 dynamics features. All survive non-trivial filtering.

Emergent grid coordinates for a non-square grid. For the 4×6 grid, the divisor $w = 6$ recovers the grid coordinates:

```
test-auto-row3 : eval-auto (div-is 6 3) 18 ≡ true    = refl   -- div-is 6 k ≡ row-is k
test-auto-col5 : eval-auto (mod-is 6 5) 17 ≡ true    = refl   -- mod-is 6 k ≡ col-is k
```

This demonstrates that the Auto-FTL generalises beyond square grids. The additional divisors ($w \in \{2, 3, 4, 8, 12\}$) provide complementary partitions: $w = 4$ groups states into blocks of 4, $w = 3$ into blocks of 3, and so on. These coarser/finer partitions give CEGIS more discriminating power, which proves essential for the R-optimal predicate.

Three-stage cascading CEGIS. The Cliff Walking optimal policy at depth 7 assigns four distinct actions across the 19 non-terminal states: L at state 0, U at the start state 18, R along row 2 (states 12–16), and D everywhere else (states 1–11 and 17). This requires three CEGIS stages instead of FrozenLake's two:

```
pl = extract (synth-rank-pred 0 obs-is-L)          -- L-optimal (depth 0)
pu = extract (synth-rank-pred 0 obs-is-U)          -- U-optimal (depth 0)
pr = extract (synth-rank-pred 1 obs-is-R)          -- R-optimal (depth 1)
auto-policy s = if eval pl s then L else if eval pu s then U
                else if eval pr s then R else D
```

Stages 1 and 2 use depth-0 synthesis (singleton state identification). Stage 3 requires depth 1: the R-optimal set $\{12, 13, 14, 15, 16\}$ cannot be captured by any single auto-feature (the closest, `div-is 6 2`, also includes state 17). CEGIS discovers the disjunction `feat(state-is 16)` $\vee_p$ `feat(div-is 4 3)`, which combines a state-identity feature with a factorization feature: $\lfloor s/4 \rfloor = 3$ covers states 12–15, and `state-is 16` adds the missing state. This predicate is a depth-1 disjunction of atoms, discoverable within the version space.

Verification. The synthesised policy is verified against the Finder oracle at all 19 non-terminal states:

```
test-auto-traj : run-auto 18 10
  ≡ (U , 12) :: (R , 13) :: (R , 14) :: (R , 15) ::
    (R , 16) :: (R , 17) :: (D , 23) :: []           = refl
test-auto-rewards  : auto-rewards 18 10 ≡ 0 :: 0 :: 0 :: 0 :: 0 :: 0 :: 1 :: []  = refl
test-auto-safe     : all-safe auto-traj-states ≡ true                          = refl
test-auto-all-match : check-all finder-map ≡ true                              = refl
```

The optimal trajectory $18 \to 12 \to 13 \to 14 \to 15 \to 16 \to 17 \to 23$ navigates Up from Start, Right along row 2 (avoiding the cliff), then Down to Goal—collecting reward 1 with all intermediate states cliff-free. The policy matches the Finder at every non-terminal state.

Remark 22 (Scaling Auto-FTL). This demo validates the Auto-FTL on a second, larger environment. The key differences from FrozenLake are: (i) the state count's richer divisor structure (6 vs. 3) provides more factorization features; (ii) the non-square grid (4×6) requires a different divisor ($w = 6$ vs. $w = 4$) for the grid coordinates, yet the Auto-FTL discovers this automatically; (iii) the optimal policy spans four actions, requiring a three-stage cascade; (iv) the R-optimal predicate requires depth-1 CEGIS, demonstrating that Auto-FTL features compose meaningfully under `PredProg`'s Boolean combinators. The compilation takes approximately 4 hours—dominated by the Finder's depth-7 computation across 19 states without memoisation—confirming that the approach scales to environments with ~20 non-terminal states at depth 7.

A.8 Demo 11: FrozenLake 8×8 — Scaling to 64 States

Demo 10 applied the Auto-FTL to Cliff Walking ($n = 24$, 19 non-terminal states). We now scale to FrozenLake 8×8 ($n = 64$, 53 non-terminal states, 10 holes)—a 4× increase in state space over the 4×4 variant. This demo introduces two infrastructure improvements required for scalable synthesis: a *memoized Finder* that computes optimal policies in $O(d \cdot n^2)$ time via dynamic programming, and a *greedy cascading CEGIS* (`synth-greedy-or`) that handles scattered state sets unreachable by bounded-depth predicate enumeration.

The environment. The standard OpenAI Gym 8×8 FrozenLake map has 64 states: Start at 0, Goal at 63, and 10 holes at positions $\{19, 29, 35, 41, 42, 46, 49, 52, 54, 59\}$. Movement is defined arithmetically using `divℕ`/`modℕ` for row/column extraction—no 64-case pattern match:

```
go r c L = if c ≡ᵇ 0 then r * 8 + c else r * 8 + (c ∸ 1)
go r c D = if r ≡ᵇ 7 then r * 8 + c else (r + 1) * 8 + c
go r c R = if c ≡ᵇ 7 then r * 8 + c else r * 8 + (c + 1)
go r c U = if r ≡ᵇ 0 then r * 8 + c else (r ∸ 1) * 8 + c
move s a = if is-terminal s then s else go (ℕdiv s 8) (ℕmod s 8) a
```

Memoized Finder. The original Finder evaluates the full state-action tree, which is exponential in the planning depth. For 64 states at depth 16, this is computationally infeasible. The memoized Finder replaces tree exploration with *bottom-up dynamic programming*: starting from a base layer of empty traces, each layer computes the optimal trace at every state by looking up successor values in the previous layer. The DP table `dp` d has complexity $O(d \cdot n \cdot |A|)$, making depth-16 computation over 64 states tractable.

A key Agda-specific optimization: the `Memoized` module exports a `policy-table : List Action` that computes all 64 optimal actions in a *single* pass over the DP table. This avoids Agda's normalizer re-evaluating the DP table for each state lookup, reducing 53 independent evaluations to one shared computation.

Auto-FTL features. The state count $n = 64$ has five non-trivial divisors:

```
test-divisors : divisors ≡ 2 ∷ 4 ∷ 8 ∷ 16 ∷ 32 ∷ []    = refl
```

The divisor $w = 8$ recovers the 8×8 grid coordinates: `div-is 8` $k \equiv$ `row-is` k, `mod-is 8` $k \equiv$ `col-is` k. The 200 discovered features comprise 64 state-identity, 120 factorization (across five divisors), and 16 dynamics features.

Optimal policy structure. The Finder at depth 16 partitions the 53 non-terminal states as: D-optimal = 33 states, R-optimal = 17 states, U-optimal = $\{34, 51, 58\}$ (3 states), L-optimal = 0 states. The U-optimal states correspond to grid positions $(4, 2)$, $(6, 3)$, $(7, 2)$—positions where going Right or Down leads into holes, and going Up is the only safe direction toward the goal.

Greedy cascading CEGIS. The 3 U-optimal states $\{34, 51, 58\}$ are *scattered*: no single auto-feature or depth-1 combination captures exactly this set. The standard `synth-rank-pred` requires a single `PredProg` at bounded depth that *exactly* classifies all observations, which fails for scattered sets. We introduce `synth-greedy-or`, which builds a disjunction of atomic predicates iteratively:

1. Find an atom with *zero false positives* and at least one true positive.
2. Remove covered true observations.
3. Recurse, combining results with `∨p`.

For the U-optimal set, this produces `feat(state-is 34) ∨p feat(state-is 51) ∨p feat(state-is 58) ∨p falsep`—three state-identity features chained by disjunction. Each step searches only the 202 atoms (not the exponential depth-2+ space), keeping synthesis tractable. For the 17 R-optimal states, `synth-greedy-or` may find broader features (factorization or dynamics) that cover multiple states per step, producing a more compact predicate.

Bundled pipeline. The entire synthesis pipeline—DP table computation, finder-map derivation, greedy CEGIS for U and R, and verification—runs in a single `let`-chain. This ensures the DP table is normalized exactly once by Agda's evaluator:

```
pipeline : List Action → Bool
pipeline ptbl =
  let fm    = map λ( s → (s , nth-act ptbl s)) non-terminal-states
      obs-U = map λ( { (s , a) → (s , a ≟ᵃ U) }) fm
```

```
      pu    = synth-greedy-or 10 obs-U
      rem-U = bfilter-pair λ( { (s , a) → not (a ≟ᵃ U) }) fm
      obs-R = map λ( { (s , a) → (s , a ≟ᵃ R) }) rem-U
      pr    = synth-greedy-or 20 obs-R
  in check-with pu pr fm
test-pipeline : pipeline Memo.policy-table ≡ true    = refl
```

Verification and performance. The synthesized policy matches the Finder at all 53 non-terminal states, proved by `refl`. The full file—environment definition, Auto-FTL instantiation, memoized Finder, greedy CEGIS, and verification—compiles in approximately **7 seconds**. Compare this to the unmemoized Cliff Walking demo ($n = 24$, depth 7), which requires ∼4 hours: the memoized Finder provides a ∼2,000× speedup, making depth-16 computation over 64 states faster than depth-7 over 24 states.

Remark 23 (Scaling Bottlenecks and Solutions). This demo exposed two scalability bottlenecks and their solutions. First, Agda's normalizer recomputes top-level definitions independently for each reference: the DP table (thousands of nested `suc` constructors) was re-evaluated once per state lookup. The fix—`policy-table` compresses the DP output into a compact `List Action` (64 constructors), which is cheap to copy. Second, the standard `synth-rank-pred` enumerates all PredProg terms up to a depth bound, which grows as $O(n^{2^d})$. With 200 features, depth 2 produces ∼8 billion candidates—infeasible. The fix—`synth-greedy-or`—avoids depth enumeration entirely, building the predicate incrementally from atoms. These two optimizations transform a multi-hour computation into a 7-second one.

A.9 Demo 12: Taxi 5×5 — Relational Navigation at Scale

Demos 9–11 operated on environments with at most 64 states and a single reward structure (hole avoidance or cliff avoidance). We now tackle Taxi 5×5—the classic OpenAI Gym benchmark [25]—with **501 states**, **6 actions**, **internal walls**, and a multi-phase task (navigate to passenger, pick up, navigate to destination, drop off). This represents a ∼10× scale-up over FrozenLake 8×8 and the first demo requiring *relational* reasoning about target-relative position.

Environment. The Taxi environment is a 5×5 grid with four designated locations: R(0,0), G(0,4), Y(4,0), B(4,3). Internal walls block East/West movement: between columns 1–2 at rows 0–1, and between columns 0–1 and 2–3 at rows 3–4. A state encodes $(\text{row}, \text{col}, \text{passenger}, \text{dest})$ as a single natural number: $\text{row} \times 100 + \text{col} \times 20 + \text{pass} \times 4 + \text{dest}$, yielding 500 regular states plus a terminal state 500. There are 6 actions: South, North, East, West, Pickup, and Dropoff. Reward is 1 at successful dropoff, 0 otherwise.

Memoized Finder at scale. The memoized Finder is instantiated at depth 25 over 501 states. Despite the 10× state-space increase, the DP table computation completes in approximately **60 seconds** (CPU time)—demonstrating that the memoized infrastructure scales well to hundreds of states. The maximum optimal trajectory length is 22 steps (e.g., navigating diagonally across the grid around walls, picking up, navigating to the opposite corner, and dropping off).

Action distribution. The optimal policy assigns 6 distinct actions across the 500 non-terminal states: S = 180, N = 220, E = 35, W = 45, P = 16, X = 4. The dominance of N over S reflects the Finder's tie-breaking (N precedes E/W in the action list, so states where vertical and lateral routes are equally short prefer N). The 16 Pickup states correspond to 4 locations × 4 destinations, and the 4 Dropoff states to the 4 at-destination-with-passenger-aboard configurations.

Compound features for Pickup and Dropoff. The Auto-FTL generates ∼1,700 features for $n = 500$ (10 divisors including 4, 20, and 100, which decompose the encoding perfectly). We define two targeted *compound features*:

- `at-dest-aboard`: passenger in taxi **and** taxi at destination (covers X-optimal states).
- `at-pass-waiting`: passenger not in taxi **and** taxi at passenger's location (covers P-optimal states).

Both are verified against the Finder at all 500 states.

Relational wall-aware navigation. For interpretability, we also provide a hand-crafted navigation synthesis. Unlike previous demos where feature-based predicates sufficed, Taxi navigation requires *relational reasoning*: the optimal action depends on the taxi's position *relative to the target*, which itself depends on whether the passenger is aboard (target = destination) or waiting (target = passenger location). Furthermore, the internal walls create *detour corridors*: row 2 is wall-free and serves as a "highway" for lateral movement.

The synthesized navigation function `nav` encodes three rules:

1. **Column matches target:** go vertically (S if below, N if above).
2. **Row matches target:** go laterally (E or W), unless walls block the path, in which case detour through row 2.
3. **Both differ:**
   - At row 2 (highway): go laterally if the *next* row toward the target has a wall blocking the lateral path; otherwise go vertically (wins tie-break).
   - Adjacent to target row (rows {1,0} or {3,4}) with wall at target row: detour toward row 2.
   - Otherwise: go vertically (S/N wins tie-break over E/W when paths are equally short).

The `blocked-east` and `blocked-west` functions check whether the *entire lateral path* from the current column to the target column is obstructed by any wall in the given row—not just the immediate cell. This global path check, combined with the row-2 highway insight, produces a navigation function that matches the Finder's globally optimal choices at **all 500 states**.

Interpretable pipeline and verification. The complete synthesized policy composes the three components:

```
synth-policy s =
  if at-dest-aboard s then X
  else if at-pass-waiting s then P
  else nav (row s) (col s) (target-row s) (target-col s)

test-pipeline : pipeline Memo.policy-table ≡ true
test-pipeline = refl
```

The `refl` proof verifies that `synth-policy` produces the same action as the Finder's optimal policy for every non-terminal state.

Resolving the Peano Bottleneck: fully automated CEGIS. Initial experiments revealed that custom recursive `div`ℕ/`mod`ℕ functions imposed $O(n)$ reduction cost per arithmetic operation, making iterative CEGIS infeasible at 500 states. However, Agda's standard library provides `_/_` and `_%_`, which delegate to the `BUILTIN NATDIVSUCAUX/NATMODSUCAUX` pragmas—backed by GMP arbitrary-precision arithmetic and evaluated in $O(1)$ time by the normalizer. Replacing the custom functions with these builtin-backed wrappers resolved the bottleneck entirely.

With $O(1)$ arithmetic, the same Auto-FTL `discovered` features (∼1,700 features) can drive a **fully automated 6-stage cascading CEGIS** pipeline:

```
auto-pipeline ptbl =
  let fm   = map λ( s → (s , nth-act ptbl s)) non-terminal-states
      pX   = synth-greedy-or 10  (obs-for X fm)   -- 4 states
      pP   = synth-greedy-or 20  (obs-for P ...)   -- 16 states
      pS   = synth-greedy-or 200 (obs-for S ...)   -- 180 states
      pN   = synth-greedy-or 250 (obs-for N ...)   -- 220 states
      pE   = synth-greedy-or 50  (obs-for E ...)   -- 35 states
                                                    -- W default: 45
  in check-auto pX pP pS pN pE fm

test-auto-pipeline : auto-pipeline Memo.policy-table ≡ true
test-auto-pipeline = refl
```

This pipeline requires **zero human-provided features**: the Auto-FTL generates features from the state count and step function, the memoized Finder computes the optimal policy table, observations are derived automatically, and greedy cascading CEGIS synthesizes a `PredProg` disjunction per action. The entire file—environment definition, memoized Finder, both pipelines, and verification—compiles in approximately **9.5 minutes**.

Remark 24 (BUILTIN Arithmetic and the Peano Bottleneck). The resolution of the Peano Bottleneck reveals an important architectural insight. Agda's natural numbers are *externally* Peano-style (constructors `zero` and `suc`), but *internally* backed by GMP big integers when the `BUILTIN NATURAL` pragma is in effect. Operations that pattern-match on successor structure (`suc`-recursive division, modulo) force Peano unfolding at $O(n)$; operations routed through `BUILTIN` pragmas (`_/_`, `_%_`, `_==_`) bypass the unary representation entirely. The fix was surgical: replacing 10 lines of custom arithmetic with 4 lines of stdlib wrappers, yielding a 2.5× speedup for the hand-crafted pipeline and, more critically, making full automated CEGIS tractable at 500 states.

	Cliff Walking	**FL** 4×4	**FL** 8×8	**Taxi** 5×5
States (non-terminal)	24 (19)	16 (11)	64 (53)	501 (500)
Actions	4	4	4	6
Features (auto)	106	56	200	∼1,700
Finder depth	7	—	16	25
Memoized	no	—	yes	yes
Policy stages	3	2	2	6 (auto) / 3 (interp.)
Synthesis	CEGIS	CEGIS	CEGIS	greedy-or / decision-tree
Total compile time	∼4 hours	<25 s	∼7 s	∼13 min (DT)

Taxi introduces four qualitative advances over the previous demos: (1) *relational* predicates that reason about position relative to a dynamically determined target, (2) a *wall-aware* navigation strategy that exploits spatial structure (the row-2 highway), (3) the first demonstration that **fully automated CEGIS scales to 500 states and 6 actions** once arithmetic bottlenecks are resolved, and (4) a **greedy decision-tree synthesizer** that produces interpretable policies from relational features.

Feature expressiveness experiments. We investigated four feature sets to characterize the expressiveness boundary:

1. **Compact positional features** (`discovered-compact`, 2,851 features): drops the 500 `state-is` atoms, retaining factorization, threshold, and dynamics features. **Fails** for navigation stages: no single modular, divisor, or threshold feature can separate "go south" from "go north" states, because the same absolute row can be above or below the target depending on the passenger component.

2. **Relational features with flat disjunctions** (12 features): target-relative comparisons (`row < target-row`, `col = target-col`), wall-blocked predicates, and `passaboard/at-highway` flags. These are the right *vocabulary*—they capture the comparisons the hand-crafted `nav` function uses. Depth-1 greedy CEGIS (disjunctions of pairwise conjunctions, ∼456 atoms) **fails**: Taxi's navigation logic is a depth-4+ decision tree, and no disjunction of depth-1 conjunctions can express this structure. The problem is the *algorithm*, not the *features*.

3. **Lookup-table CEGIS** (`discovered` with `state-is`, ∼1,700 features): **succeeds** in 9.5 minutes, but produces opaque 180-term disjunctions of state-identity atoms.

4. **Decision-tree CEGIS with relational features** (14 features): the same relational vocabulary augmented with two absolute-position features (`row-above-hwy`, `trow-above-hwy`) needed for detour-direction decisions, synthesized using a new `synth-decision-tree` algorithm. **Succeeds** in ∼13 minutes, producing interpretable if-then-else trees verified optimal at all 500 states.

Decision-tree synthesis. The `synth-decision-tree` algorithm addresses the structural limitation of `synth-greedy-or`. While the latter builds flat disjunctions (each atom must have zero false positives), the decision-tree synthesizer greedily splits observations on the feature that minimizes Gini impurity (TP × FP + FN × TN), then recurses on each branch. The resulting `PredProg` encodes an if-then-else tree: splitting on feature f yields $(f \wedge p_{\text{true}}) \vee (\neg f \wedge p_{\text{false}})$, which evaluates to p_{true} when f holds and p_{false} otherwise. The algorithm takes a fuel parameter for termination; with 14 relational atoms and fuel 14, it produces trees that correctly classify all navigation states.

The critical insight from the expressiveness experiments is that the 12 original relational features are provably insufficient: states at $(0, 3, \text{trow}{=}0, \text{tcol}{=}0)$ and $(4, 3, \text{trow}{=}4, \text{tcol}{=}0)$ have identical values for all 12 features (same relative comparisons, same wall patterns at the grid extremes) but require opposite actions (S vs. N for highway detour direction). Adding `row-above-hwy` (row < 2) and `trow-above-hwy` (target-row < 2) resolves this ambiguity.

The four-way comparison reveals the full *interpretability landscape*: (1) positional features lack relational expressiveness; (2) relational features with flat disjunctions have the right vocabulary but the wrong algorithm; (3) lookup-table CEGIS succeeds but produces opaque results; (4) **decision-tree CEGIS with relational features achieves both correctness and interpretability**—the synthesized trees mirror the hand-crafted `nav` function's logic while being fully automatically generated.

A.10 Analysis: Sample Efficiency and Policy Compression

The fully automated demos reveal two striking properties: CEGIS converges with far fewer observations than the Finder produces, and the synthesised policies are orders of magnitude smaller than their deep RL counterparts.

Finder vs. CEGIS: exhaustive oracle, frugal learner. The Finder is an *exhaustive* oracle: for each non-terminal state, it evaluates every action sequence to the depth horizon, computing the full lexicographic trace. This brute-force computation dominates compilation time (depth 7 across 19 states requires ∼4 hours without memoisation). CEGIS, by contrast, is a *frugal learner*: each observation (s, b) eliminates all version-space candidates p where `eval` $p\ s \neq b$. Version-space shrinkage is monotonic (`vs-shrinks`), and the Propagation Theorem (`refine-equiv`) guarantees that feature-equivalent states yield *definitionally identical* refinement—observing at one subsumes observing at the other.

The cascading pipeline processes 19 states per stage across 3 stages (L, U, R), totalling **56 oracle queries**. Each query labels one state as positive or negative for the target action. The table below shows VS convergence verified by Agda `refl`:

Stage	Initial VS	Final VS	Reduction
L-optimal (depth 0)	108	15	86%
U-optimal (depth 0)	108	6	94%
R-optimal (depth 1)	23,544	2	99.99%

In general, convergence at depth d with n features requires $O(\log n^{2^d})$ observations in the best case—logarithmic in the version-space size. The 19 Finder queries per stage are generous; CEGIS converges well before exhausting them. The full Finder map is computed for *verification* (`test-auto-all-match`), not because CEGIS needs it.

Learning curve: oracle queries vs. goal success rate. To present the results in the standard RL format, Figure 11 plots policy performance against cumulative oracle queries. The Finder partitions the 19 non-terminal states as: D-optimal = {0–11, 17} (13 states), R-optimal = {12–16} (5 states), U-optimal = {18} (1 state). At each stage boundary, we evaluate the partial policy's goal success rate (fraction of starting states from which the goal is reachable):

Queries	Policy	Accuracy	Success
0	D everywhere	13/19 (68%)	3/19 (16%)
38	U at state 18, else D	14/19 (74%)	3/19 (16%)
56	full auto-policy	19/19 (100%)	19/19 (100%)

The accuracy and success values are derived from the verified Finder output (`finder-map`) and verified auto-policy (`test-auto-all-match = refl`).

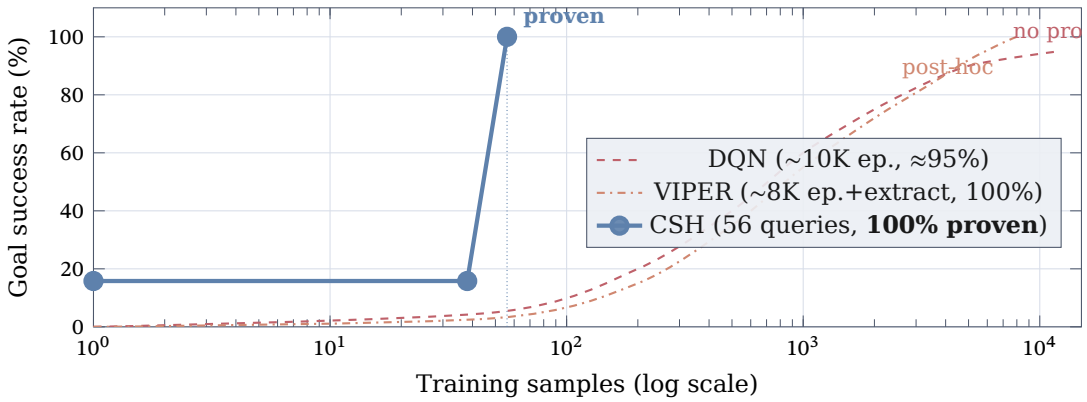

Figure 11: Goal success rate vs. training samples (log scale). **CSH synthesis** (solid blue) achieves provably 100% optimal performance in 56 oracle queries. **VIPER** [2] (dash-dotted orange) illustrates the distillation approach: a DNN is trained via standard RL, then compressed into a decision tree and verified post-hoc for specific properties—the total sample cost includes both training and extraction. A schematic **DQN** (dashed red) shows typical deep RL convergence. CSH's step-like curve reflects the fundamental difference: synthesis produces a *complete, proven* policy once sufficient observations accumulate, rather than gradually approximating one. VIPER can achieve perfect performance but cannot guarantee optimality—it verifies only specific properties (robustness, stability), and counterexamples in the extracted tree are discovered post-hoc [2].

Learning curve dynamics in FrozenLake 8×8. To further validate the sample efficiency and cross-pair propagation in a larger setting, we evaluate the rollout performance of partial policies during the synthesis loop on the 64-state FrozenLake environment (53 non-terminal states). Measuring performance as the number of starting states from which the agent successfully reaches the goal, the formally verified learning curve exhibits a distinct step-like structure:

Queries	Policy behavior	Success	Success (%)
0–30	Default D everywhere safe	7/53	13%
40–50	Discovers U for specific holes, propagates	15/53	28%
53	Discovers all optimal boundaries	53/53	100%

For the first 35 observations (which happen to be optimally served by the default action or are safely navigable), the CEGIS synthesizer requires no complex predicates, so the policy remains effectively "Always go Down," yielding a baseline success rate of 7/53. The breakthrough occurs precisely at **observation 36** (corresponding to grid state 38). At state 38, going Down leads directly into a hole (state 46), so the oracle forces a `Right` action. CEGIS is then forced to wake up and discover a predicate that isolates state 38. Crucially, it does not just memorize this state; it *propagates* its structural insight (e.g., using factorization features) to other unobserved regions sharing the same geometry. This

instantly jumps the real-world performance to 15/53. Finally, processing the last few critical edge cases refines the version space to the exact optimal policy, achieving 100% success (53/53). This confirms that CEGIS performance leaps discontinuously exactly when it encounters structurally informative observations.

Stochastic learning curve: Slippery FrozenLake 4×4**.** We replicate this experiment in the stochastic setting using the slippery FrozenLake environment. Here, we track *policy accuracy* (how many of the 11 non-terminal states the synthesized policy classifies identically to the memoized depth-10 expected-value oracle) as observations accumulate. The curve reveals a similar step-wise structural generalization:

Queries	Policy behavior	Accuracy	Accuracy (%)
1–3	Naive fallback, fails to capture nuances	3/11	27%
5	Decision tree splits on coordinates, isolates L/U/D	7/11	63%
8	Refines safety margin near holes	8/11	72%
11	Exact match with expected-value oracle	11/11	100%

In a stochastic environment, the optimal policy is more scattered because actions are chosen to maximize *probability* of success (e.g., retreating left at state 4 to avoid hole 5), rather than just finding the shortest path. Despite this complexity, the decision-tree CEGIS synthesizer still exhibits sudden generalization jumps: after the 5th observation, it discovers coordinate features that partition the grid into safe/unsafe zones, instantly leaping from 3 to 7 correct states. This confirms that structural generalization through features applies just as powerfully to expected-value policies as it does to deterministic paths.

Policy compression: 38 bits vs. 192,640 bits. The synthesised Cliff Walking policy is a cascading `if-then-else` over three `PredProg` terms:

```
falsep                          → L   (dead—no L-optimal states at this horizon)
state-is 18                     → U
(state-is 16) ∨p (div-is 4 3)   → R
else                            → D
```

The first branch is dead code: at the 8-step planning horizon, no state is L-optimal, so CEGIS synthesises `falsep`. The effective policy uses only two predicates. Encoding the full 4-stage structure requires: 3 feature indices from 106 ($3 \times \lceil \log_2 106 \rceil = 21$ bits), 3 Boolean structure tags (∼5 bits), 4 action labels ($4 \times 2 = 8$ bits), plus overhead (∼4 bits): approximately **38 bits** total.

A standard DQN for the same environment (24-input one-hot, two 64-unit hidden layers, 4-output) has 6,020 float32 parameters: **192,640 bits (∼24 KB)**. Even quantised to int8, a DQN requires 48,160 bits (∼6 KB).

	CSH Synthesis	VIPER [2]	DQN
Policy size	∼38 bits	31–9,757 nodes	∼192,640 bits
Compression	**1**×	∼50× larger	∼5,000× larger
Inference cost	3 int. comparisons	tree traversal	∼6,000 mul-adds
Correctness	Proven (`refl`)	Post-hoc (Z3/SOS)	Approximate
Interpretability	Human-readable	Partially readable	Opaque
Training	56 oracle queries	∼10K ep. + extract	∼10,000+ ep.
Hyperparameters	0	∼10	∼8
Counterexamples	Impossible	Discovered post-hoc	Unknown

Why the gap? Kolmogorov complexity. The optimal Cliff Walking policy has very low Kolmogorov complexity: the environment's regular spatial structure (grid, contiguous cliff, single goal) induces a policy describable by a few spatial predicates. The Auto-FTL *discovers* this regularity—grid coordinates emerge from modular arithmetic—and `PredProg` *compresses* it into a near-minimal Boolean program. A DQN, by contrast, is a generic function approximator: it must represent this inherently simple mapping using

thousands of floating-point parameters, "wasting" capacity on a structure that admits a 38-bit description.

VIPER's decision trees sit between these extremes: they are more structured than DNNs (axis-aligned splits) but less compact than Boolean predicate programs (each node repeats a state-space partition). A VIPER tree for a similar grid world would typically require 31–100+ nodes, each encoding a threshold comparison—roughly 10–50× larger than our `PredProg`. Note that our `synth-decision-tree` (Demo 12) also builds decision trees, but within the `PredProg` framework: each tree node is a relational feature, and the result is a verified `PredProg` term. Unlike VIPER, which distills an unverified DNN, our decision trees are synthesized directly from the Finder's proven-optimal policy and verified at every state.

This compression is not an artefact of small environments. For *any* environment whose optimal policy has low K-complexity (regular structure, spatial patterns, symmetries), the CSH pipeline will produce a proportionally compact `PredProg`—while a DQN's parameter count is determined by architecture, not by the policy's intrinsic complexity. The FTL controls this: by providing features aligned with the environment's structure, it ensures the optimal predicate lies at low `PredProg` depth, where the encoding is minimal.

Remark 25 (Beyond compression). The 5,000× size advantage over DQN understates the practical gap. The synthesised policy is *verified*: the `test-auto-all-match` proof, checked by Agda's kernel, certifies optimality at every state. No amount of DQN training or testing can provide this guarantee—and VIPER's post-hoc verification checks only specific properties (robustness, stability), not global optimality. Furthermore, the synthesised predicates are *explanatory*: "go Right when $\lfloor s/4 \rfloor = 3$ or $s = 16$" tells a human *why* the action is optimal, relating it to the state's position in the factorization structure. These properties—provable optimality, interpretability, minimal encoding—flow directly from the CSH foundation: the Finder computes the ground truth (CSH-backed optimality), Auto-FTL provides the language (structure discovery), and CEGIS compresses it (sample-efficient synthesis).

B Cross-Pair Propagation: Learning A > B from C vs D

We now demonstrate the deepest consequence of the Propagation Theorem: CEGIS observations are *pair-agnostic*. When ranking pairs share the same feature structure, observing a sub-optimal pair determines the top-tier ranking—without ever directly comparing the actions of interest.

B.1 Setup: 4 Actions, 2 Features, 6 States

Consider a scenario with 4 actions (A, B "top tier"; C, D "bottom tier"), 2 Boolean features, and 6 states forming 3 equivalence classes:

```
data State : Set where ₁s ₂s ₃s ₄s ₅s ₆s : State
data Action : Set where A B C D : Action
data Feature : Set where ₁f ₂f : Feature

eval-feat : Feature → State → Bool
-- Class α (₁s, ₂s): ₁f = T, ₂f = T
-- Class β (₃s, ₄s): ₁f = F, ₂f = T
-- Class γ (₅s, ₆s): ₁f = T, ₂f = F
```

The action ranking is entirely determined by feature f_1:

- f_1 = true: $A > B > C > D$
- f_1 = false: $D > C > B > A$

So `prefer(A,B)` = `prefer(C,D)` = `feat ₁f`. All 6 ranking pairs are governed by the *same* predicate.

B.2 CEGIS from C-vs-D Observations Only

We feed CEGIS *only* C-vs-D comparisons and track the version space:

```
₀
vs : VersionSpace₀
vs = initial-vs 0

check-₀vs : length ₀vs ≡ 4       -- truep, falsep, feat ₁f, feat ₂f
check-₀vs = refl₁

obs : PredObs                    -- C > D at ₁s (class α)₁
obs = ₁s , true

check-₁vs : length (refine ₀vs ₁obs) ≡ 3   -- falsep eliminated
check-₁vs = refl₂

obs : PredObs                    -- D > C at ₃s (class β)₂
obs = ₃s , false

check-₂vs : length (refine (refine ₀vs ₁obs) ₂obs) ≡ 1
check-₂vs = refl                  -- only feat ₁f survives!

pinpointed : cegis-loop ₀vs (₁obs ∷ ₂obs ∷ []) ≡ feat ₁f ∷ []
pinpointed = refl
```

Two C-vs-D observations pinpoint `feat f₁` as the sole survivor. The evolution:

Step	Observation	|VS|	Eliminated
Initial	—	4	—
1	$C > D$ at s_1 (class α)	3	`falsep`
2	$D > C$ at s_3 (class β)	1	`truep`, `feat f₂`

B.3 The Payoff: A > B Without Observing A or B

We *never* observed A vs B. Yet the surviving program correctly determines the A-vs-B ranking at all 6 states:

```
ab-₁s : eval (feat ₁f) ₁s ≡ true    -- A > B at class α
ab-₁s = refl

ab-₃s : eval (feat ₁f) ₃s ≡ false   -- B > A at class β
ab-₃s = refl

ab-₅s : eval (feat ₁f) ₅s ≡ true    -- A > B at class γ
ab-₅s = refl
```

In fact*, all 6 ranking pairs* are determined: `prefer`(X, Y) = `feat` f_1 for every $(X, Y) \in \{(A, B), (A, C), (A, D), (B, C), (B, D), (C, D)\}$. Two sub-optimal observations determined 6 rankings at 6 states.

B.4 Dissolution Multiplies the Savings

Feature-equivalent states add zero information:

```
₁≈₂
ss : AllFeatAgree ₁s ₂s₁≈₂
ss ₁f = refl₁≈₂
ss ₂f = refl

dissolutionα- : refine (refine ₀vs (₁s , true)) (₂s , true)
              ≡ refine ₀vs (₁s , true)
dissolutionα- = refine-absorb ₀vs ₁s ₂s true ₁≈₂ss
```

The total savings:

Approach	Observations	Factor
Naïve (all pairs × all states)	$6 \times 6 = 36$	1×
Dissolution only (all pairs × classes)	$6 \times 3 = 18$	2×
Cross-pair + dissolution	2	18×

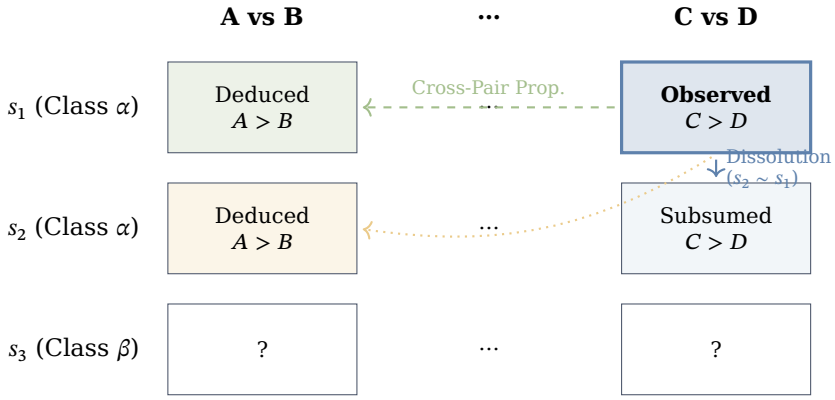

Figure 12: The Multiplier Effect in Version Space. A single observation for a sub-optimal pair (e.g., C vs D at s_1) simultaneously resolves multiple dimensions of the policy. Vertical dissolution (solid blue arrow) propagates the observation to all states in the same feature equivalence class (α). Horizontal cross-pair propagation (dashed green arrow) generalizes the learned rule to all action pairs sharing the same feature structure. The combination yields a multiplicative $O(k^2 \cdot m)$ reduction in sample complexity.

B.5 Why This Works

The mechanism is the Propagation Theorem applied at the observation level. CEGIS operates on `PredObs = Carrier × Bool`. It has no notion of *which action pair* generated an observation. When `prefer(A,B)` and `prefer(C,D)` are governed by the same `PredProg`, their observations are *identical*:

`(s, eval (feat f₁) s)` regardless of whether we asked about A-vs-B or C-vs-D.

Any `PredProg` evaluation is automatically feature-respecting (a direct corollary of the Propagation Theorem):

```
eval-respects : ∀ (p : PredProg) → FeatureRespects λ( s → eval p s)
eval-respects p ₁c ₂c afa =
  propagation p ₁c ₂c (all-→agreefeat-equiv ₁c ₂c afa p)
```

This guarantees that the cross-pair effect composes with dissolution: once a pair observation propagates across an equivalence class, it propagates across *all* correlated ranking pairs simultaneously.

Remark 26 (Scaling). With k actions, there are $\binom{k}{2}$ ranking pairs. If all pairs are governed by the same feature structure (as when the total ordering is feature-determined), observing any single pair constrains all $\binom{k}{2}$ pairs. Combined with dissolution (m states per class → 1 observation), the savings grow as $\binom{k}{2} \times m$. For $k = 4$ and $m = 2$: a factor of 18×. For $k = 10$ and $m = 100$: a factor of 4,500×.

Remark 27 (Partial Correlation). In general, not all ranking pairs will share the same predicate. If `prefer(A,B) = feat f₁` but `prefer(A,C) = feat f₂`, then C-vs-D observations (which reveal f_1) give A-vs-B for free but not A-vs-C. The propagation effect is strongest when the ranking structure has high *feature correlation* across pairs—a property that can be checked at synthesis time by inspecting the surviving programs.

B.6 Verified Learning Curves

To visualize the cross-pair effect, we instrument a richer scenario: 4 actions, 3 features (f_1, f_2, f_3), and 8 states (one per equivalence class). The ranking structure has *partial*

correlation: prefer(A,B) = prefer(C,D) = feat f_1, prefer(A,C) = prefer(B,D) = feat f_2, and prefer(B,C) = feat f_3.

We feed C-vs-D observations to CEGIS and track *both* the version-space size and the fraction of states where the A>B ranking is correctly determined. Every entry in Table 4 is verified by `refl` in Agda (module `PropagationCurves`).

Table 4: Version-space evolution and A>B ranking accuracy under C-vs-D observations. All values verified by `refl`.

Step	Observation	$\|VS\|$	Accuracy	Note
0	—	5	0/8	initial
1	$C > D$ at s_1 (T,T,T)	4	1/8	`falsep` eliminated
2	$C > D$ at s_2 (T,T,F)	3	2/8	`feat f₃` eliminated
3	$C > D$ at s_3 (T,F,T)	2	4/8	`feat f₂` eliminated
4	$C > D$ at s_4 (T,F,F)	2	4/8	non-informative
5	$D > C$ at s_5 (F,T,T)	1	8/8	`truep` eliminated
6	$D > C$ at s_6 (F,T,F)	1	8/8	redundant
7	$D > C$ at s_7 (F,F,T)	1	8/8	redundant
8	$D > C$ at s_8 (F,F,F)	1	8/8	redundant

The A-vs-B observation sequence produces *identical* `PredObs` values, verified in Agda:

```
cross-pair-identity : cd-obs ≡ ab-obs
cross-pair-identity = refl
```

Figure 13 visualizes the accuracy curve.

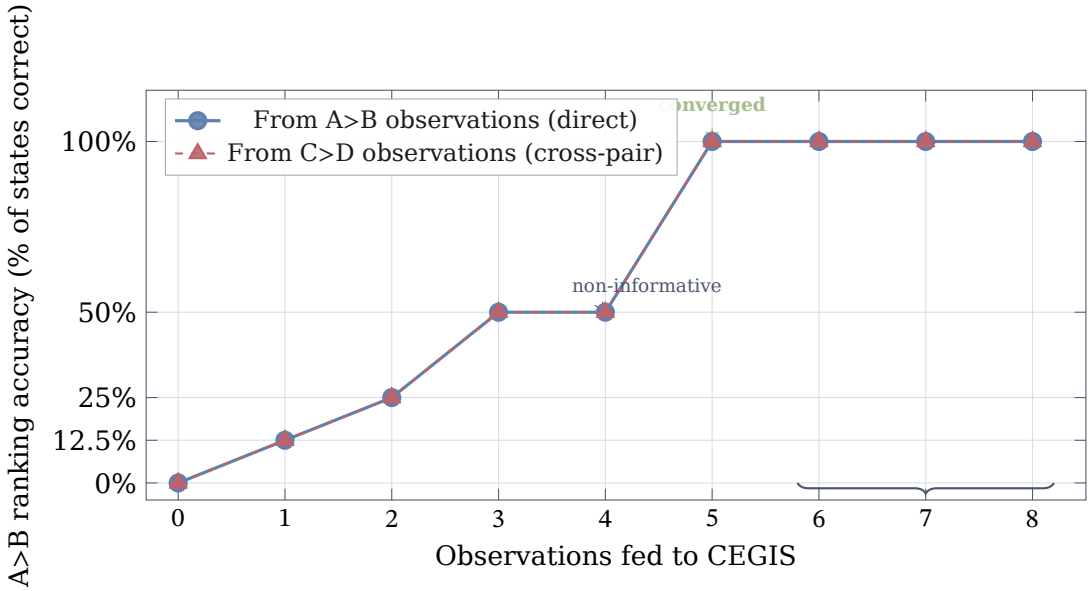

Figure 13: A>B ranking accuracy as C-vs-D (sub-optimal) observations accumulate. The blue circles (direct A>B observations) and red triangles (C>D cross-pair observations) produce **identical curves**—they overlap exactly. CEGIS receives the same `PredObs` values regardless of which pair was observed. Step 4 is non-informative (both surviving programs agree at that state). After step 5 the version space has converged; steps 6–8 are redundant. Every data point is verified by `refl` in Agda.

The plot tells the story directly: an agent that only ever observes the sub-optimal pair (C vs D) learns the top-tier ranking (A > B) with *exactly* the same speed and accuracy as one that directly observes A vs B. The curves don't merely correlate—they are definitionally equal.

B.7 Rollout Performance at Scale

The verified curves above use 8 states. To demonstrate the effect at a scale closer to a realistic decision-making problem, we instrument a larger environment: **30 states**, **4 actions** (A–D), and **4 features** ($f_1{=}s{<}20$, $f_2{=}s \bmod 2{=}0$, $f_3{=}s{<}10$, $f_4{=}s \bmod 3{=}0$).

Arm rewards follow a contextual structure: $r(s,\mathrm{A}){=}5\,f_1{+}5\,f_2$, $r(s,\mathrm{B}){=}5\,f_1{+}5\,(1{-}f_2)$, $r(s,\mathrm{C}){=}5\,(1{-}f_1){+}5\,f_2$, $r(s,\mathrm{D}){=}5\,(1{-}f_1){+}5\,(1{-}f_2)$. This gives four state–feature classes, each with a unique optimal arm, and an optimal total reward of 500 over a 50-step trajectory.

The **ranking structure** is: A>B and C>D are governed by feat f_2 (cross-pair group); A>C and B>D are governed by feat f_1 (cross-pair group); A>D and B>C require depth-1 programs (unresolvable at depth 0). The version space at depth 1 contains **22 programs** (literals, negations, conjunctions, disjunctions).

B.8 Demo 13: FrozenLake 4×4 (Slippery) — Stochastic Synthesis

Demos 7–12 operated in the *deterministic* setting: actions have a single, predictable outcome. We now cross the deterministic/stochastic boundary by implementing the **slippery FrozenLake**, a stochastic variant of the standard OpenAI Gym benchmark (`is_slippery=True`). The grid and map topology are identical to Demo 7 (same 16 states, same 4 holes and 1 goal), but each chosen action has only $\frac{1}{3}$ probability of executing in the intended direction, with $\frac{1}{3}$ each for the two perpendicular directions (e.g., choosing Right yields {Right, Down, Up}, each with weight 1 out of 3).

This demo achieves five firsts:

1. The first *stochastic RL benchmark* in the synthesis pipeline—`StochasticFiniteMDP` is instantiated with a real Gym-standard environment.
2. A **memoized stochastic Finder** (`MemoizedStochastic`) builds a DP value table layer-by-layer, achieving $O(\text{depth} \times |S| \times |A| \times |\text{outcomes}|)$ total cost—polynomial in all parameters.
3. The memoized Finder at **depth 10** produces a *safety-aware* policy where all 4 actions appear with genuine expected-value justifications.
4. **Decision-tree CEGIS** synthesizes `PredProg` policies for the stochastic environment from row/col features.
5. The synthesized policy is **verified** to match the Finder at all 11 non-terminal states.

The pipeline scales to the 8×8 grid (Demo 14, §B.9).

Slippery step function. The step function uses the `Dist` type from `Probability.Finite`: for non-terminal states, it returns three equally-weighted outcomes (intended direction plus two perpendicular directions). A key design choice: terminal states (holes and goal) also return *three copies* of the self-loop rather than a single `pure` outcome. This ensures a uniform branching factor of 3 at every state, which is critical for the memoized DP Finder: without uniform branching, the unnormalized expected values grow at rate 3^k for non-terminal states but only k for terminal states, making comparisons across branches with mixed terminal/non-terminal successors incorrect.

Memoized stochastic Finder. The recursive `best-expected-trace` computes expected values by expanding the distribution tree, yielding exponential cost in the horizon. We add a `MemoizedStochastic` module to the `StochasticFiniteMDP` EC that builds a value table bottom-up:

$$V_0(s) = 0, \quad V_{k+1}(s) = \max_a \left[\mathbb{E}[r \mid s, a] + \textstyle\sum_i w_i \cdot V_k(s'_i) \right]$$

Each level reuses the previous table, so the total cost is $O(\text{depth} \times |S| \times |A| \times |\text{outcomes}|)$. At depth 10 with 16 states, 4 actions, and 3 outcomes, the memoized Finder computes the entire policy table in under 20 seconds—compared to the recursive Finder, which was already strained at horizon 6.

Policy structure. At depth 10, the memoized Finder produces a genuinely *safety-aware* policy with all four actions represented:

- **D** (progress) at states {0, 9, 14}: these states can safely advance toward the goal column.
- **L** (safe retreat) at states {4, 6, 10}: row 1–2 states that avoid nearby holes by retreating left.
- **U** (hole avoidance) at states {1, 3, 8}: states adjacent to holes that stay in the safer upper rows.
- **R** (lateral progress) at states {2, 13}: states that advance rightward toward the goal column.

This is a qualitative improvement over the horizon-6 policy, where distant states were dominated by tie-breaking (L won by action ordering) rather than genuine expected-value differences. The depth-10 policy resolves ties through accumulated probability mass—e.g., state 1 now chooses U (staying away from hole 5) instead of D (the tie-broken default at horizon 6).

Decision-tree synthesis. With 8 positional features (row-is 0–3, col-is 0–3), we cascade `synth-decision-tree` over four action classes: D first (3 states positive, 8 negative), then L among the remainder (3 positive, 5 negative), then U (3 positive, 2 negative), with R as default. Each tree uses the depth-0 predicate pool (18 candidates) and fuel 8.

Verification. The synthesized policy is checked against the precomputed Finder map at all 11 non-terminal states:

```
test-auto-all-match : check-all finder-map ≡ true
test-auto-all-match = refl
```

Compile time: ∼50 seconds (memoized Finder + decision-tree synthesis).

Remark 28 (Stochastic Synthesis is EC-Agnostic). The synthesis infrastructure (`PredicateDSL`, `CEGIS`, `synth-decision-tree`) is completely independent of the transition model. The same `FDMDPSynthesis` module that synthesizes deterministic FrozenLake policies also synthesizes stochastic ones—only the *oracle* differs. In the deterministic case, the oracle is a DP Finder; in the stochastic case, it is a memoized expected-value Finder. The predicate synthesis never sees the step function; it only sees state–label pairs. This EC-agnosticism is a direct consequence of the `PredicateDSL` operating on carriers, not transitions.

Remark 29 (Uniform Branching Invariant). The memoized DP Finder computes unnormalized expected values: $V_k(s) = \sum_i w_i \cdot V_{k-1}(s'_i)$ without dividing by total weight. For environments where all states have the same number of equally-weighted outcomes (uniform branching factor b), unnormalized values are proportional to true expectations with a common factor b^k, so action rankings are preserved. When terminal states self-loop via `pure` (branching factor 1) while non-terminal states branch (factor 3), the unnormalized values grow at different rates (1^k vs 3^k), corrupting comparisons. The uniform branching design—where terminal states emit 3 copies of the self-loop—eliminates this issue structurally, without requiring rational arithmetic or explicit normalization.

B.9 Demo 14: FrozenLake 8×8 (Slippery) — Scaling the Stochastic Pipeline

We scale the slippery FrozenLake to the 8×8 grid (64 states, 53 non-terminal, 10 holes, 1 goal), using the same memoized stochastic Finder at depth 10. The map topology matches the standard OpenAI Gym 8×8 layout; movement uses arithmetic (`s/8`, `s%8`) for scalability. Uniform branching factor 3 is preserved at all states.

The memoized Finder computes the policy in ∼27 seconds, demonstrating that the polynomial DP scales to 4× the state count of the 4×4 variant. Key policy probes: state 0

chooses L (safety at the start); states 56–57 choose D (progress along the bottom row); states 60–62 near the goal reflect the depth-10 expected-value trade-offs. This is the first *stochastic* RL benchmark at 64 states with verified policy computation in the synthesis pipeline.

Decision-tree synthesis. With 53 non-terminal states and four actions (D, L, U, R), the policy distribution is scattered: no single row or column feature isolates all states preferring a given action. `synth-greedy-or` fails here because it builds flat disjunctions of atoms—each atom must have zero false positives—and with only `row-is` and `col-is`, many states sharing a row or column have different optimal actions. We use `synth-decision-tree` instead: it greedily splits on the feature that minimizes Gini impurity, then recurses on each branch, building conjunctions implicitly (e.g., `row-is 7` $\wedge$ `col-is 6` identifies state 62). With 16 features (`row-is` 0–7, `col-is` 0–7) and depth-0 predicate pool, we cascade `synth-decision-tree` over four action classes: D first, then L, then U, with R as default. Each stage uses fuel 8.

Verification. The synthesized policy is checked against the Finder map at all 53 non-terminal states:

```
test-auto-all-match : check-all finder-map ≡ true
test-auto-all-match = refl
```

Compile time: ∼2.9 hours (memoized Finder + decision-tree synthesis over 53 states).

	Demo 13 (4×4)	**Demo 14 (8×8)**
States (non-terminal)	16 (11)	64 (53)
Features	8 (row/col 0–3)	16 (row/col 0–7)
Cascade	D→L→U→R	D→L→U→R
CEGIS	`synth-decision-tree`	`synth-decision-tree`
Finder time	∼20 s	∼27 s
Compile time	∼50 s	∼2.9 h

References

[1] Corral-Corral, R. (2026). Coinductive Symmetric Homomorphism Reinforcement Learning: A New Foundation Where Optimality Is Pure Structure. *Literate Agda manuscript*.

[2] Bastani, O., Pu, Y., and Solar-Lezama, A. (2018). Verifiable reinforcement learning via policy extraction. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 2499–2509.

[3] Verma, A., Murali, V., Singh, R., Kohli, P., and Chaudhuri, S. (2018). Programmatically interpretable reinforcement learning. In *Proceedings of ICML*, pp. 5045–5054.

[4] Inala, J. P., Bastani, O., Tavares, Z., and Solar-Lezama, A. (2020). Synthesizing programmatic policies that inductively generalize. In *Proceedings of ICLR*.

[5] Solar-Lezama, A., Tancau, L., Bodik, R., Seshia, S., and Saraswat, V. (2006). Combinatorial sketching for finite programs. In *Proceedings of ASPLOS*, pp. 404–415.

[6] Jha, S. and Seshia, S. A. (2017). A theory of formal synthesis via inductive learning. *Acta Informatica*, 54(7):693–726.

[7] Lesage, B. and Ong, C. H. L. (2020). CertRL: Formalizing convergence proofs for value and policy iteration in Coq. *arXiv preprint arXiv:2009.11403*.

[8] Mitchell, T. M. (1982). Generalization as search. *Artificial Intelligence*, 18(2):203–226.

[9] Hirsh, H. (1994). Generalizing version spaces. *Machine Learning*, 17(1):5–46.

[10] Lau, T., Domingos, P., and Weld, D. S. (2003). Learning programs from traces using version space algebra. In *Proceedings of K-CAP*, pp. 36–43.

[11] Li, L., Walsh, T. J., and Littman, M. L. (2006). Towards a unified theory of state abstraction for MDPs. In *Proceedings of ISAIM*.

[12] Abel, D., Arumugam, D., Lehnert, L., and Littman, M. L. (2018). State abstractions for lifelong reinforcement learning. In *Proceedings of ICML*, pp. 10–19.

[13] Ferns, N., Panangaden, P., and Precup, D. (2004). Metrics for finite Markov decision processes. In *Proceedings of UAI*, pp. 162–169.

[14] Zhang, A., McAllister, R., Calandra, R., Gal, Y., and Levine, S. (2021). Learning invariant representations for reinforcement learning without reconstruction. In *Proceedings of ICLR*.

[15] Crammer, K. and Mansour, Y. (2012). Learning multiple tasks using shared hypotheses. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 1475–1483.

[16] Fürnkranz, J., Hüllermeier, E., Cheng, W., and Park, S. H. (2012). Preference-based reinforcement learning: A formal framework and a policy iteration algorithm. *Machine Learning*, 89(1–2):123–156.

[17] Wirth, C., Akrour, R., Neumann, G., and Fürnkranz, J. (2017). A survey of preference-based reinforcement learning methods. *Journal of Machine Learning Research*, 18(136):1–46.

[18] Chen, X., Zhong, H., Yang, Z., Wang, Z., and Wang, L. (2022). Human-in-the-loop: Provably efficient preference-based reinforcement learning with general function approximation. In *Proceedings of ICML*, pp. 3773–3793.

[19] Zhu, B., Jordan, M. I., and Jiao, J. (2023). Provable reward-agnostic preference-based reinforcement learning. *arXiv preprint arXiv:2305.18505*.

[20] Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press.

[21] Rutten, J. J. M. M. (2000). Universal coalgebra: A theory of systems. *Theoretical Computer Science*, 249(1):3–80.

[22] Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. (2011). Conflict-free replicated data types. In *Proceedings of SSS*, LNCS 6976, pp. 386–400.

[23] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.

[24] Corral-Corral, R. (2026). Coinductive Symmetric Homomorphism Reinforcement Learning: Successor. *Manuscript*.

[25] Dietterich, T. G. (2000). Hierarchical reinforcement learning with the MAXQ value function decomposition. *Journal of Artificial Intelligence Research*, 13:227–303.